School of Mathematics, Statistics
Te Kura Mātai Tatauranga

PO Box 600
Wellington
New Zealand

Tel: +64 4 463 5341
Internet: office@msor.vuw.ac.nz

Random Forest-Based Synthetic Sample Generation for Class Imbalance Learning in High-Dimensional Gene Expression Data

Corvin IDLER

Supervisor: Dr. Binh Nguyen

Submitted in partial fulfilment of the requirements for
Master of Data Science.

Abstract

Class imbalance is a common challenge in biomedical classification problems, particularly in high-dimensional gene expression datasets where the number of features substantially exceeds the number of samples. Traditional synthetic oversampling methods such as SMOTE rely on Euclidean nearest-neighbor relationships, which may become unreliable in high-dimensional feature spaces. This project proposes RandomForestSMOTE, a synthetic sample generation method that uses Random Forest leaf co-occurrence to define a supervised neighborhood measure and applies feature-masked interpolation to generate minority-class samples.

The proposed method was evaluated on three high-dimensional OpenML gene expression datasets using a nested cross-validation framework. Its performance was compared with standard SMOTE, BorderlineSMOTE, ADASYN, SVMSMOTE, and random oversampling across six classification algorithms: logistic regression, random forest, XGBoost, k-nearest neighbors, linear support vector machine, and naive Bayes. The experimental results show that RandomForestSMOTE achieved the highest mean outer-cross-validation AUROC when averaged across all datasets and classifiers. However, the improvement over the no-oversampling baseline was small, and performance varied across dataset–classifier combinations. These findings suggest that Random Forest-based neighborhood information is a promising direction for synthetic sample generation in high-dimensional data, but the observed gains are modest and require further validation on additional datasets.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr. Binh Nguyen for his guidance and insightful feedback. I am also thankful to Dr. Ryan Admiraal for helping me to navigate the procedural landscape of getting this research project off the ground. I am indebted to my managers Peter Ellis and Phil Bright for their support of my academic endeavors. Finally, I owe a special debt of gratitude to my family (Radostina, Nick-Ryan and Phoebe-Riley) for their love and patience during my studies.

Contents

Figures

Chapter 1

Introduction

Class imbalance is said to be an important problem in the domain of classification[WLD+25] [CYY+24] [WCN23b] [WCN23a]. Class imbalance refers to a situation where one or more classes (commonly referred to as the minority class(es)) have significantly fewer samples than others (the majority class(es)). For example, in a medical diagnosis dataset, one might have 95% "healthy" cases and only 5% "disease" cases, which would constitute a highly imbalanced dataset. It is frequently claimed that class imbalance in the training data for a classifier leads to a predictive bias towards the majority class [WLD+25] [CYY+24] at the expense of the minority class, which in many instances might constitute the more interesting class (e.g. in medical diagnosis or fraud detection). Besides the problem of relative class imbalance between the majority and the minority class, there is frequently the aggravating problem of low absolute numbers of samples from the minority class, which can prevent classifiers from learning the underlying concept of the minority class at all. [BL10] finds that the class imbalance problem (classifier bias) is more accentuated in high-dimensional datasets. So the need for mitigation measures to function in these scenarios is even more pronounced than for low-dimensional datasets.
Gene expression datasets (the data domain chosen for this thesis) exhibit many of the problems outlined above (c.f. Chapter 3.1). Stemming from the health domain, classification of gene expression samples usually revolves around identifying some disease (e.g. cancer), with the number of healthy samples oftentimes far outnumbering the number of disease cases. Depending on the rarity of a disease, these datasets can also suffer from an absolute lack of disease data points. Furthermore, gene expression datasets are oftentimes high-dimensional in the sense that the number of features (more than ten thousand) by far outstrips the number of samples. High dimensionality doesn't just aggravate the class imbalance problem itself for many classifiers, it also poses unique challenges to many existing countermeasures for class imbalance, like synthetic sample generation as done in [CBHK02], which warrants the exploration of alternative methods to deal with class imbalance in the classification of gene expression samples (c.f. Section 2.1.2 and 2.2).
Despite the topic of class imbalance having been well studied (c.f. the surveys in [GXZ+25] [CYY+24] [JK19]) [JTM25] [KPM19]), it is still part of ongoing research and is not as clear-cut as often made out to be. For example, [Kra16] showed that class imbalance is of no or lesser concern if both classes are well represented and their distributions are non-overlapping (and therefore well separable). [JS02] comes to a similar conclusion and finds that with increasing complexity of the datasets and class distribution, class imbalance becomes a concern, but that for simple linearly separable classes, the class imbalance problem is not a major issue. So while class imbalance might not be a universal concern in all circumstances and for all datasets, it is more often than not something one might have to take into account when building classification pipelines, and a great variety of methods exists to address the class

imbalance problem, as outlined in the next section.

1.1 Taxonomy of imbalance learning methods

The approaches to address the class imbalance problem can be broken down into different categories. [GXZ+25] establishes a rather comprehensive and detailed taxonomy as shown in Figure 1.1. The four main categories are:

- data re-balancing
- feature representation
- training strategy
- ensemble learning

The taxonomy used by [GXZ+25] is rather detailed, and other authors (e.g. [WCN23b]) only distinguish data-level, algorithm-level, and ensemble learning, or even simpler data-centric vs. model-centric methods [WCN23b].
The approach we develop in this piece of research (c.f. Section 2.4) falls into the main category of data re-balancing, (linear) synthetic sample generation, more specifically (c.f. Section 1.1.1). The justification for this choice is given in Section 1.2. We explain the main categories of the taxonomy in some more detail in the sections below.

1.1.1 Data re-balancing

Data re-balancing methods are often called data-level methods in taxonomies by other authors (e.g. [WCN23b]) and concern themselves with fixing the imbalance issue in the dataset itself. Famous approaches are oversampling the minority class, undersampling the majority class [GXZ+25], or generating synthetic samples. Generating synthetic samples from the existing samples aims to generate new instances of the minority class that contain the true underlying concepts of the minority class and match the real distribution of the minority class [GXZ+25]. Synthetic sample generation has been a rather popular approach, particularly with regard to the number of academic publications of variations of this method. The most famous example of synthetic sample generation is the Synthetic Minority Oversampling Technique (SMOTE) [CBHK02]. Many new variants have been published since, including Borderline-SMOTE [HWM05] and ADASYN [HG09], which are implemented in the Imbalanced-Learn python package [LNA17]. At the core, these methods interpolate linearly in feature space between a minority sample and a randomly selected second sample from the k closest minority class neighbors [GXZ+25] of the first sample. These methods are algorithmically and conceptually rather simple but often struggle in the context of complex distributions and high dimensions [GXZ+25]. We will revisit these methods in more detail in Section 2.1.

1.1.2 Feature representation

Feature representation methods comprise important approaches like cost-sensitive learning, metric learning, contrastive learning, etc. Cost-sensitive learning assigns different misclassification costs for false negatives and false positives to counteract the biases created by the class imbalance. It is noteworthy that cost-sensitive learning is a well-justified and often needed approach, regardless of the presence of class imbalances, as in many real-world problems, there is an actual difference between the real-world cost of a false positive vs. a

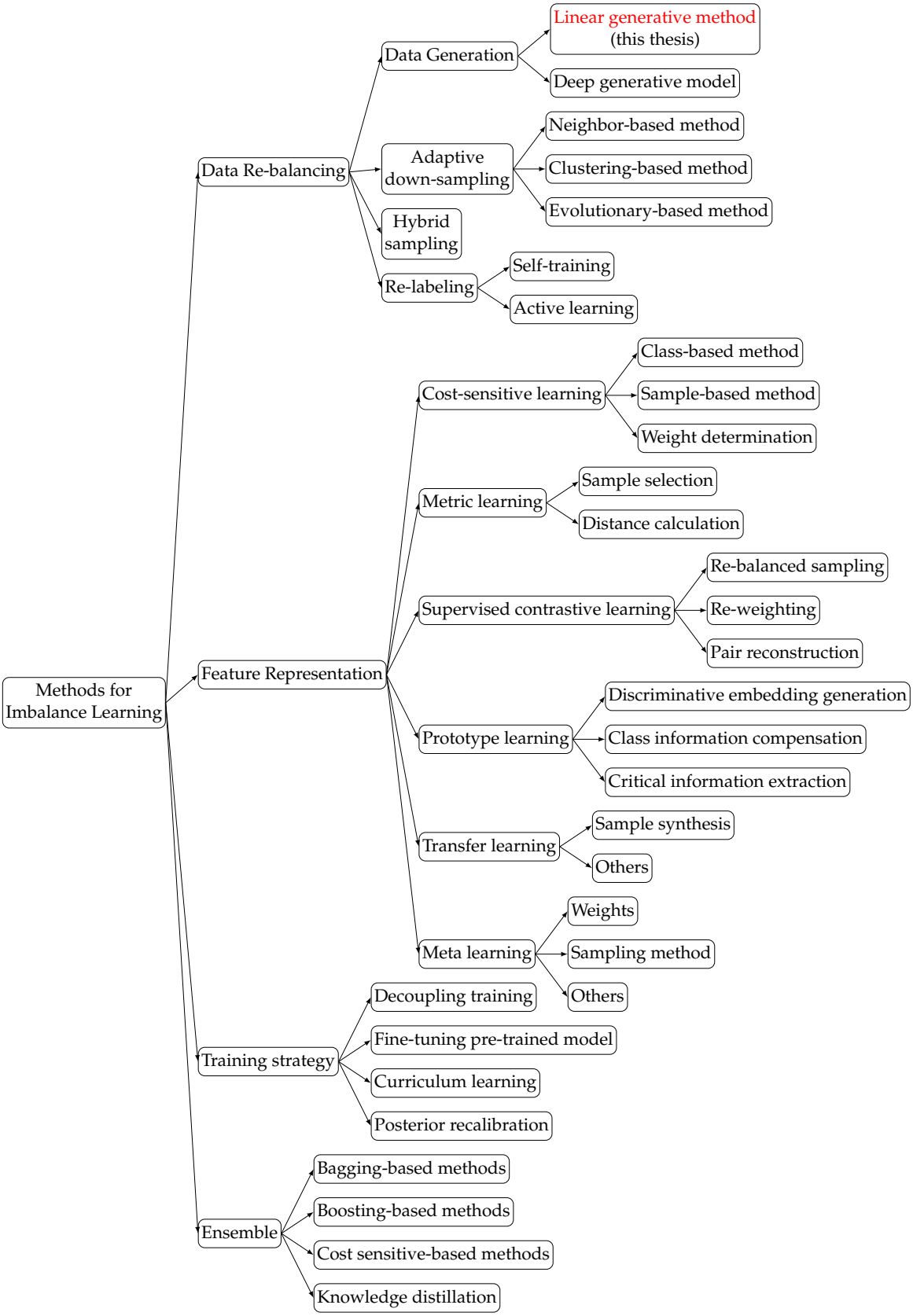

Figure 1.1: Taxonomy of imbalanced learning methods according to [GXZ+25]

false negative. Costs can either be imposed during the training phase by weighting samples or classes differently in the cost function the classifier tries to optimize, or they can be realized as a post-processing step when turning classification scores into class labels by choosing cut-off values for the class label assignment that optimize chosen misclassification costs. With regards to finetuning the raw classification scores, it is to be noted that this is also oftentimes a needed step anyway (regardless of the presence of class imbalances) as many algorithms don't output calibrated classification scores that represent true class probabilities [BMWX21] [NMC05] [Bos08] [GPSW17]. This is a problem, for example, in the medical and other fields where one is interested in an actual probability of the sample belonging to a particular class, and one would have to use calibration post-processing like Platt scaling [Pla99] or Isotonic Regression [ZE02, ZE01] [RWD88] to try and mitigate these issues. A different approach compared to the two above-mentioned ones is taken by metric learning methods, which transform the feature space to improve class separation and therefore reduce the impact that class imbalance has on the overall classification performance.

1.1.3 Training strategies

Training strategies make adjustments in the training pipeline and training strategy. For example, one might use models that were pretrained on similar but balanced data (e.g. text or gene expression data or images) and fine-tune them on the data at hand instead of training them from scratch. Another approach is to decouple the training process into representation learning and classification at the algorithmic level of the classification process and apply various strategies to counteract the bias towards the majority class that most algorithms exhibit when presented with a strongly imbalanced dataset [GXZ+25].

1.1.4 Ensemble methods

Last but not least, ensemble methods combine the wisdom of many different models, which might make them less biased towards the majority class, as the combination process of different models might allow the bias to be balanced out [GXZ+25]. For example, boosting by definition focuses on previously misclassified samples, so any initial bias towards the majority class might, to some degree, be compensated for in the boosting of misclassified minority class samples and the subsequent models that focus on these boosted instances [GXZ+25].

1.2 Methods investigated in this research report

The new sample generation method developed in this research report (c.f. Section 2.4), as well as the methods against which this new method is compared (c.f. Section 3.5), belong to the data re-balancing (linear data generation) category of the above taxonomy.
It cannot be said that gene expression datasets lend themselves in any distinct way to data re-balancing approaches. Quite the contrary, due to their high-dimensional nature, existing data generation methods like SMOTE [CBHK02] might struggle in this setting. The main reason is that many of these methods interpolate between neighboring data points to generate new samples, but, for example, the Euclidean distance breaks down as a useful neighborhood measure in high-dimensional data spaces, as the distance between all samples converges to the same value. This requires alternative neighborhood measures to be investigated and used, and makes the investigation of data generation methods for high-dimensional gene expression datasets an interesting research question. That said, one might ask if the use of data generation methods in high-dimensional settings is not simply a poor

choice of method for a given problem and only of academic interest. The following paragraphs try to justify why this investigation might have merit even for real-world applications beyond pure academic curiosity.
Given that practical real world classification problems often require or at least benefit from some form of cost-sensitive learning and/or decision threshold optimization regardless of any presence of class imbalance (c.f. Section 1.1.2) and given the proliferation of ensemble methods in modern classification pipelines, it is not evident why there might still be merit in concerning oneself with data level methods at all. If cost-sensitive learning might be needed already because of specific real-world misclassification costs, then one might as well kill two birds with one stone and let cost-sensitive learning also take care of any potential class imbalance problems. While that argument might hold in some circumstances, we outline below why data-level methods still might have a place in modern classification pipelines:

- Data-level methods allow addressing the problem once at the data level and then not worry about it anymore, regardless of which classifier one uses afterwards (assuming no classifier-dependent hyperparameter optimization[1] of the data level methods is needed).

- for closed-source legacy systems, data-level methods might be one of the only viable solutions to address the class imbalance problem if the original classifier cannot be migrated to an ensemble method or tweaked to use cost-sensitive classification. Furthermore, not all classifiers (e.g. K-NN) have a cost function that can easily be extended to a cost-sensitive learning approach.

- particularly relevant for gene expression data is that data-level methods might also help with pre-processing steps like feature selection and dimensionality reduction, both of which are frequently used in the context of gene expression data. These parts of the classification pipeline might all be negatively affected by class imbalances, but are not necessarily mitigated by a cost-sensitive classifier or ensemble classifier, as these operate further downstream in the classification pipeline.

1.3 High-level research gap

Many existing linear data generation methods try to generate new samples by linearly interpolating between neighboring existing samples. Particularly, the famous SMOTE: Synthetic Minority Oversampling Technique [CBHK02], and its many variants, rely on the Euclidean distance to establish neighboring samples in feature space. Unfortunately, in very high dimensions, the Euclidean distance becomes problematic as the distance between all samples converges to the same value as the number of dimensions grows high. This makes it difficult to determine semantically meaningful neighbors in high-dimensional spaces. The algorithm might end up interpolating between samples that are very far away along important feature axes, which is likely to negatively affect the classification performance. The breakdown of the Euclidean distance as a useful neighborhood measure in high-dimensional spaces (like gene expression data) motivates the exploration of mitigation measures. Blunt approaches, like unsupervised dimensionality reduction as well as supervised and unsupervised feature selection, to reduce the number of dimensions, are often themselves negatively affected by the class imbalance problem and might not be able to bring the number of features down to a reasonable level anyway. Therefore, exploring alternative proximity or neighborhood measures that work in the context of sample interpolation in high-dimensional spaces might

[1] like the hyperparameter k in SMOTE

be a more promising approach to solving the problem at hand. The exact research gap is narrowed down in more detail in Chapter 2 (Section 2.2 and 2.3).

1.4 Contributions

To overcome the limitations of the Euclidean distance in the context of neighborhood definitions in high-dimensional spaces, we present a novel Random Forest-based neighborhood measure for synthetic oversampling in high-dimensional datasets. Our technique utilizes a Random Forest to define a new neighborhood measure based on the co-occurrence count of samples in purity-filtered leaves assessed across many leaves of many trees in the Random Forest. If a decision tree splits samples into the same high-purity leaf, then these samples must share some notion of proximity or neighborhood at least along the feature axes that were used for the split decisions in the tree along the path from the root to the leaf in question. Our algorithm allows the user to fine-tune a purity criterion that the leaves of the decision trees need to fulfill to be taken into account for the neighborhood or proximity calculation. This allows one to take a deliberate design decision during the application of the algorithm to either generate new samples in the contested boundary region of two classes, or rather, inside the uncontested non-overlapping region of a class. As part of the proximity assessment of samples in the above fashion, the algorithm also keeps track of which features played a role in the leaves that were taken into account. This allows us to enforce that interpolation only takes place along feature dimensions for which it has been established that they are relevant for proximity or neighborhood in the above-mentioned sense. For all other features, the original value of one of the two samples will be taken at random to avoid creating nonsensical noise values by interpolating across feature dimensions for which the two samples are very far apart from each other (in the above proximity sense).
We describe the whole method in more detail in Section 2.4 and evaluate its performance compared to established synthetic sample generation methods in extensive empirical experiments on real-world high-dimensional gene expression data in Chapter 3 and 4. The experimental setup (incl. the datasets, code, and hyperparameter selection) as well as the results (in-sample and out-of-sample) are described in great detail to ensure maximum reproducibility of our experiments and findings, and assessment if our findings are likely to generalize to other datasets and classifiers or not.

1.5 Structure of the report

The remainder of the report is structured as follows: In Chapter 2, we describe the famous SMOTE algorithm in more detail and elaborate on some of its weaknesses. We then go on to introduce a novel Random Forest-based Synthetic Minority Oversampling Technique that is intended to improve on some of SMOTE's weaknesses (particularly in the context of high-dimensional datasets). In Chapter 3, we describe in detail the experiments conducted for this piece of research. The datasets used, the different algorithms we compared, the evaluation metrics, and the overall experimental setup and approach. Chapter 4 delves into the results of the experiments and the comparative performance of our novel method. Last but not least, Chapter 5 gives a conclusion for the research that was conducted and outlines potential for further research.

Chapter 2

Motivation, research gap and algorithmic aspects

This chapter covers the algorithmic aspects of synthetic sample generation, with some sections focusing on high-dimensional datasets. We first introduce the famous SMOTE: Synthetic Minority Oversampling Technique, some of its challenges, some of its extensions, and then present a novel synthetic oversampling technique designed for high-dimensional datasets. Our new technique utilizes a Random Forest to define a new neighborhood measure based on the co-occurrence count of samples in purity-filtered leaves of the Random Forest. The logic behind this is that if a decision tree splits samples into the same high-purity leaf, then they must share some notion of proximity or neighborhood along the features that were used for the split decisions in the tree on the way from the root to the leaf. Our algorithm tries to measure this notion of proximity across many leaves of many trees of the Random Forest. We describe this method in more detail in Section 2.4 and evaluate its performance in Chapter 3 and 4.

2.1 Synthetic Minority Oversampling Technique

As this research report concerns itself with synthetic sample generation in high-dimensional feature spaces, it is worthwhile to explain the most famous synthetic sample generation algorithm, namely SMOTE: Synthetic Minority Oversampling Technique [CBHK02] in some detail.

2.1.1 Inner workings

The basic SMOTE algorithm is comparatively simple in algorithmic terms, as can be seen in the pseudo code diagram Algorithm 1 below (simplified from [CBHK02]). The core idea is as follows: For each minority class sample in the original dataset, one has to find the k nearest minority class neighbors (with k being a hyperparameter to be tuned). Depending on the needed number of new synthetic samples, the following steps are performed once or several times for each minority class sample in the original dataset: One selects randomly (with replacement) one of the previously determined k nearest neighbors and performs a linear interpolation of the feature vectors between the original sample and the selected neighbor. A random vector determines where on the interpolation lines between the two samples the new sample will be generated. The use of a random vector allows for generating several new and different samples even if the same nearest neighbor were to be selected several times. In mathematical terms, the idea can be expressed as follows:

Let $\vec{x}_i \in \mathbb{R}^d$ be a minority sample, $\vec{x}_{nn} \in \mathbb{R}^d$ one of its randomly selected k nearest minority class neighbors, and $\vec{\lambda} \sim \mathcal{U}^d(0,1)$ a multivariate random variable drawn from the unit hypercube of dimension d. With $\odot$ representing the Hadamard product, a synthetic sample $\vec{x}^*$ is then generated as:

$$\vec{x}^* = \vec{x}_i + \vec{\lambda} \odot (\vec{x}_{nn} - \vec{x}_i).$$

If N synthetic points are required to be generated for each $\vec{x}_i$, then a new $\vec{x}_{nn}$ and $\vec{\lambda}$ is chosen N times.

Algorithm 1 SMOTE (S, T, N, k)

Input: Set of minority class samples S, Number of minority class samples T; Oversampling rate N in %; Number of nearest neighbors k
Output: $(N/100) \times T$ synthetic minority class samples
if $N < 100$ **then**
    Randomly sample a fraction of S so that $|S_{new}| = (N/100) \times T$
    $S \leftarrow S_{new}$; $T \leftarrow \lfloor (N/100) \times T \rfloor$; $N \leftarrow 100$
end if
$N \leftarrow \lfloor (N/100) \rfloor$ ▷ Number of synthetic examples per original sample
numattrs $\leftarrow$ Number of attributes of the samples in S
newindex $\leftarrow 0$ ▷ keeps a count of the global number of synthetic samples generated
$Synthetic[N * T][numattrs]$ ▷ empty array for the new synthetic samples
for $i \leftarrow 0$ to $T - 1$ **do**
    take sample $S[i]$ from S and compute k nearest neighbors for $S[i]$, and save the indices in *nnarray*
    GENSYNSAMPLES($N, S[i]$, *nnarray*)
end for

function GENSYNSAMPLES($N, S[i]$, *nnarray*)
    samplecount = 0
    while $samplecount < N$ **do**
        Choose a random integer between 0 and $k - 1$, call it nn
        select the neighbor sample $NS \leftarrow nnarray[nn]$
        Generate a random floating-point vector λ from a unit hypercube with the same dimension as *numattrs*
        for $attr \leftarrow 0$ to $numattrs - 1$ **do**
            $dif \leftarrow NS[attr] - S[i][attr]$
            $Synthetic[newindex][attr] \leftarrow S[i][attr] + \lambda[attr] \times dif$
        end for
        $newindex \leftarrow newindex + 1$
        $samplecount \leftarrow samplecount + 1$
    end while
    return
end function

2.1.2 Shortcomings and weaknesses

One possible reason for SMOTE's popularity is probably that the basic idea is rather simple, and the algorithmic complexity is rather low. That said, this simplicity might also be the reason for a fair number of shortcomings and weaknesses the original method exhibits:

- in the presence of sufficient class overlap between the minority and majority class, the algorithm might create samples in the majority class region.
- in the presence of complex distributions (e.g. with the presence of concave pockets), the algorithm might create samples in the majority class region.
- in the presence of a high level of noise samples in the minority class, the algorithms will potentially amplify this noise by using these noisy samples in the resampling process.
- in very high dimensions, the neighbor selection based on the Euclidean distance becomes problematic as the distance between all samples converges to the same value as the number of dimensions grows high. This makes it increasingly difficult to determine "real" (semantically meaningful) neighbors or even utilize the concept of neighborhood at all. The algorithm might interpolate between samples that are actually very far away along important feature axes
- as the algorithm only creates new samples inside the convex hull of the existing minority samples and only by interpolating between two close-by samples (and sometimes even several times between the same two samples), the variance of the new samples might be artificially low.
- due to the interpolation, the algorithm might destroy some of the original structure of the minority class distributions (like small local clusters)
- as the algorithm deliberately changes the prior probability of classes, many classification algorithms will output classification scores that cannot be well calibrated anymore to represent actual posterior class probabilities. In applications where knowing the actual posterior class probability matters (e.g. the medical field or default risk estimation), this can be a major shortcoming. It is a problem inherent to all synthetic sample generation methods.
- the algorithm is to a degree sensitive to the choice of k and might require hyperparameter tuning to find a good choice for k for a given dataset and given classifier.
- due to the dependence on interpolation for the generation of synthetic samples, the use of the algorithm (in its original formulation) is restricted to numerical features.

An increasing body of literature provides evidence that is critical of the usage of SMOTE, at least in certain conditions. [AIP24] finds that augmenting datasets by oversampling the minority class (either randomly or with SMOTE-type techniques) does not lead to better results (and oftentimes worse results) compared to optimizing the cutoff threshold for the classification scores used to assign class labels. [EAE22] compares the usefulness of SMOTE techniques between what they call "weak" classifiers and what they call strong modern state-of-the-art classifiers. They find that SMOTE might be useful for "weak" classifiers like support vector machines (SVM), but less so or not at all for strong classifiers like boosted decision tree ensembles (XGB, Catboost), which many authors consider a class imbalance mitigation measure in their own right. In the same vein, [CLH+25] finds that SMOTE-type methods might improve the classification performance of some algorithms (e.g. Naive Bayes and support vector machines) but not in others (e.g. XGBoost). [THAA22] performed empirical tests to show that virtually all synthetic sample generation algorithms generate a certain percentage of new samples that can be shown (based on similarity to some hold-out majority class samples) to belong rather to the majority than the minority class. They

argue that synthetic sample generation methods in their current forms are unreliable for the purpose of imbalanced learning and classification and should be avoided in real-world applications. [CLH+25] have shown that oversampling methods introduce changes to the structure or distribution of the classification scores that are hard to correct for even with calibration methods (particularly for logistic regression). This is very problematic for scenarios where true posterior class probabilities matter.

With regards to high-dimensional datasets (the focus of this research), [BL13] states that SMOTE is not helpful or at least less helpful than Random Oversampling in high-dimensional datasets. They, strangely enough, claim that a k-NN classifier based on Euclidean distance does benefit from it if variable selection has been performed (in which case it might not be a high-dimensional dataset anymore). [MLV19, MVFH22] also find that the use of Euclidean distance is not useful in high-dimensional spaces, as with an increasing number of dimensions, the distance between any two samples in the feature space converges to the same value for all pairs. [EA19] finds declining classification performance with increasing dimensions when SMOTE is used. From the many problems and shortcomings of the original algorithms, the focus of this research is the problem the algorithm has with high-dimensional datasets. The next section outlines some of the existing approaches in that context.

2.2 Existing approaches for SMOTE in high-dimensional datasets

When dealing with the weaknesses of SMOTE in high-dimensional feature spaces, one of the first things that comes to mind are techniques to reduce the dimensionality of the dataset as a pre-processing step in the classification pipeline. And indeed, there are many examples where feature selection methods and dimensionality reduction methods were used in the context of genetic datasets. [TDSCDFCDM19] advocates for the use of minimum redundancy maximum relevance (mRMR) feature selection for gene expression datasets (regardless of the presence of class imbalances). [ASZ20] promotes the idea of using feature selection methods in biomedical data, namely Relief [KR92, RSK03], Fisher score [GLH11], and χ^2 [SH05]. [OLBF24] explores different dimensionality reduction methods (e.g. PCA, ICA, auto-encoder) in the context of predictive analytics of transcriptomic data without a particular focus on the class imbalance problem. [AT21] also promotes the use of dimensionality reduction methods (namely PCA, SVD, and self-organising maps) for the classification of gene sequencing data, again without a particular focus on the class imbalance problem. In [SLK22], the authors propose to use dimensionality reduction methods like PCA and a convolutional autoencoder to reduce the dimension in a class-imbalance setting. Interestingly, they seem to perform over- or undersampling steps before the dimensionality reduction and find random undersampling of the majority class followed by convolutional autoencoders to be the most beneficial pipeline (in terms of AUC). [PSS+23] compared different dimensionality reduction methods with SVM-SMOTE for cancer classification based on gene expression data. This approach performed the SMOTE step before the dimensionality reduction step. [AAARA24] also performs oversampling (e.g. SVM-SMOTE) before dimensionality reduction via χ^2 and information gain measures. A different approach is taken in [BDJ16, BDJ18] where the authors propose to perform the synthetic sample generation step inside a learned manifold. So dimensionality reduction by means of PCA, kernel PCA, or autoencoders is proposed to be the first step, and then SMOTE can be performed on the manifold with reduced dimensions. Along the same line of thinking, [BDW+21] proposes the use of t-SNE [MH08] for non-linear dimensionality reduction and subsequent use of SMOTE for synthetic sample generation. The authors of [Sah23] propose the use of PCA for microarray gene expression data (regardless of the presence of class imbalance). Un-

fortunately, they fail to provide a performance comparison to a baseline without the use of PCA. [KCL23] compares different dimensionality reduction algorithms (PCA, kernel PCA, autoencoder) for cancer detection based on gene expression data, with the use of SMOTE after the dimensionality reduction step in the classification pipeline. A less explicit feature selection approach is taken in [MVFH22], who use a feature-weighted distance metric when determining the neighbors in the SMOTE algorithm. They use a relevance/redundancy-based feature weighting algorithm that allows for a feature-weighted distance metric to determine the k-NN used for SMOTE.

The difference in the order of dimensionality reduction vs. synthetic sample generation across the different publications highlights a certain circular dependency created by the combination of SMOTE and dimensionality reduction methods. Feature selection and dimensionality reduction methods potentially suffer themselves from the class imbalance problem [PRHK14]. Particularly unsupervised dimensionality reduction or feature selection methods will just focus on representing the complete dataset as best as possible, with an unavoidable bias towards the samples of the majority class. This would make the application of data-level methods to correct the class imbalance problem a good first step. But at the same time, SMOTE and other interpolation-based synthetic sample generation algorithms struggle to identify reliable neighboring samples in high-dimensional data spaces and would require dimensionality reduction or feature selection to happen first. A classic circular dependency or double bind that has no clear resolution and requires other approaches to handle synthetic sample generation in high-dimensional datasets to be explored.

Out of curiosity, we evaluated the use of dimensionality reduction methods as a step in the classification pipeline (in their own right and without any attempts to rectify any present class imbalances, c.f. Figure 3.2). Details are provided in Chapter 3 and 4, and they seem to have some benefits in some cases without there being a single clear best dimensionality reduction method across all scenarios. We do not combine them with any approach to rectify the class imbalance problem as part of this research, as the circular dependency highlighted above can not be resolved, and empirically evaluating the best order between dimensionality reduction and sample generation across several datasets, classification algorithms, and dimensionality reduction methods was computationally prohibitively expensive.

2.3 Forest based synthetic sample generation

Decision tree ensembles, including Random Forests, have a good reputation as reliable, high-performing methods in modern classification pipelines (at least for tabular data). As an ensemble method, they constitute a class imbalance mitigation measure in their own right (according to the taxonomy in Section 1.1). Compared to boosted decision tree ensembles, the training of Random Forests can be highly parallelized as all trees are trained independently from each other. This makes Random Forests computationally attractive and conceptually intuitive. In the context of the research question at hand, one cannot help but wonder if and how one could exploit the Random Forest's seemingly strong capability of understanding even complex class distributions and decision boundaries for the purpose of generating new synthetic samples of a minority class. In a naive way, one might think that if an algorithm is highly capable of internalizing class distributions in a way to effectively discriminate samples from two different classes, there must be a way to use this internal representation as well for generating new synthetic samples.

Unfortunately, such an approach is not self-evident, as decision trees as well as ensembles of decision trees are usually considered a discriminative method, focusing on estimating the conditional distribution $p(y \mid X)$. Generative methods like Generative Adversarial Net-

works [GPAM+20] or Variational Autoencoders [KW13], on the other hand, try to estimate the joint distribution $p(X, y)$ and therefore lend themselves more naturally to generating new samples. That said, there are several approaches described in the literature that use decision trees for density estimation of the joint unconditional distribution of the data. Unfortunately, many of these approaches mainly focus on evaluating the distribution at some point in the feature space (e.g. [WH22]) or on calculating some marginal distribution for the purpose of data imputation (e.g. [CPC20]) rather than on generating complete new samples from a minority class. One could always try to employ rejection sampling, but unfortunately, this approach is computationally infeasible, as sampling a 10 thousand dimensional feature space at random while hoping to end up in an area of the feature space representative of the minority class has extremely low chances of success. [Bra25] describes an approach that uses a boosted decision tree ensemble for data generation. Samples are generated via Gibbs sampling by changing one feature at a time and using the decision tree ensemble to perform rejection sampling. Gibbs sampling is probably more promising than simple rejection sampling, but it is still computationally intense and can have conversion problems. The authors in [Bra25] also propose a sampling method that uses an ensemble of density estimation trees [RG11]. It is unclear, though, from their publication how they would achieve a good coverage of the feature space (unless the trees are very, very deep) and how they would focus on generating only samples from the minority class. For a feature space of over 10000 features, one would have to use very deep trees to cover the space sufficiently, but that would also increase the risk of overfitting training data. [Bra25] advocates for the usage of bagging by training shallower trees on a subset of the feature space. It is not clear, though, how one would construct a sample spanning the whole feature space from such an ensemble of shallow trees. [WBKW23] describes an approach to train an adversarial Random Forest classifier that specializes in recognizing synthetic samples derived from the trees of its own forest. One single Random Forest is used for discrimination and sample generation. Unfortunately, samples are only ever generated from one tree at a time, and it is not immediately clear how to restrict new samples to the minority class.

The question of how to tap into the wisdom of the ensemble for every single sample and not just "on average" across many samples is left unanswered in many approaches or circumvented on purpose by using single trees for each sample. In the discriminative use case, it might be justified and possible to combine many weak classifiers into one strong ensemble, but it's far less intuitive for the generative use case, how to combine many bad or weak samples into one good or strong one. The weakness of generating samples from individual trees of an ensemble lies in the fact that the information contained in other trees is not drawn upon. Furthermore, the trees would have to be very deep to cover a meaningful portion of the relevant feature space, but using very deep trees during the training phase is likely to lead to an overfit of the training data with negative effects on the new samples that one would be able to create from such trees. If one were to use many shallow trees (all covering parts of the feature space), it would not be clear either how to combine samples from such trees (all with different but potentially overlapping coverage of the feature space) into one strong sample. [NGB24] is one of the only approaches that tries to make use of the full decision tree ensemble for generating new samples. They describe a method to generate samples from a Random Forest by creating the intersection of feature space regions across trees of a forest. They iteratively restrict the feature space by moving down from the root of a tree one level towards the leaves and one tree at a time until no more steps towards a leaf can be taken for any tree of the forest while still intersecting with the existing narrowed down feature space of the previous iteration. In other words, the approach doesn't try to find the intersection in the feature space of all leaves of all trees, which might lead to an empty intersection. It rather attempts to find an intersection at higher levels of the decision

trees (using splitting points closer to the roots of the trees) and refining this intersection by moving towards the leaves of the decision trees until conflicts occur. The approach is interesting due to the fact that the authors try to capitalize on the combined wisdom of the full ensemble of trees to represent the distribution to sample from. There is also an implicit answer to the question of how to deal with parts of the feature space that don't form a part of the support of the leaves of the trees in the forest (namely, one would sample uniformly over the intervals spanned by the training samples in the forest for features that are not restricted by branches and leaves in the trees of the forest). Unfortunately, the authors don't describe a method to restrict the sample to minority class samples, which would therefore require an inefficient rejection sampling approach. That said, one could probably amend their iterative sampling algorithm to favor branches of the trees that are biased towards samples of the minority class. This approach is the most promising tree ensemble method so far for our purpose. One concern might exist for datasets with two very distinct sub-clusters of the minority class, as the "consensus approach" taken in the algorithm would probably just end up creating a bounding hypercube around the two sub-clusters (potentially including vast areas of the feature space belonging to the majority class).

2.4 Novel Random Forest-based Synthetic Minority Oversampling Technique

In this section, we describe our novel Random Forest-based Synthetic Minority Oversampling Technique. Our main contributions and elements of the algorithm are

- a Random Forest-based neighborhood measure based on sample co-occurrence in purity filtered minority class dominated leaves
- a feature masked sample interpolation mechanism to ensure interpolation between samples is restricted to feature axes that contributed to establishing sample proximity in feature space
- a filter mechanism that allows only the top x% of globally closest sample pairs of the dataset to generate new synthetic samples

Due to the above-mentioned shortcomings of the existing tree-based methods, we were forced to explore alternative ways to capitalize on the information contained in a trained Random Forest for the purpose of synthetic sample generation. Given that the main weakness of the SMOTE algorithm in high-dimensional spaces stems from the breakdown of the Euclidean distance as a useful neighborhood measure, we focused on ways to use the Random Forest to find "good" neighboring samples of the minority class for the purpose of interpolating between them.
The proposed solution is a new neighborhood measure based on counting the co-occurrence of samples in minority-class-dominated leaves of a Random Forest, where the leaves also meet certain stricter class purity criteria beyond simply containing more than 50% minority class samples. The rationale behind the neighborhood measure is that if a decision tree splits samples into the same high-purity leaf, then these samples must share some notion of proximity or neighborhood at least along the feature axes that were used for the split decisions in the tree along the path from the root to the leaf in question. These samples will basically all sit in the same hypercube bounding box in feature space that is defined by the leaf in question. The more often samples end up in the same hypercube bounding boxes defined by the high-purity leaves within a Random Forest, the closer one can assume those samples to be to each other in the part of the feature space that matters for class discrimination.

Our algorithm deliberately allows for fine-tuning a purity interval that the leaves of the decision trees need to fall into as a hyperparameter. This allows one to take a deliberate design decision during the application of the algorithm to either generate new samples in the contested boundary region of two classes (low leaf purity) or, rather, inside the uncontested non-overlapping region of a class (high leaf purity).
As shown in Figure 2.1, our algorithms draw up a neighborhood tensor where the neighborhood relation of each sample with each other sample in the dataset is recorded. For each sample combination (e.g. the black square and black diamond in the schema in Figure 2.1), we record a vector of the length of the number of feature dimensions of the dataset. Each time two samples end up in the same leaf of a tree of the random forest, we increment the elements of the vector (for this sample combination) that correspond to the features that were used for splitting decisions on the path from the root to the leaf. This leads to a 3-dimensional neighborhood dataset (3D-tensor).

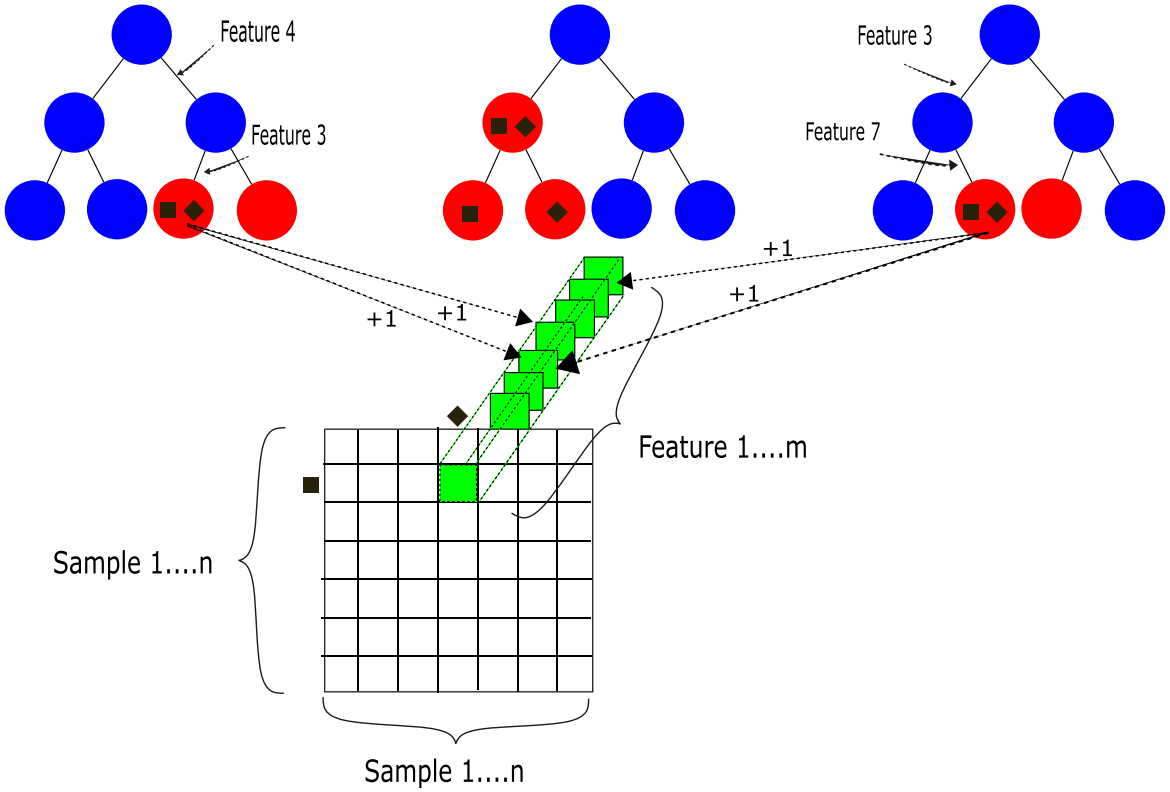

Figure 2.1: Schematics of RandomForestSMOTE neighborhood tensor

Keeping track of the features that played a role in establishing a neighborhood or proximity relationship between two samples allows us to ensure that the actual interpolation between neighboring samples (at a later stage) is feature-masked in the sense that interpolation only takes place along feature dimensions for which it has been established that they were part of the above-described neighborhood measure. For all other features, the original value of one of the two samples will be taken at random to avoid creating nonsensical noise values by interpolating across feature dimensions for which the two samples are very far apart from each other (in the above proximity sense).
The 3rd tensor dimension (the green vector in Figure 2.1) can be condensed to a single neighborhood score for each combination of samples by taking the L1 or L2 norm of the vector. A L1 norm treats every feature that was part of a decision path equally and additively. Ceteris paribus, the L1 norm makes sure that two samples that co-occur in 5 leaves (each time

with the same features on the splitting path) will not score any differently than two samples that co-occur in 5 leaves (each time with different features involved in the splitting path). The L2 norm, on the other hand, over-emphasizes (due to squaring) the scenario where two samples co-occur in 5 leaves (each time with the same features on the splitting path). The following numerical example illustrates the above point:
If we consider a vector reflective of a case where a good variety of different features were involved in establishing sample proximity

$$\mathbf{x} = \begin{bmatrix} 0 & 1 & 2 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

We get the following L1 Norm result.

$$\|\mathbf{x}\|_1 = |0| + |1| + |2| + |1| + |0| + |1| + |0| + |0| + |1| + |0| = 6$$

and the following L2 Norm result

$$\|\mathbf{x}\|_2 = \sqrt{0^2 + 1^2 + 2^2 + 1^2 + 0^2 + 1^2 + 0^2 + 0^2 + 1^2 + 0^2} = \sqrt{8} \approx 2.83$$

If, on the other hand, we consider a vector where mainly one feature was involved many times in establishing proximity between two samples (with the same overall number of co-occurrences in leaves as in the first example above).

$$\mathbf{x} = \begin{bmatrix} 0 & 0 & 5 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

We get the following L1 Norm result (which is identical to the first example above)

$$\|\mathbf{x}\|_1 = |0| + |0| + |5| + |0| + |0| + |1| + |0| + |0| + |0| + |0| = 6$$

but a rather different and higher L2 Norm result (compared to the first example above)

$$\|\mathbf{x}\|_2 = \sqrt{0^2 + 0^2 + 5^2 + 0^2 + 0^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2} = \sqrt{26} \approx 5.10$$

We couldn't justify that the second vector example should result in a much higher neighborhood score than the first example (quite the contrary) and therefore chose the L1 norm over the L2 norm for our experiments. We did not explore any other norms (e.g. an inverse L2 norm that would punish cases where only one or very few features would be used over and over to establish sample proximity). That could be explored in future research.
Once a proximity score has been calculated for each sample combination, one can create a sorted list or table of sample pairs (based on their proximity score) and can take the top x % "closest" neighboring samples for the purpose of interpolation to generate new samples. The idea is to interpolate between the globally closest neighbors (those samples that end up in the same leaves most often) to minimize the risk of interpolating between samples where the interpolation lines might cross into the majority class territory in feature space. But we acknowledge that this design decision introduces some strong bias with regard to where new samples get generated (namely, in regions where very close neighbors already exist). In our experiments, we chose the top 5-10% closest neighbor pairs for the purpose of generating samples via interpolation, with the exact percentage threshold being a hyperparameter that was optimized during the experiments. From the top (closest) x% neighbor pairs, we select pairs proportionally to their relative score strength. The closer the neighbor pair, the more new synthetic samples will be generated by them.
Another set of hyperparameters the algorithm takes is the number of trees and their maximum depth for the Random Forest. The full algorithm is described in the pseudo-code diagram Algorithm 2 below.

Algorithm 2 RandomForestSMOTE_Oversampling

Input: Dataset (X, y), Random Forest parameters $(n\_estimators, max\_depth)$, leaf purity bounds (min_purity, max_purity), top pair fraction α
Output: Resampled dataset (X_{res}, y_{res})
Convert X, y to arrays.
Identify minority class c_{min} and majority count N_{max}.
Extract (X_{min}, y_{min}) (minority samples), and compute the number of needed synthetic samples $N_{new} = N_{max} - |X_{min}|$.
Train Random Forest classifier on (X, y) with class weighting enabled.
Obtain leaf assignments for all samples (List of all leaves across all trees a sample ends up in).
Compute feature-weight tensor W using minority class leaves with purity in $[min\_purity, max\_purity]$, finding minority class sample pairs that end up in the same leaf, and then recording the features that constituted the splitting points on the path to these leaves. W encodes pairwise feature-level "shared decision path importance" across all qualifying leaves of the forest.
Compute pair scores S from W (L1 norm). The longer the shared decision path, the higher the scores, and the more shared decision paths, the higher the score.
Select top α fraction of minority pairs based on S.
Allocate the needed N_{new} synthetic samples proportionally to pair scores (tighter neighbor pairs get to generate more samples). Each selected neighbor pair will end up with a number n_{ij} of new samples it has to generate.
Initialize empty set X_{syn}.
for each selected pair (i, j) with allocated count n_{ij} **do**
    Let w_{ij} be the feature mask from W.
    for $k = 1$ to n_{ij} **do**
        Sample $\lambda \sim U$ (random vector from unit hypercube of same dimension as w_{ij}).
        Construct x_{new} feature-wise:

$$x_{new}[f] = \begin{cases} X_{min}[i, f] + \lambda[f] \cdot (X_{min}[j, f] - X_{min}[i, f]) & \text{if } w_{ij}[f] > 0 \\ X_{min}[i, f] \text{ or } X_{min}[j, f] \text{ (random)} & \text{otherwise} \end{cases}$$

        Add x_{new} to X_{syn}.
    end for
end for
Set y_{syn} as c_{min} for all synthetic samples.
Return $X_{res} = [X; X_{syn}]$, $y_{res} = [y; y_{syn}]$.

While the original algorithm is restricted to binary classification, it could obviously also be extended to a multi-class scenario, as we only need to be able to select the leaves of interest based on which class dominates these leaves. So one would be able to generate samples for any of many possible classes. In the following Chapter 3 and 4, we put this new method to the test and see how it compares to existing SMOTE-based approaches with a high-dimensional dataset. At a high level, our approach is conceptually similar to a feature-weighted SMOTE approach as done in [MVFH22]. But instead of using feature selection or weighting heuristics like relevancy/redundancy that are often used in filter approaches, we use feature importance derived from a Random Forest. The next chapter will go into detail about the experiments that were conducted as part of this research.

Chapter 3

Experimental Setup

3.1 Datasets

For the purpose of this study, we used 3 gene expression datasets from OpenML: [Ope08a], [Ope08c], [Ope08b]. These datasets represent a binary classification problem for different types of cancer based on gene expression data as predictor variables. All of these datasets are characterized to various degrees by class imbalance as well as high-dimensional feature spaces (above 10000 numerical features). The datasets were originally published in connection with [SK10]. The characteristics of the datasets are summarized below. We took the top 3 datasets of all Gene Expression Machine Learning Repository (GEMLeR) [SK10] datasets on OpenML, sorted by the level of class imbalance. The level of class imbalance is measured by the imbalance ratio (number of instances of the majority class divided by the number of instances of the minority class). This measure is recorded in the ImbRatio column in Table 3.1 below. The datasets did not contain any batch or cohort IDs.

Table 3.1: OpenML Dataset Characteristics

ID	Name	MajClsSize	MinClsSize	NumInstances	NumClasses	NumFeatures	ImbRatio
1142	OVA_Endometrium	1484	61	1545	2	10936	24.33
1146	OVA_Prostate	1476	69	1545	2	10937	21.39
1139	OVA_Omentum	1468	77	1545	2	10936	19.06

The datasets are based on datasets from the Expression Project for Oncology (expO) [Int05, BTW+07] published and maintained by the International Genomics Consortium. The authors of [SK10] modified the original datasets and used an unsupervised highest variance filter for the gene expression features to retain only the 20% of genes with the highest variance across all samples. The motivation for this was to eliminate genes with practically constant signal to achieve faster computation times and lower memory requirements [SK10]. The 3 datasets are of the "one-vs-all" (OVA) type. To quote the original description of the datasets, the meaning is as follows [Fak12]:

> OVA benchmarking datasets collection consists of two-class classification problems where one of the cancer type groups is compared to samples from all other types that are marked "Other" instead of their cancer type.

The dataset has no missing values, and all gene expression features consist of numerical values in the range of $[0.2, 863981.0]$. Unfortunately, it was only discovered after the experiments concluded that the datasets contained calibration data from 54 calibration probes used in the Affymetrix GeneChip U133 Plus 2.0 arrays that were utilized to generate the data. We

determined that including the calibration data in the feature dataset does not negatively affect the main findings of this research report. The datasets don't pose any particular challenge apart from their high dimensionality, namely, over ten thousand features, having more features than the number of samples (by a factor of 7), and being highly class-imbalanced. As all three datasets are OpenML/expO one-vs-all gene expression tasks with similar sample size, feature dimensionality, and source characteristics, it is to be noted that performing classification on the three above-mentioned datasets is a strongly related task. Therefore, the fact that three datasets instead of one dataset were used to create averaged results does not constitute broad independent evidence for the performance comparison of the various methods. Assessing the performance of the various methods on a more diverse group of datasets is something left for future research.

3.2 General experimental framework

The overall experimental framework used in the experiments was a 5-fold nested cross-validation [HTF09][p. 241ff.] with 300 hyperparameter optimization trials for each iteration of the outer loop. The setup is illustrated in Figure 3.1. The full dataset gets split into 5 folds. Four of these folds are passed on to the inner loop of the nested cross-validation, while one fold remains untouched as a hold-out test dataset to assess out-of-sample performance. The same logic is applied in a 5-fold inner cross-validation loop where hyperparameter optimization takes place. Hyperparameters are configuration settings of a learning algorithm that cannot be learned directly from the data. The optimal choice for k in the k-nearest neighbor classifier is an example of a hyperparameter. One way to let the data help make an informed decision for the optimal values of hyperparameters is to try different settings and see which one performs best. To avoid over-fitting the hyperparameter to the training dataset, one would rank them by their out-of-sample rather than their in-sample performance. The inner 5-fold cross-validation aims to do just that. It allows us to identify hyperparameter settings that perform well (on average) across 5 different combinations of training and test data folds (c.f. the orange and red folds in the diagram in Figure 3.1). We used the RepeatedStratifiedKFold (outer cross-validation loop) and StratifiedKFold (inner cross-validation loop) method from the scikit-learn Python package [PVG+11] with sampling shuffling enabled. Using stratified sampling for the determination of the 5 folds ensures that the class imbalance ratio of the two classes remains the same across all folds. Enabling shuffling or random selection makes sure the original order of the samples is not preserved, mixing up any structure that might exist in the original dataset. A fixed random seed of 42 was used for both functions to ensure replicability of our experiments and results. The dataset did not contain any batch or cohort ID that would have warranted or allowed for the use of the GroupKFold or StratifiedGroupKFold functions. If the samples of the different cancer types that form the basis of the datasets all originate from distinct batches (e.g. one for each cancer type), there is a risk that the classifiers don't learn underlying biological phenomena but rather certain batch-based measuring characteristics (batch effects). This bears the risk that the overall classification performance might be over-optimistic, but in our opinion, the relative performance of the methods we investigate should be far more resilient against this potential problem, unless there is a strong argument for why certain combinations of sample generation algorithms benefit more from batch effects than others. The Optuna Hyperparameter Optimization framework [ASY+19] was used to choose iteratively 300 different hyperparameter combinations for evaluation inside the inner cross-validation loop (c.f. Section 3.8). For each of the 300 hyperparameter settings, the whole classification pipeline, as depicted in Figure 3.2 and 3.3, was trained on the orange folds and

evaluated on the red folds. An average is taken over the evaluation results of all 5 combinations of folds (iterations of inner cross-validation), and the hyperparameter set with the highest inner cross-validation score is selected and passed back to the outer cross-validation loop. There, the green folds are used to re-train the classification pipeline of Figure 3.2 and 3.3 using the best hyperparameter set as determined by the inner cross-validation loop and then tested on the yellow fold. This process is repeated 5 times for different splits of training and test data in the outer loop of the nested cross-validation. Each outer iteration leads to an out-of-sample performance estimate of the classification pipeline, and the mean over all outer iterations gives a robust estimate of the out-of-sample performance of the selected classification pipeline and training approach (incl. hyperparameter optimization via cross-validation). The strict separation of training and test data between inner and outer cross-validation prevents leakage within the implemented classification pipelines in Figure 3.2 and 3.3. However, as discussed in Section 3.3 and Chapter 5, the OpenML datasets had already undergone global variance filtering before the experiments, which may inflate absolute performance estimates.
The above process was performed for 3 different datasets, 6 different classification algorithms (c.f. Section 3.6), and 6 different sample generation approaches in one set of experiments, as well as 3 different dimensionality reduction methods in a second set of experiments. We describe the steps of the pipelines in Figure 3.2 and 3.3 in more detail in the Sections 3.3 to 3.6. It is important to note that the sample generation step of the classification pipeline in Figure 3.3 was only used during the training phase of the experiments[1] as it would have been nonsensical to evaluate the classifier performance out-of-sample on synthetic samples.

3.3 Pre-processing

As mentioned in Section 3.1, the datasets were partly preprocessed already by a variance filter that retained in an unsupervised fashion the 20% of genes (features) with the highest variance across all samples. **It is to be noted that this type of preprocessing constitutes a case of data snooping between training data and test data and therefore is likely to inflate absolute classification performance compared to a real-world scenario where the test data is not yet available when unsupervised feature selection based on variance takes place. We hope that the relative performance ranking of the various methods in our experiments is largely unaffected by this type of data snooping**. As part of the classification pipeline, we ran the StandardScaler[2] of the Python scikit-learn package [PVG+11] over all features to turn them into z-scores by removing the mean of each feature and scaling each feature to unit variance. This was deemed necessary to avoid any unwanted biases in distance-based methods like the K-nearest neighbor classifier [FH51, CH67].

3.4 Dimensionality reduction

One set of experiments explored the use of linear unsupervised dimensionality reduction methods in their own right without combining them with synthetic sample generation (c.f. Figure 3.2). These experiments were part of early exploratory research on the topic, but the line of investigation was ultimately abandoned. The benefits of the following methods were explored:

[1] https://imbalanced-learn.org/stable/references/generated/imblearn.pipeline.Pipeline.html
[2] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

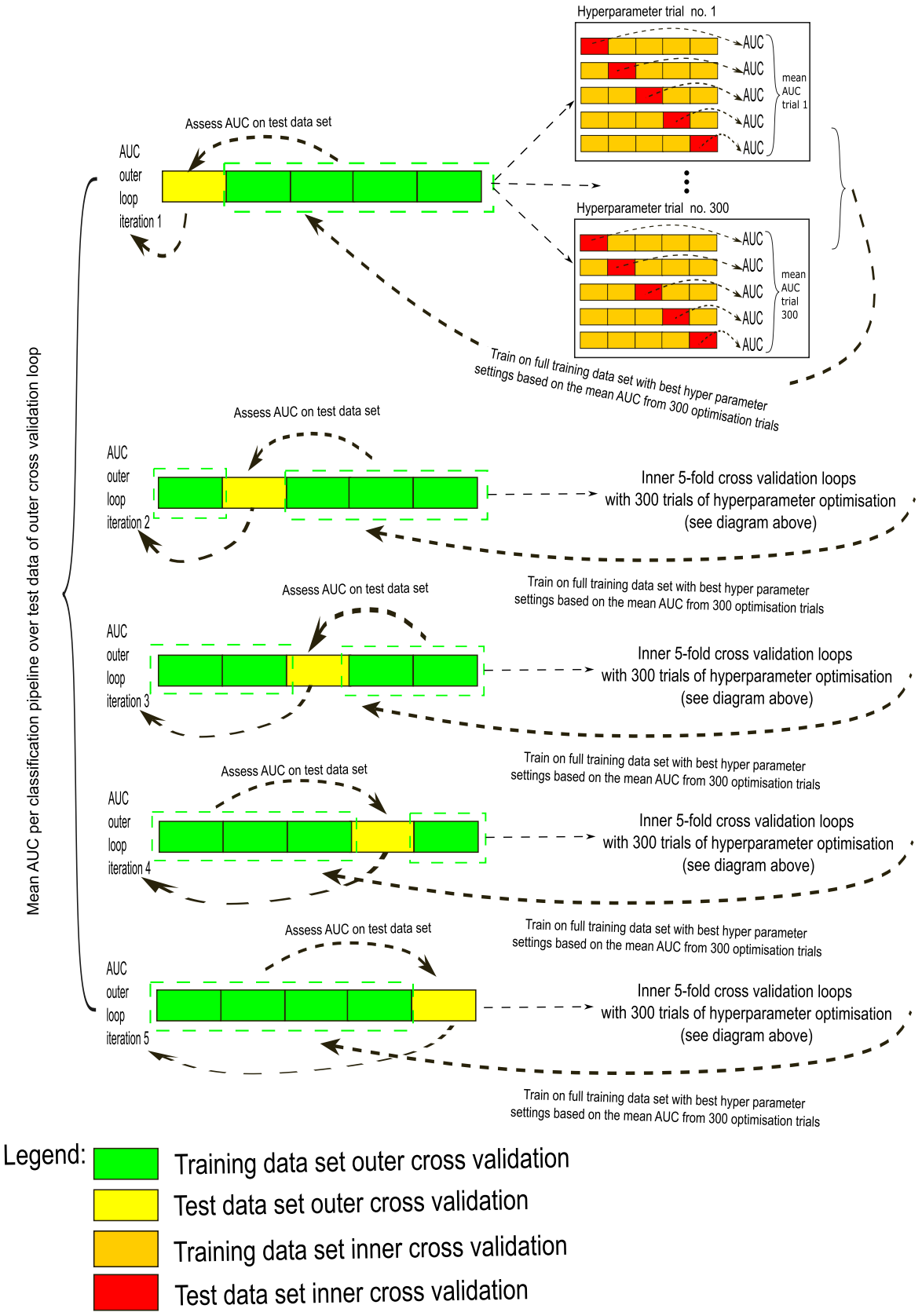

Figure 3.1: 5-fold nested cross-validation with hyperparameter optimization

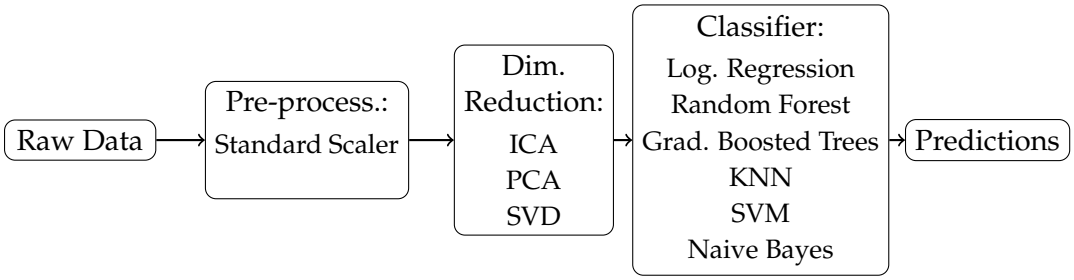

Figure 3.2: Pipeline for first set of experiments to evaluate dimensionality reduction in its own right

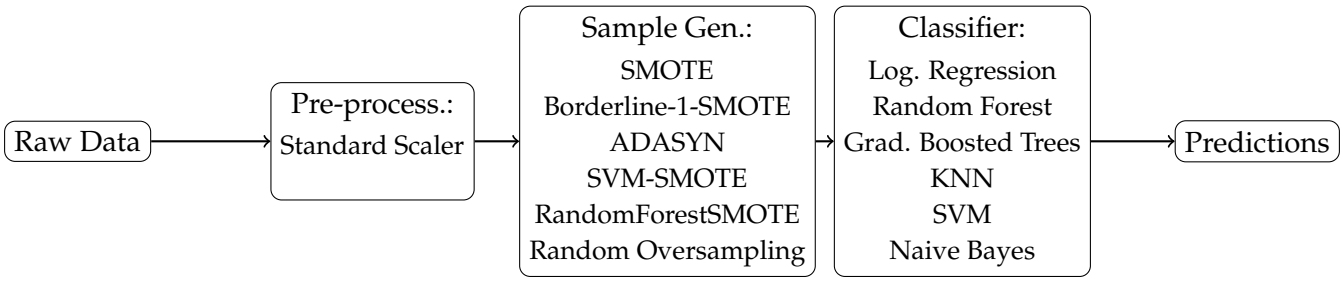

Figure 3.3: Pipeline for second set of experiments to evaluate synthetic sample generation methods

1. Fast Independent Component Analysis (ICA) [HO00]
2. Principal Component Analysis (PCA) [Hot33]
3. Singular Value Decomposition (SVD) [EY36, MP11]

In retrospect, it is to be noted that given the pre-centering and pre-scaling of the data during the pre-processing step, one would expect the results from the PCA and the SVD to be identical, and it is redundant to run experiments for both. That said, maybe subtle differences on an algorithmic level in the scikit-learn Python package [PVG+11] might lead to different results and warrant exploring both methods. During the experiments, a varying number of components from the result of these factorization methods were kept to perform the actual dimensionality reduction. The use of supervised and non-linear dimensionality reduction and manifold learning methods was considered but was not pursued due to their high computational requirements (incl. hyperparameter tuning).

3.5 Sample generation algorithms

As part of the experiments for generating synthetic samples of the minority class, we used the following algorithms of the imbalance-learn Python package [LNA17] to generate synthetic samples until the dataset is fully balanced:

1. Synthetic Minority Oversampling Technique (SMOTE) [CBHK02]
2. Borderline-1-SMOTE [HWM05]
3. ADASYN [HG09]
4. SVM-SMOTE [NCK09]

5. RandomForestSMOTE (c.f Section 2.4)

6. Random Oversampling [KLC23]

Borderline-SMOTE is a SMOTE variant that focuses on generating new synthetic samples at the class boundary (borderline). It exists in two variants: The first one (which we used) generates samples only from nearest neighbors that all belong to the minority class, while the second variant would also allow majority class samples to be part of the nearest neighbors. Borderline samples are identified if half or more (but not all) of their neighbors belong to the majority class. ADASYN [HG09] tries to focus on minority samples that are hard to learn (for example, near the decision boundary or surrounded by many majority class neighbors). SVM-SMOTE is similar to Borderline-SMOTE [HWM05], but the borderline area is approximated by support vectors from an SVM trained on the original imbalanced training set [NCK09]. We compared the 3 above mentioned SMOTE variants with our own novel Random Forest-based Synthetic Minority Oversampling Technique, which we introduced in Section 2.4, as well as Random Oversampling. It is important to note that the sample generation step of the classification pipeline was only used during the training phase of the experiments[3] as it would have been nonsensical to generate and include synthetic samples in the out-of-sample performance evaluation of classifiers.

3.6 Classifier

For our experiments, we used the following classification algorithms:

- Logistic regression [Cox58] with a L1 penalty term (Lasso regularization [Tib96])
- Random forests [Bre01]
- Gradient boosted decision tree ensembles (XGBoost) [Fri00]
- KNN classifier [FH51, CH67]
- SVM with a linear kernel [CV95, Vap97]
- Naive Bayes [Mur12]

For all algorithms, we relied on the implementation in the scikit-learn Python library [PVG+11], except for the gradient boosted decision tree ensemble, for which we used the XGBoost implementation [CG16].

3.7 Code and compute platform

As a compute platform for all experiments, we used spot instances of the high-performance computing server (h3-standard-88, 88 vCPUs, 352 GB memory) on the Google Cloud Platform. The code for all experiments can be found on GitHub `https://github.com/econdatatech/Data588/blob/main/OSExperiments.ipynb`, `https://github.com/econdatatech/Data588/blob/main/DimreductionExperiments.ipynb` and `https://github.com/econdatatech/Data588/blob/main/HyperparameterImportance.ipynb`.

[3] `https://imbalanced-learn.org/stable/references/generated/imblearn.pipeline.Pipeline.html`

3.8 Hyperparameter optimization

For all experiments, we used the Optuna Hyperparameter Optimization framework [ASY+19]. For each combination of classification algorithm and dimensionality reduction algorithm or sample generation algorithm, respectively, we performed 300 trials of 5-fold cross-validation to tune the hyperparameters in an inner cross-validation loop; these steps were repeated 5 times as part of the outer cross-validation loop. For each classifier and dimensionality reduction method or sample generation method, a set of hyperparameters was defined together with a value range over which to optimize them. Mean AUC values were calculated over 5 different folds of an inner cross-validation loop to determine the best hyperparameter setting, which was then passed on to the outer cross-validation loop, where the hyperparameter setting is used to train the classification pipeline on all 4 folds of the training dataset, and an evaluation is performed on the hold-out or test data fold. This process is performed 5 times as part of the outer cross-validation (c.f. Figure 3.1). The Optuna optimization engine steered the sampling of new hyperparameter values based on the results of previous runs. Optuna was run mainly with default settings, namely MedianPruner[4] for early stopping and the Tree-structured Parzen Estimator (TPE)[5][Wat25] as the sampling algorithm. The MedianPruner early stopping stops seemingly unpromising trials at early stages if it finds that the intermediate results from the hyperparameter optimization compare poorly against the median of all previous completed trials.

Hyperparameter for dimensionality reduction experiments

The hyperparameters for the linear dimensionality reduction experiments are listed in Table 3.2. The hyperparameters in our experiments are either integer, float, or categorical in nature. Categories are listed in the choices column. Integer and float ranges are defined by the low and high columns. Step defines the level or granularity in an integer range to tell Optuna in what step sizes the interval can be explored. We set it to 1 for all our experiments. The log column indicates if the log parameter was set to TRUE or FALSE in Optuna. According to [ASY+19], it is

> a flag to sample the value from the log domain or not. If log is true, the value is sampled from the range in the log domain. Otherwise, the value is sampled from the range in the linear domain.

Sampling in the log domain can be important to make sure each order of magnitude is equally likely. It is a desirable feature if the hyperparameter range spans several orders of magnitude.
For logistic regression, the hyperparameter C is the "Inverse of regularization strength [...] smaller values specify stronger regularization." [PVG+11]. In Naive Bayes, the var_smoothing parameter controls a small constant to be added to the variance estimates of features. This helps to prevent extremely small estimated variances and ensures numerical stability in the likelihood calculations. For Random Forests, the n_estimators steers the maximum number of trees in the forest to balance computational burden with discriminative performance, and max_depth to steer the maximum depth of each tree to prevent overfitting and ensure good generalization performance. For linear support vector machines, the hyperparameter C is a regularization parameter. "The strength of the regularization is inversely proportional to

[4] https://optuna.readthedocs.io/en/stable/reference/generated/optuna.pruners.MedianPruner.html#optuna.pruners.MedianPruner

[5] https://optuna.readthedocs.io/en/stable/reference/samplers/generated/optuna.samplers.TPESampler.html#optuna.samplers.TPESampler

Table 3.2: Hyperparameter distributions by study for linear dimensionality reduction experiments

param_name	dist_name	log	step	low	high	choices
Logistic Regression						
C	Float	TRUE		1e-04	100	NA
ncomp	Int	FALSE	1	5	988	NA
Naive Bayes						
var_smoothing	Float	TRUE		1e-11	1e-07	NA
ncomp	Int	FALSE	1	5	988	NA
Random Forest						
n_estimators	Int	FALSE	1	100	1000	NA
max_depth	Int	FALSE	1	2	20	NA
ncomp	Int	FALSE	1	5	988	NA
SVM (Linear)						
C	Float	TRUE		1e-04	100	NA
ncomp	Int	FALSE	1	5	988	NA
XGBoost						
learning_rate	Float	FALSE		0.01	0.3	NA
max_depth	Int	FALSE	1	3	10	NA
n_estimators	Int	FALSE	1	100	1000	NA
ncomp	Int	FALSE	1	5	988	NA
k-NN						
n_neighbors	Int	FALSE	1	3	30	NA
weights	Categorical	NA	NA	NA	NA	uniform, distance
ncomp	Int	FALSE	1	5	988	NA

C. Must be strictly positive." [PVG+11]. For XGBoost, the learning rate "shrinks the contribution of each tree [...]. There is a trade-off between learning_rate and n_estimators. Values must be in the range [0.0, inf)" [PVG+11]. The n_estimators steers the maximum number of trees in the ensemble to balance computational burden with discriminative performance, and max_depth to steer the maximum depth of each tree to prevent overfitting and ensure good generalization performance. In the k-NN algorithm, the n_neighbors determines the size of the neighborhood for the determination of similarity-based class membership of a sample. The weights parameter determines if neighbors in the k-NN classifier are weighted uniformly or by distance. Last but not least, the ncomp present across all classification algorithms determines the number of components to keep from the dimensionality reduction step (independent components, principal components). It steers the level of aggressiveness of the dimensionality reduction step.

Hyperparameter for synthetic sample generation experiments

The hyperparameters for the sample generation experiments are listed in Table 3.3. All hyperparameters specific to the classification algorithms are as described already in the above paragraph. The additional hyperparameters that appear in the table have the following meaning:
The min_leaf_purity and max_leaf_purity parameters relate to the novel Random Forest-

based Synthetic Minority Oversampling Technique described in Section 2.4. They steer the purity requirement a leaf has to fulfill to be allowed to be taken into consideration when counting the co-occurrence of samples in leaves to determine their neighborhood relationship. Higher minimum purity values allow neighborhood relationships only to be established in rather pure areas of the feature space without much noise and class overlap. Lower levels of minimum purity would also allow neighborhood relationships to be established in more contested areas of the feature space (e.g. in the boundary region of the classes in case of class overlap). The max_leaf_purity is always set at least 10 % over the min_leaf_purity to make sure at least some leaves qualify at all. It can be set to establish an upper bound on leaf purity if one were to be interested in generating samples exclusively in contested areas of the feature space. The top_pair_fraction hyperparameter steers which percentage fraction of the closest neighboring pairs should be selected for the purpose of interpolation to generate new synthetic samples. The os_max_depth parameter limits the maximum depths of the trees in the Random Forest of the novel Random Forest-based Synthetic Minority Oversampling Technique to prevent overfitting to the training data and ensure good generalization performance. The number of trees was hard-coded to 500 for the experiments to limit the number of hyperparameter options one has to explore, but this value could also be optimized if need be. Hyperparameters of other sample generation methods are as follows: The smote_k value corresponds to the hyperparameter k in the original SMOTE algorithm as described in Section 2.1.1. It steers the size of the neighborhood that is taken into account when choosing potential candidate samples to interpolate in between for the sake of generating new samples. It is used in all other SMOTE variants we explored for the same purpose. The hyperparameter m_neighbors is used in the BORDERLINE-SMOTE and the SVM-SMOTE algorithm to determine the number of neighboring samples to be used to determine if the sample itself is located in the borderline region. This is an independent hyperparameter from the k_neighbors to select the interpolation candidates (which are usually restricted to belong to the minority class).

Table 3.3: hyperparameter distributions by study for sample generation experiments

param_name	dist_name	log	step	low	high	choices
Logistic Regression						
C	Float	TRUE		1e-04	100	NA
smote_k	Int	FALSE	1	2	10	NA
smote_m	Int	FALSE	1	5	20	NA
min_leaf_purity	Float	FALSE		0.5	0.8	NA
max_leaf_purity	Float	FALSE		min_leaf_purity +0.1	1	NA
top_pair_fraction	Float	FALSE		0.05	0.1	NA
os_max_depth	Int	FALSE	1	4	10	NA
Naive Bayes						
var_smoothing	Float	TRUE		1e-11	1e-07	NA
smote_k	Int	FALSE	1	2	10	NA
smote_m	Int	FALSE	1	5	20	NA
min_leaf_purity	Float	FALSE		0.5	0.8	NA
max_leaf_purity	Float	FALSE		min_leaf_purity +0.1	1	NA
top_pair_fraction	Float	FALSE		0.05	0.1	NA
os_max_depth	Int	FALSE	1	4	10	NA
Random Forest						

n_estimators	Int	FALSE	1	100	1000	NA
max_depth	Int	FALSE	1	2	20	NA
smote_k	Int	FALSE	1	2	10	NA
smote_m	Int	FALSE	1	5	20	NA
min_leaf_purity	Float	FALSE		0.5	0.8	NA
max_leaf_purity	Float	FALSE		min_leaf_purity +0.1	1	NA
top_pair_fraction	Float	FALSE		0.05	0.1	NA
os_max_depth	Int	FALSE	1	4	10	NA
SVM (Linear)						
C	Float	TRUE		1e-04	100	NA
smote_k	Int	FALSE	1	2	10	NA
smote_m	Int	FALSE	1	5	20	NA
min_leaf_purity	Float	FALSE		0.5	0.8	NA
max_leaf_purity	Float	FALSE		min_leaf_purity +0.1	1	NA
top_pair_fraction	Float	FALSE		0.05	0.1	NA
os_max_depth	Int	FALSE	1	4	10	NA
XGBoost						
learning_rate	Float	FALSE		0.01	0.3	NA
max_depth	Int	FALSE	1	3	10	NA
n_estimators	Int	FALSE	1	100	1000	NA
smote_k	Int	FALSE	1	2	10	NA
smote_m	Int	FALSE	1	5	20	NA
min_leaf_purity	Float	FALSE		0.5	0.8	NA
max_leaf_purity	Float	FALSE		min_leaf_purity +0.1	1	NA
top_pair_fraction	Float	FALSE		0.05	0.1	NA
os_max_depth	Int	FALSE	1	4	10	NA
k-NN						
n_neighbors	Int	FALSE	1	3	30	NA
weights	Categorical	NA	NA	NA	NA	uniform, distance
smote_k	Int	FALSE	1	2	10	NA
smote_m	Int	FALSE	1	5	20	NA
min_leaf_purity	Float	FALSE		0.5	0.8	NA
max_leaf_purity	Float	FALSE		min_leaf_purity +0.1	1	NA
top_pair_fraction	Float	FALSE		0.05	0.1	NA
os_max_depth	Int	FALSE	1	4	10	NA

3.9 Metrics and evaluation criteria

Based on the confusion table (c.f. Table 3.4) that captures the various true and false classification rates

		Predicted	
		Positive	Negative
Actual	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

Table 3.4: Confusion matrix showing classification outcomes

One can define a plethora of different classification performance metrics, some of which are listed below [JK19]:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Error Rate} = 1 - \text{Accuracy} = \frac{FP + FN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \text{Sensitivity} = \text{TPR} = \frac{TP}{TP + FN}$$

$$\text{Selectivity} = \text{Specificity} = \text{TNR} = \frac{TN}{TN + FP}$$

$$F_1\text{-measure} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$G\text{-mean} = \sqrt{\text{Recall} \times \text{Selectivity}}$$

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Selectivity}}{2}$$

$$\text{Cohen's } \kappa = \frac{(TP + TN) - \frac{(TP+FN)(TP+FP)+(FP+TN)(FN+TN)}{(TP+TN+FP+FN)^2}}{1 - \frac{(TP+FN)(TP+FP)+(FP+TN)(FN+TN)}{(TP+TN+FP+FN)^2}}$$

Unfortunately, the majority of these metrics are not well-suited for imbalanced datasets, as they would not capture how well the minority class (which is oftentimes the more interesting class) is classified. They would be dominated by the performance over the majority class (negative class) [JK19]. That said, some of these measures are more suited to imbalanced learning than others. The Matthews Correlation Coefficient (MCC) [Mat75] takes all four confusion matrix values into account and is therefore useful even for imbalanced datasets. It can be understood as a correlation coefficient between the predicted and actual classes. Cohen's Kappa [Coh60] also uses all entries of the confusion matrix and tries to capture the chance-adjusted proportion of correctly classified instances. It is well-suited for imbalanced learning as the imbalance is taken into account when correcting for getting the classification right by chance. The F_1-measure tries to strike a balance between Precision and Recall. Precision alone doesn't capture the number of false negatives, so an algorithm could pick very few easy positive examples and look good. Recall takes the opposite point of view and considers how many positive examples were "left on the table" and classified as negative instances. Combining the two measures tries to capture both aspects, and there is usually a trade-off between Precision and Recall. G-mean and balanced accuracy, on the other hand, try to strike a balance between Recall and Specificity to ensure that the correct classification of the positive and negative classes is taken into account. For the last 3 above-mentioned measures, one can shift the emphasis to one or the other of the elementary measures that make up the compound measures by modifying the decision threshold for turning classification scores into class decisions and class labels. This points to one weakness of the above measures, namely that it is assumed such a threshold has been predetermined (either as a heuristic of e.g. 0.5 or through optimization, e.g., of known real-world misclassification

costs). Therefore, these metrics by themselves don't allow for comparing algorithms and models for a large variety of decision thresholds at once.
For that reason the Receiver-Operating-characteristics curve (ROC) [Spa89] [PF99] was popularized to compare the performance of different classifier across a multitude of decision thresholds by plotting the FPR[6] (x-axis) against the TPR[7] (y-axis) for the whole range of decision thresholds from 0 to 1 (c.f. Figure 3.4 and Equation 3.1 or the discrete form in Equation 3.2). If one takes the area under the ROC (AUROC or AUC), one can condense the performance of a classifier across a multitude of possible decision thresholds into one single number. A particular classifier might perform better at a particular decision threshold than another, but the AUROC integrates over all decision threshold performances. If one were to approach the problem with a cost sensitive classification approach (or what [FSS+25] calls a consequentialist view) in mind, one would chose the decision threshold at a value that optimizes Precision and Recall at the optimum real world costs of the various misclassifications and decide between classifiers based on which one has the lowest overall misclassification cost (after per-classifier optimized decision thresholds are applied).
If one has no ex-ante information about what misclassification costs are involved and therefore suffers from what [FSS+25] calls "threshold uncertainty", then the AUROC can be a choice as it integrates over the performance at all possible decision thresholds uniformly without preference.

$$\text{AUROC} = \int_{t=0}^{1} \text{TPR}(t) \cdot \frac{d\,\text{FPR}(t)}{dt}\, dt \tag{3.1}$$

$$\text{AUROC} = \sum_{k=1}^{n-1} (\text{FPR}_{k+1} - \text{FPR}_k) \cdot \frac{\text{TPR}_{k+1} + \text{TPR}_k}{2} \tag{3.2}$$

Another popular measure, which is oftentimes promoted for applications in the context of class imbalance, is the so-called area-under-the precision-recall curve (AUPRC) [MZH+24], which, analogous to the AUROC plots the recall (x-axis) against the precision (y-axis) for a multitude of classification decision thresholds between 0 and 1 (c.f. Figure 3.4 and Equation 3.3 or the discrete form in Equation 3.4).

$$\text{AUPRC} = \int_{t=0}^{1} \text{Precision}(t) \cdot \frac{d\,\text{Recall}(t)}{dt}\, dt \tag{3.3}$$

$$\text{AUPRC} = \sum_{k=1}^{n-1} (\text{Recall}_{k+1} - \text{Recall}_k) \cdot \text{Precision}_{k+1} \tag{3.4}$$

It was shown in [MZH+24] that AUPRC is not, in general, superior to AUROC in a class imbalance setting. It might introduce unwanted bias by overly favoring models or model improvements in subpopulations of the dataset with more frequent positive labels. While AUROC weighs all false positives equally, AUPRC weighs false positives with a higher classification score more than those with a lower classification score. Maybe that is a desired bias in some settings, but we did not want to introduce this bias as part of our experiments and hence did not use the AUPRC as an evaluation metric. Ultimately, [MZH+24] also takes a consequentialist view and promotes the use of evaluation measures that align with the cost of misclassification (be it false positives or false negatives). In the absence of such a preference, the AUROC is a relatively neutral middle ground.
That said, the use of AUROC for the purpose of model comparison is not without criticism either. [Han09] argue and show that the AUC implicitly averages over classifier-dependent

[6] false positive rate
[7] true positive rate

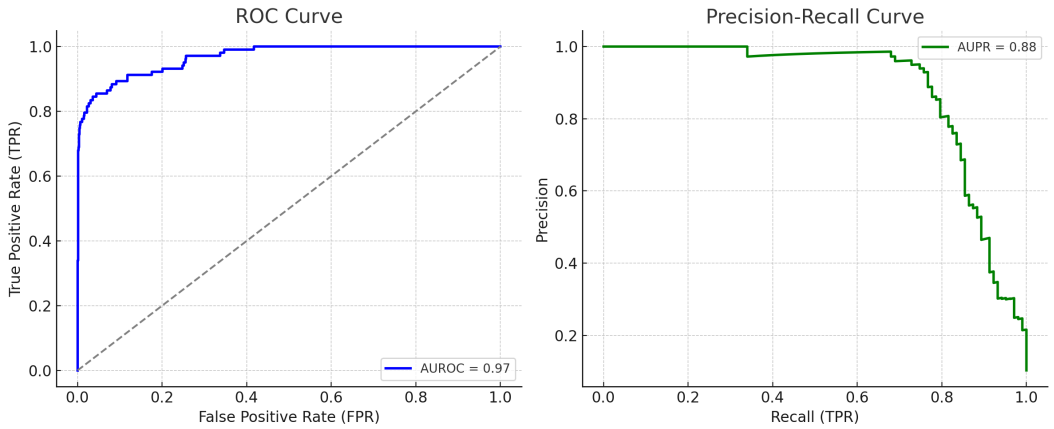

Figure 3.4: Illustration of ROC and PR curves

hypothetical cost distributions dependent on the distribution of classification scores (which are by definition dependent on each classifier). As the AUC sums over the various decision thresholds, which (in a discrete scenario) are dictated by the classification scores from the classifier and usually vary across classifiers for the same dataset, they argue that the classifiers themselves determine (implicitly) what decision thresholds are iterated over. [Han09] [HA13] find it incoherent that the method of calculation of a measure to compare classifiers is partly dependent on the output of the classifiers themselves (namely, the distribution of class scores and therefore also thresholds). Both [HA13] and [FHOF11] state that AUROC is an appropriate approach for assessing and comparing the quality of ranking instances across different classifiers. For example, in a scenario where limited resources are available, and one needs to select the top X samples in terms of their classification score, they find the use of AUROC for the purpose of model comparison justified. Two alternative metrics for model comparison in a binary classification context are the Brier Score [Bri50] and the Log-loss (cross entropy) [FSS+25]

$$\text{Brier Score} = \frac{1}{n}\sum_{i=1}^{n}\left(\hat{p}_i - y_i\right)^2 \tag{3.5}$$

$$\text{Log Loss} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{p}_i) + (1-y_i)\log\left(1-\hat{p}_i\right)\right] \tag{3.6}$$

n : Number of observations/samples
$y_i \in \{0,1\}$: True label of the i-th observation (binary)
$\hat{p}_i \in [0,1]$: Predicted probability that observation i belongs to class 1 (binary)

These measures are of particular interest when the estimation of true class probabilities actually matters. E.g., the true probability of having a certain cancer (based on the feature vector). Unfortunately (as mentioned in Chapter 1), most classification algorithms don't output well-calibrated class probabilities by default [BMWX21] [NMC05] [Bos08] [GPSW17]. They would therefore potentially score poorly on the above metrics for a combination of factors, namely actual cases of misclassification as well as biases in the classification scores (like over-confidence). One would therefore have to subject the outputs of each classifier to one of many possible calibration algorithms (c.f. [NMC05] [Pla99] [ZE02, ZE01] [RWD88] [Bos08]

[GPSW17]) to be able to assess their true classification performance with the Brier-Score or Log-Loss. This would introduce one more ”moving part” or source for errors into the classification pipeline. Furthermore, [CLH+25] has shown that sample generation methods (subject of this research) introduce changes to the structure or distribution of the classification scores that are hard to correct for, even with calibration methods (particularly for logistic regression). It would therefore be ill-advised to use evaluation metrics that are sensitive to properly calibrated class probabilities (like the Brier-Score or the Log-Loss) to compare the performance of classification pipelines that include sample generation steps. It is for that reason that, for the purpose of this research project, the AUROC was chosen as the final performance metric. Given the health sector nature of the dataset (cancer diagnosis based on gene expression data), it is noteworthy that, according to [FSS+25], the AUROC dominates as a comparison measure in publications in the healthcare sector. In the bigger scheme of things, we considered the objections raised in [Han09] and [HA13] as the least of all shortcomings. An in-depth discussion of evaluation metrics for the interested reader can be found in [JTM25].

Chapter 4

Results

Key findings: Dimensionality reduction produced mixed results and often failed to generalize from inner to outer cross-validation. RandomForestSMOTE achieved the highest aggregate AUROC among oversampling methods, but most performance differences were small and highly dependent on the classifier and dataset.

4.1 Result sets

Table 4.1 tries to illustrate the different result sets that were created by our experiments. As we used a cloud compute spot instance (which can be interrupted by the cloud compute provider at any time during a demand surge), we had to deal with several instances with interrupted Optuna studies that did not manage to get through all 300 hyperparameter experiments. We restarted the hyperparameter optimization in these cases from scratch. For the above-mentioned reason, the following study names had to be excluded from the result set as they were related to corrupted, unfinished Optuna hyperparameter studies: OVA_ProstateXGBoostNoneRandom0588508, OVA_OmentumXGBoostNoneRandom0588508, OVA_OmentumXGBoostNoneRFsmote083667, OVA_OmentumXGBoostNoneRFsmote1896865, OVA_OmentumXGBoostNoneRFsmote2244098.

4.2 Dimensionality reduction

The first set of experiments we conducted was to quantify the benefit of dimensionality reduction methods in their own right for classification problems with high-dimensional feature sets and the presence of class imbalance. Without combining any dimensionality reduction methods with the use of SMOTE, we wanted to see if these methods by themselves are hurtful or beneficial, and if so, which method might be best.

4.2.1 Aggregated results - inner cross-validation

Key takeaway: All dimensionality reduction methods produced small average improvements in inner-cross-validation AUROC, with ICA performing best overall. However, the magnitude and direction of the effect varied substantially across classifiers and datasets.

Outer cross-validation	**inner cross-validation**
Iteration 1: AUC on test fold using best hyperparameter settings identified by inner cross-validation	average AUC over 5 iterations of inner cross-validation (using best hyperparameter setting from 300 experiments)
Iteration 2: AUC on test fold using best hyperparameter settings identified by inner cross-validation	average AUC over 5 iterations of inner cross-validation (using best hyperparameter setting from 300 experiments)
Iteration 3: AUC on test fold using best hyperparameter settings identified by inner cross-validation	average AUC over 5 iterations of inner cross-validation (using best hyperparameter setting from 300 experiments)
Iteration 4: AUC on test fold using best hyperparameter settings identified by inner cross-validation	average AUC over 5 iterations of inner cross-validation (using best hyperparameter setting from 300 experiments)
Iteration 5: AUC on test fold using best hyperparameter settings identified by inner cross-validation	average AUC over 5 iterations of inner cross-validation (using best hyperparameter setting from 300 experiments)
$= 5 \times$ AUC	$= 5 \times$ average AUC

Table 4.1: Result types for experiments

Table 4.2: Best mean AUC (5-fold inner cross-validation) and percentage improvement of each dimensionality reduction method over the full dimensional datasets (averaged over all datasets and classifier)

dimred	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
ICA	0.9526	0.9433	0.0093	1.1180
PCA	0.9499	0.9433	0.0066	0.8324
SVD	0.9498	0.9433	0.0065	0.8287

We can see in Table 4.2 that on average, all methods provide a very small benefit (in terms of AUC) compared to a classification pipeline without any dimensionality reduction. The ICA is leading in front of the PCA and SVD. That said, from the detailed results in Annex A.1.1, we can see that a more nuanced view is warranted, as the reported averaged results can change depending on the dataset or classification algorithm. For the Omentum dataset, for example, the AUC (averaged over all classifiers) showed that PCA and SVD were actually slightly harmful compared to a baseline without dimensionality reduction applied (c.f. Table A.2). All dimensionality reduction methods were also harmful for all ensemble classifiers (Random Forest and Gradient Boosted Decision Trees), as can be seen in Table A.3. In most cases, the benefit of these methods was small, with the exception when applied in combination with the Naive Bayes classifier, which gets to hugely benefit from them.

4.2.2 Aggregated results - outer cross-validation

Key takeaway: Out-of-sample results were expectedly weaker than the inner-cross-validation results. Only PCA and SVD remained beneficial on average, while ICA changed from the best-performing method in the inner cross-validation to a harmful method in the outer cross-validation. Magnitude and direction of the effect varied again substantially across classifiers and datasets.

Table 4.3: Mean AUC (5-fold outer cross-validation) aggregated by dimensionality reduction method across all datasets and classifiers

dimred	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
SVD	0.9431	0.9406	0.0025	0.2708
PCA	0.9428	0.9406	0.0022	0.2392
ICA	0.9352	0.9406	-0.0054	-0.5755

The results in comparison to the above section change quite a bit once we look at the results from the outer cross-validation (out-of-sample with regard to optimized hyperparameters) in Table 4.3. We can see that only 2 out of 3 dimensionality reduction methods, namely SVD and PCA, still provide a benefit. That said, the magnitude of the AUC improvement is considerably smaller (around a quarter of a percentage point). Even more surprising is that the ICA, which had the best performance according to the inner cross-validation AUCs, turns out to be harmful to the AUCs of the outer cross-validation. One could argue that the hyperparameters were potentially overfit during the inner cross-validation (despite the fact that cross-validation was used to minimize that risk). From the detailed results in Annex A.1.2, we can see that again a more nuanced view is warranted, as the reported averaged results can vastly change depending on the dataset or classification algorithm. For the Omentum dataset, again, the AUC (averaged over all classifiers) showed that dimensionality reduction methods were harmful compared to a baseline without dimensionality reduction applied (c.f. Table A.6). This time, the effect was even more pronounced and included all dimensionality reduction methods (with ICA being the most harmful). Dimension reduction methods were again harmful for all ensemble classifiers (Random Forest and Gradient Boosted Decision Trees), but also for the SVM and k-NN classifiers, as can be seen in Table A.7. Basically, the beneficial effect of SVD and PCA in the aggregated results of Table 4.3 is almost exclusively driven by positive results for the Naive Bayes classifier and better relative performance than the ICA for the k-NN classifier, and also for the Omentum dataset.

4.2.3 Hyperparameter observations

Key takeaway: No clear universal dimensionality reduction configuration emerged. Optimal numbers of retained dimensions varied widely across datasets and classifiers and were often surprisingly large.

A detailed breakdown and analysis of the hyperparameter optimization results can be found in Annex B.1. It can be noted that the optimal number of dimensions, as determined by the hyperparameter optimization, covers quite a large range, from 6 dimensions in the case of the Endometrium dataset combined with the Random Forest and SVD, all the way to 988 dimensions in the case of the Prostate dataset combined with XGBoost and SVD. Otherwise, the detailed hyperparameter results don't contribute meaningfully to answering the research question of this report. We decided to report them anyway in Annex B.1, as they are available as a byproduct of our experiments, and may be of interest to some readers or help with the replicability of our results and findings.

4.2.4 Comments and conclusion for dimensionality reduction experiments

Key takeaway: Dimensionality reduction cannot be recommended as a universally beneficial preprocessing step. Benefits and harms depend strongly on the classifier and dataset,

and performance rankings obtained during model selection frequently failed to transfer to out-of-sample data.
The above results showed that applying dimensionality reduction methods is not always beneficial and can actually harm the performance of classifiers measured in AUC. The variability of the benefit or harm of applying dimensionality reduction methods across combinations of datasets and classifiers is apparent in Table A.4 and A.7. There are some clear winners like the Endometrium dataset or the Naive Bayes classifier, but overall, there is little benefit and quite some harmful cases, particularly for decision tree ensembles. It appears that the relative overall performance ranking of the various dimensionality reduction methods is often not driven by stellar positive effect across many or even few scenarios, but frequently by an absence of substantial performance degradation in a few scenarios where competing methods utterly fail.
There is no single dimensionality reduction method that dominates all other methods across classifiers or across datasets. The different relative performance results of the various methods between the inner cross-validation performance and the outer cross-validation performance also gave reason for concern. If the dominant dimensionality reduction method from the inner cross-validation (model selection and hyperparameter tuning for a particular dataset and classifier) is not also the dominant method on a test set from the outer cross-validation, then choosing a dimensionality reduction method with the mechanism of an inner cross-validation is not a reliable choice for out-of-sample data and makes such an approach rather ineffective.
But even if the relative performance results from the inner cross-validation were to carry over to unseen data, it would constitute a heavy computational burden to empirically select the best dimensionality reduction method (incl. its hyperparameters) for each classification problem (e.g. dataset $\times$ classifier combination) individually. We couldn't even establish a dominant dimensionality reduction method for a single domain like gene expression data.
From Section 4.2.3 we can see that in many cases (e.g. Logistic Regression, SVM, k-NN) a very high number of independent or principal components are chosen as the optimal hyperparameter. Frequently, several hundred or close to a thousand components are chosen as the best setting. While this is still better than the original ten thousand features, it is still a very high number of features.
In summary, even when one avoids combining dimensionality reduction methods with synthetic sample generation methods (with an unresolved question about in which order to apply the two steps), it turns out there is quite a range of problems when using dimensionality reduction methods in their own right:

- large variability of benefit/harm, depending on datasets and classifier combination
- good overall relative performance of a method across all scenarios, oftentimes more a result of being less harmful in extreme cases than competing methods than a consistent, strong benefit across a wide range of datasets and classifiers.
- best dimensionality reduction methods for each dataset and classifier combination to be determined empirically (computationally expensive)
- results from hyperparameter optimization (incl. choice of optimal dimensionality reduction method) don't carry over well to out-of-sample data
- in quite some cases, the optimal number of dimensions is still very high (in the order of magnitude of the number of samples)

4.3 Synthetic sample generation

In the second set of experiments, we conducted we tested whether SMOTE variants that are not specifically optimized for high-dimensional datasets lead to an AUC performance improvement or decline compared to a baseline of imbalanced data without synthetic sample generation. We also tested the performance of our novel Random Forest-based Synthetic Minority Oversampling Technique, which we described in Section 2.4. Last but not least, we also looked at the performance of simple Random Oversampling.

4.3.1 Aggregated results - inner cross-validation

Key takeaway: RandomForestSMOTE achieved the highest average inner-cross-validation AUROC, closely followed by Random Oversampling. The average gains over the baseline were positive but small. However, the magnitude and direction of the effect varied substantially across classifiers.

Table 4.4: Best mean AUC (5-fold inner cross-validation) and percentage improvement of each sample generation method over the baseline (no sample generation), averaged over all datasets and classifier

oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
RandomForestSMOTE	0.9462	0.9433	0.0029	0.3053
RandomOverSampler	0.9454	0.9433	0.0021	0.2248
SVMSMOTE	0.9438	0.9433	0.0005	0.0549
BorderlineSMOTE	0.9392	0.9433	-0.0041	-0.4335
SMOTE	0.9390	0.9433	-0.0043	-0.4516
ADASYN	0.9387	0.9433	-0.0046	-0.4824

Table 4.4 shows the AUC results from the inner cross-validation for the various oversampling techniques averaged over all classifiers and datasets compared to a baseline result with no use of oversampling techniques. We can see that our RandomForestSMOTE technique leads by a small margin over Random Oversampling. Both methods provide a small benefit compared to the baseline, with all other SMOTE methods (except SVMSMOTE) being slightly harmful. The effect sizes (positive or negative) are much smaller, though, than those of the dimensionality reduction experiments. The more detailed results breakdown in Annex A.2.1 shows some consistency of the summary results as well if broken down by datasets (c.f. Table A.10). The Random Oversampling and the RandomForestSMOTE method both land consistently in the top two (like in the summary results). The RandomForestSMOTE method scores the best result twice and the Random Oversampling method once. The RandomForestSMOTE algorithm is the only method that has a beneficial effect on all 3 datasets (averaged over all classifiers) and could therefore be considered a good choice that adapts well to the particularities of the various datasets. ADASYN, SMOTE, and BorderlineSMOTE are consistently harmful across all datasets (averaged over all classifiers). The breakdown of results by classifier (averaged over all datasets) in Table A.11, on the other hand, reveals that no single method dominates all other methods for all classifiers. RandomForestSMOTE performed best for Logistic Regression and k-NN (even though it was harmful for k-NN). Random Oversampling did best for Random Forest and Naive Bayes (being the only method not harmful for Naive Bayes), and SVMSMOTE took the first spot for SVM

and XGBoost. It is noteworthy (in the context of the overall good average performance of Random Oversampling and RandomForestSMOTE) that both methods were the only harmful sample generation algorithms for the XGBoost classifier. Furthermore, it can be noted that oversampling methods do more harm than help for Naive Bayes (with the exception of the Random Oversampling), as well as the k-NN classifier (where all sample generation algorithms were harmful). The effect size of the harmful cases (Naive Bayes and k-NN) is much bigger than the effect size of the beneficial cases (e.g. Logistic Regression). Which turns this comparison again[1] more into a question of which methods cause the least harm in the really bad scenarios rather than which method outperforms all others with strong positive effects across a wide range of scenarios.

4.3.2 Aggregated results - outer cross-validation

Key takeaway: RandomForestSMOTE again achieved the highest average out-of-sample AUROC and was one of only two methods with a positive average effect. However, its advantage over Random Oversampling was small and did not consistently hold across datasets and classifiers. The magnitude and direction of the effect for all methods varied substantially across classifiers and datasets.

Table 4.5: Average AUC (outer 5-fold cross-validation) and percentage improvement of each sample generation method over the baseline (no sample generation) aggregated by sample generation method across all datasets and classifier

oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
RandomForestSMOTE	0.9419	0.9406	0.0013	0.1374
RandomOverSampler	0.9410	0.9406	0.0004	0.0420
SVMSMOTE	0.9404	0.9406	-0.0002	-0.0233
SMOTE	0.9361	0.9406	-0.0045	-0.4772
ADASYN	0.9350	0.9406	-0.0056	-0.5924
BorderlineSMOTE	0.9324	0.9406	-0.0081	-0.8662

Remark: The observed AUROC differences were not tested for statistical significance and should therefore be interpreted descriptively rather than as evidence of a statistically reliable ordering of methods.

Table 4.5 shows the AUC results from the outer cross-validation for the various oversampling techniques averaged over all classifiers and datasets compared to a baseline result with no use of oversampling techniques. Some important results from the inner cross-validation (c.f. Table 4.4) carry over to these out-of-sample results. Namely, the RandomForestSMOTE algorithm is again, on average, the best performing sample generation algorithm, followed (again) by the Random Oversampling approach. The positive effect size in both cases is smaller, though, which might be expected on a true out-of-sample dataset. Both these algorithms are the only two methods that exhibit a positive effect in aggregate over all datasets and classifiers. SVMSMOTE comes in third (as was the case for the inner cross-validation results), but it is already slightly harmful, as are all other remaining methods. A more nuanced perspective is offered by the detailed result tables in Annex A.2.2. The RandomForestSMOTE method is the best performing method for the Endometrium and Omentum dataset, but comes in only at fourth place for the Prostate dataset (even behind the standard SMOTE method). Furthermore, all methods turn out to be harmful for the Omentum

[1] as was the case with the dimensionality reduction experiments

dataset (with the RandomForestSMOTE being the least harmful) (c.f. Table A.14). The per-classifier AUC results from the outer cross-validation, as shown in Table A.15, also have some interesting insights to offer. All sample generation methods are harmful for the k-NN classifier, and (except for the RandomForestSMOTE and Random Oversampling), also for the Naive Bayes classifier. The RandomForestSMOTE algorithm shows quite a variety in ranking across the different classifiers. Best for Naive Bayes, second best for k-NN and XGBoost, but second last for SVM and Logistic Regression, and last for the Random Forest classifier. So the first spot in the overall aggregated results across all classifiers and datasets is not driven by a consistent out-performance of all other algorithms across all scenarios, but rather by some very strong relative performance in a few scenarios (e.g. Naive Bayes and k-NN, where most other methods are extremely harmful). It's also worth mentioning that the relative performance in comparison to other sample generation methods is not consistent between the results of the inner cross-validation and the outer cross-validation. For example, the RandomForestSMOTE algorithm performed rather poorly on the XGBoost classifier in the inner cross-validation but comparatively well in the outer cross-validation. It is the other way round for the SVM classifier. This poses a problem in the sense that it calls into question the hyperparameter optimization (including the selection of the optimal sample generation algorithm) in an inner cross-validation, if the results (incl. relative performance ranking) are not transferable to out-of-sample data.

4.3.3 Hyperparameter observations

Key takeaway: The preferred RandomForestSMOTE hyperparameter settings varied across classifiers, suggesting that no single configuration is likely to perform best across all classification problems.

A detailed breakdown and analysis of the hyperparameter optimization results can be found in Annex B.2. The detailed hyperparameter results don't contribute meaningfully to answering the research question of this report. We decided to report them anyway in Annex B.2, as they are available as a byproduct of our experiments, and maybe they are of interest to some readers or help with the replicability of our results and findings. Maybe the main takeaway for our newly introduced RandomForestSMOTE algorithm is that at least some hyperparameter choices seem to depend on the classifier used. Some classifier seem to prefer rather pure leaves (e.g. Random Forest classifier or Naive Bayes) while other classifier seemingly preferred to include samples from the borderline region and settled for rather low purity thresholds of the leaves in the RandomForestSMOTE algorithm (e.g. Logistic Regression, SVM XGBoost), without forcing all leaves to be in that borderline region, but rather being very lenient in general about which leaves to be taken into account. This is an interesting finding in the sense that it shows there might not be a unique heuristic choice for this hyperparameter, and allowing this hyperparameter to exist and be determined empirically might be warranted. The maximum depth level for the trees of the RandomForestSMOTE algorithm often varied across the whole allowed parameter range (Logistic Regression, SVM, XGBoost), but some classifiers had a clear preference for shallow trees to be used (k-NN, Naive Bayes, Random Forest). Interestingly, the choice of what top fraction of the neighborhood list should be used for sample generation was rather variable across all classifiers, suggesting the best choice might be highly dependent on the particular fold of data used in any iteration. Only the Naive Bayes showed a clear preference for a large top_pair_fraction of the neighborhood matrix/list. The XGBoost algorithm rather consistently settled for the midpoint of the parameter range of this parameter. Again, the takeaway is that there might be no unique heuristic choice for this hyperparameter, and allowing this hyperparameter to exist and be determined empirically might be warranted (less so than in the case of the

minimum leaf purity parameter).

4.3.4 Comments and conclusion for oversampling experiments

Key takeaway: RandomForestSMOTE achieved the best aggregate performance among the oversampling methods evaluated, but the performance advantage was modest and highly scenario-dependent. Consequently, the results do not support a universal recommendation for its use across all datasets and classifiers.
The results of the above experiments confirmed that many traditional SMOTE variants are, on average, harmful when applied to high-dimensional gene expression datasets. These methods (on average) lag behind standard random sampling, as well as our newly introduced RandomForestSMOTE algorithm, which showed the best average results of all methods. That said, the overall averaged positive effect is far below half a percent AUC improvement, and it depends on the particular use case if the performance increase warrants the additional computational efforts. The detailed results showed several problems with our approach:

- the best choice of sample generation technique can be dataset dependent, but is certainly dependent on the choice of classifier.
- the best choice of sample generation method from a 5-fold inner cross-validation might not hold true out-of-sample (which defeats the efforts of running a cross-validation-based model selection).
- the aggregated good performance of the RandomForestSMOTE is not driven by a consistent out-performance of all other methods in all scenarios, but rather by some very strong relative performance in a few scenarios (e.g. Naive Bayes and k-NN), where most other methods are extremely harmful, which is then reflected accordingly in the overall aggregated performance.

Chapter 5

Conclusion & Future research

5.1 Conclusion

5.1.1 Main contributions

Our main contributions are as follows:

- we outlined the existing research gap for data-level methods in high-dimensional feature spaces
- we proposed a new Random Forest-based neighborhood measure based on the co-occurrence of samples in purity filtered leaves of a Random Forest, and proposed a novel sample generation algorithm based on the above proximity measure, as well as feature importance masked interpolation
- we conducted empirical experiments to assess the impact of dimensionality reduction methods (in their own right)
- we conducted empirical experiments to assess the impact of synthetic sample generation methods and to compare the performance of our novel method against established methods, which are suspected to underperform in high-dimensional feature spaces

5.1.2 Main findings

The main findings of our experiments are:

- neither for the dimensionality reduction nor the sample generation methods is there a single method that (out of sample) dominates all other methods across all classifier and dataset combinations.
- there are in both sets of experiments, a few combinations of dataset and classifier that strongly benefit or suffer from the application of dimensionality reduction or sample generation methods, which sit alongside a very large number of combinations that show hardly any or very little performance impact at all.
- in both sets of experiments, the relative performance (ranking) of dimensionality reduction and sample generation methods did not carry over from the inner cross-validation to the out-of-sample assessment in the outer cross-validation. One would expect the absolute performance to drop on out-of-sample data. But if the relative performance does not hold, then any model selection efforts inside the inner cross-validation are pointless. Either the performance difference inside the inner cross-validation was so small that there was no actual difference between some models and

methods, and the performance ranking would look different merely due to "sample errors" of new datasets. Or the hyperparameters were overoptimized (despite the mechanism of cross-validation trying to avoid that) inside the inner cross-validation, and therefore failed to carry over to out-of-sample data.

- the overall experimental summary results are heavily influenced by some extreme phenomena. For example, the Naive Bayes and k-NN classifiers showed severe performance degradation once combined with sample generation methods. In the case of the dimensionality reduction methods, Naive Bayes showed extreme benefit from dimensionality reduction methods, while XGBoost showed a strong performance decline, and one case of k-NN also showed a rather strong performance decline. The absence of some level of coherence in the results makes it difficult to give any advice on which methods are "best". One can compute summary results, but one needs to be aware that these results might be heavily driven by a few extreme cases. And a different selection of classifiers would certainly lead to different summary results. In any of the aggregated results (e.g. averaged over classifiers or datasets or both), the relative performance is often more a question of which methods caused the least harm in the really bad scenarios rather than which method outperforms all others with strong positive effects across a wide range of scenarios.

- in aggregate (averaged over all classifiers and datasets), our novel RandomForestSMOTE algorithm achieved the highest mean outer- and inner-cross-validation AUROC. That said, the performance difference over the second place (RandomOverSampling) as well as over the no-oversampling baseline was rather small, and the additional computational burden might not be justified in all circumstances. Furthermore, the good relative performance in the aggregated summary results very quickly breaks down when broken down by classifier, and to some degree also when broken down by dataset. In combination with the fact that the relative performance results of the inner cross-validation often don't carry over to out-of-sample performance, it is difficult to give advice when (or if) to use our new method. Some further research would be needed.

5.2 Limitations

Several limitations of this study should be considered when interpreting the results.

1. The experimental evaluation was conducted on a relatively small number of datasets. Although the datasets were selected to represent challenging high-dimensional, imbalanced gene expression classification tasks, the findings may not generalize to all domains or data types. Different conclusions may be obtained on larger collections of datasets, datasets with different imbalance ratios, or datasets originating from other application areas.

2. All datasets were previously pre-processed without any split in training and test data (with the top 20% of the highest variance features retained). Furthermore, the datasets contained calibration data and potentially batch effects. All of the above aspects can lead to data snooping and, therefore, ultimately overly optimistic absolute classification performance.

3. The study focused on a specific set of dimensionality reduction methods, oversampling techniques, and classification algorithms. While these methods represent commonly used approaches in the literature, many alternative methods exist. The relative

performance of RandomForestSMOTE and the other oversampling techniques may differ when combined with classifier methods not examined in this study or when some classifiers of this study are removed from the experiments and result sets. This aspect is particularly important because the on average good performance of RandomForestSMOTE is not necessarily due to a superior performance across many dataset-classifier combinations, but at least partly because it is less harmful in extreme cases where many other methods cause great performance degradation.

4. Classifier performance was evaluated using AUROC. However, other evaluation metrics may lead to different conclusions regarding the relative effectiveness of the competing methods.

5. Although nested cross-validation was used to reduce optimistic bias during model selection, performance estimates remain subject to sampling variability arising from the limited number of available observations in many gene expression datasets. Consequently, some observed differences between methods may be dataset-specific rather than universally applicable.

6. The proposed RandomForestSMOTE method was evaluated primarily with respect to predictive performance. The study did not investigate other potentially important characteristics, such as computational efficiency, scalability to very large datasets or the interpretability of the generated synthetic samples. These factors may influence the practical applicability of the method in real-world settings.

7. Hyperparameter optimization results are seemingly unstable. Method and model selection may be unreliable as the relative performance from inner cross-validation most of the time doesn't carry over to the results of the outer cross-validation. One possible explanation is that small performance differences in the inner cross-validation are mostly noise.

8. The observed effect sizes are rather small and may not be statistically significant or of practical significance. Computational cost may outweigh benefits.

Accordingly, the conclusions of this thesis should be interpreted as evidence that RandomForestSMOTE is a promising oversampling approach for the high-dimensional, imbalanced classification problems considered here, rather than as evidence of universal superiority across all classification settings.

5.3 Future research

- perform the same experiments as done in the report, but exclude gene expression calibration data (in case that creates batch effects and data snooping)
- Perform the experiments again on gene expression data of full dimensionality (close to 50 thousand features), as the existing data was globally pre-filtered to the 20% of features with the highest variance. Such pre-processing on the whole dataset constitutes some level of data snooping that will lead to absolute performance being overstated.
- try to get hold of a version of the datasets that include batch information to perform cross-validation with the help of GroupKFold or StratifiedGroupKFold
- try the experiments with other high-dimensional datasets to see if the results carry over to other domains outside of the realm of gene expression data

- perform a hyperparameter importance analysis for the hyperparameter of the RandomForestSMOTE method across various datasets and classifiers to see which hyperparameter could be set to a good heuristic default value without too much damage (to ease the computational burden). Reducing the number of hyperparameters might also help to improve the relationship between inner cross-validation results and ranking and out-of-sample performance, as one might be able to avoid overfitting hyperparameters. Based on our hyperparameter importance analysis, the max_leaf_purity seems to be a good candidate for a hyperparameter that might not be needed
- one could revisit and evaluate some of the design decisions of the RandomForestSMOTE method. Namely, the hard threshold of the top_pair_fraction. Instead, one could let all sample pairs generate new samples, but with a probability related to their proximity. There is also room to explore if sample pair selection should be deterministic or stochastic (based on their proximity) and in what mathematical form exactly (linearly proportional to their proximity or some non-linear relation to smooth out extreme cases?).
- One could experiment with other choices than the L1 norm to turn the 3rd dimension of the neighborhood tensor into a single neighborhood score. For example, an inverse L2 score could be used to penalize sample pairs that base their neighborhood on one single or very few features in many cases (leaves).
- try to run the experiments again using different evaluation and performance measures like the area-under-the precision-recall curve (AUPRC), or introduce the burden of calibrating the classification scores and use log loss or Brier score.
- compare the performance of the RandomForestSMOTE method with other SMOTE variants in low dimensional dataset to assess if it is a universally useful method or just shines in high-dimensional datasets
- if cheap computational resources are available it could be interesting to empirically answer the question if combining dimensionality reduction methods and SMOTE methods are useful and if so in which order these two steps are best to be run. There might not be a universally true answer, but establishing empirically that the optimal order of steps might depend on the dataset or classifier (or both) would be a valuable research finding.
- one could explore non-linear dimensionality reduction methods and supervised dimensionality reduction methods in future research if cheap computational resources are available.

Annex A

Detailed result tables

A.1 Dimensionality reduction

A.1.1 Inner cross-validation results

Summary performance by classifier

As can be seen in Table A.1, tree-based ensemble methods were the strongest classification algorithms in terms of AUC on the datasets without dimensionality reduction (mean_baseline_auc_rank). For the mean_auc (averaged over all dimensionality reduction methods), on the other hand, Logistic Regression surprisingly leads the field before SVM (c.f. mean_auc_rank).

Table A.1: Best mean AUC (5-fold inner cross-validation) for each classification algorithm, averaged across all datasets and dimensionality reduction methods with ranking.

algo	mean_auc	mean_baseline_auc	mean_auc_rank	mean_baseline_auc_rank
Logistic Regression	0.9602	0.9565	1	3
SVM	0.9562	0.9517	2	4
XGBoost	0.9519	0.9686	3	1
k-NN	0.9517	0.9498	4	5
Random Forest	0.9451	0.9586	5	2
Naive Bayes	0.9394	0.8748	6	6

Results broken down by dataset

Table A.2: Best mean AUC (5-fold inner cross-validation) and percentage improvement over baseline (no dimensionality reduction) by dataset and dimensionality reduction method, averaged over all classifiers

dataset	dimred	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium					
OVA_Endometrium	ICA	0.9481	0.9284	0.0197	2.4102
OVA_Endometrium	SVD	0.9453	0.9284	0.0168	2.0916
OVA_Endometrium	PCA	0.9451	0.9284	0.0167	2.0798

OVA_Omentum					
OVA_Omentum	ICA	0.9104	0.9095	0.0008	0.1897
OVA_Omentum	PCA	0.9084	0.9095	-0.0011	-0.0038
OVA_Omentum	SVD	0.9082	0.9095	-0.0014	-0.0284
OVA_Prostate					
OVA_Prostate	ICA	0.9993	0.9920	0.0074	0.7541
OVA_Prostate	SVD	0.9960	0.9920	0.0041	0.4227
OVA_Prostate	PCA	0.9960	0.9920	0.0041	0.4213

Table A.2 shows the benefit (or harm) from dimensionality reduction broken down by dataset and averaged over all classifiers. We can see that the Endometrium dataset gets to benefit most (ca. 2 percentage points) from the application of dimensionality reduction methods (averaged over all classification algorithms). The ICA is providing the most benefit. The Prostate dataset shows only a small or moderate (ca. half to three-quarters of a percentage point) improvement in AUC when dimensionality reduction methods are applied. The ICA is again the most beneficial method. To a degree, the reason might be that the baseline performance (AUC) on the Prostate dataset across all classifiers is already an impressive 0.9920, which is hard to improve upon. Lastly, the Omentum dataset shows little improvement (ICA) to slightly worse (PCA, SVD) AUC averaged over all classifiers.

Results broken down by classifier

Table A.3: Best mean AUC (5-fold inner cross-validation) and percentage improvement over baseline (no dimensionality reduction) by classification algorithm and dimensionality reduction method, averaged over all datasets

algo	dimred	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
Logistic Regression					
Logistic Regression	ICA	0.9613	0.9565	0.0048	0.5077
Logistic Regression	SVD	0.9602	0.9565	0.0037	0.3810
Logistic Regression	PCA	0.9593	0.9565	0.0028	0.2864
Naive Bayes					
Naive Bayes	SVD	0.9396	0.8748	0.0649	7.6941
Naive Bayes	PCA	0.9395	0.8748	0.0648	7.6777
Naive Bayes	ICA	0.9392	0.8748	0.0644	7.6357
Random Forest					
Random Forest	ICA	0.9513	0.9586	-0.0073	-0.7821
Random Forest	SVD	0.9421	0.9586	-0.0164	-1.7410
Random Forest	PCA	0.9419	0.9586	-0.0166	-1.7618
SVM					
SVM	ICA	0.9570	0.9517	0.0053	0.5753
SVM	PCA	0.9558	0.9517	0.0040	0.4284
SVM	SVD	0.9557	0.9517	0.0040	0.4243
XGBoost					
XGBoost	ICA	0.9525	0.9686	-0.0161	-1.6905
XGBoost	PCA	0.9522	0.9686	-0.0164	-1.7161

XGBoost	SVD	0.9510	0.9686	-0.0176	-1.8453
k-NN					
k-NN	ICA	0.9543	0.9498	0.0045	0.4619
k-NN	PCA	0.9505	0.9498	0.0007	0.0800
k-NN	SVD	0.9503	0.9498	0.0005	0.0588

Table A.3 breaks down the performance improvement or decline depending on the classification algorithm, and we can see the results are far from the same for all classifiers. Naive Bayes benefits most from the use of dimensionality reduction methods, with only small differences in the benefit between the various dimensionality reduction methods. The improvement is about 7.5 percent on the AUC, which is rather impressive. The SVD is the most beneficial method for Naive Bayes (contrary to the finding that ICA is the overall winner across all classifiers). Intuitively, Naive Bayes benefits most from the dimensionality reduction, as the algorithm has no inherent regularization or feature selection method and is therefore forced to take into account noisy or irrelevant features unless processing steps like dimensionality reduction methods lend a helping hand. There is a large group of classifiers that benefit to a small or moderate degree (around half a percentage point in AUC improvement) from the application of dimensionality reduction methods. Namely, Logistic Regression, SVM, and k-NN, with ICA being most beneficial in all cases. Interestingly enough, the benefits of SVD and PCA are measurably different for the k-NN, XGBoost, and Logistic Regression (even though the methods should be virtually identical for our z-score transformed dataset). The most likely explanation is that the hyperparameter optimization ended up in two different local minima that influenced the overall result. Last but not least, there is a small group of powerful ensemble classifiers that suffer from the application of dimensionality reduction methods. Namely, Random Forests and Gradient Boosted Decision trees, both of which show between 0.7 and 1.8 percentage points AUC degradation. ICA performed the least harmful for both classifiers. So part of the reason for ICA being the overall best performer across all classifiers is that it caused slightly less damage for the decision tree-based ensemble classifier. The result is somehow intuitive, as tree-based ensemble methods are able to perform some implicit supervised feature selection or feature weighting, and one would imagine this to lead to better performance than an unsupervised dimensionality reduction as a pre-processing step.

Results broken down by dataset, dimensionality reduction method, and classifier

Table A.4: Best mean AUC (5-fold inner cross-validation) and percentage improvement over baseline (no dimensionality reduction) by dataset, dimensionality reduction method, and classification algorithm, sorted by descending percentage improvement

dataset	dimred	algo	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium	ICA	Naive Bayes	0.9374	0.8191	0.1184	14.4500
OVA_Endometrium	PCA	Naive Bayes	0.9309	0.8191	0.1118	13.6496
OVA_Endometrium	SVD	Naive Bayes	0.9305	0.8191	0.1114	13.5999
OVA_Omentum	SVD	Naive Bayes	0.8907	0.8364	0.0542	6.4840
OVA_Omentum	PCA	Naive Bayes	0.8898	0.8364	0.0534	6.3819
OVA_Omentum	ICA	Naive Bayes	0.8809	0.8364	0.0444	5.3135
OVA_Prostate	ICA	Naive Bayes	0.9992	0.9688	0.0305	3.1437
OVA_Prostate	PCA	Naive Bayes	0.9978	0.9688	0.0291	3.0017
OVA_Prostate	SVD	Naive Bayes	0.9978	0.9688	0.0290	2.9985
OVA_Endometrium	SVD	Logistic Regression	0.9642	0.9458	0.0185	1.9508

OVA_Endometrium	PCA	Logistic Regression	0.9627	0.9458	0.0169	1.7910
OVA_Endometrium	ICA	Logistic Regression	0.9597	0.9458	0.0140	1.4764
OVA_Omentum	ICA	SVM	0.9201	0.9093	0.0107	1.1769
OVA_Prostate	ICA	k-NN	0.9994	0.9903	0.0090	0.9118
OVA_Endometrium	SVD	SVM	0.9560	0.9475	0.0085	0.8941
OVA_Endometrium	PCA	SVM	0.9555	0.9475	0.0080	0.8441
OVA_Endometrium	ICA	k-NN	0.9522	0.9479	0.0043	0.4516
OVA_Endometrium	ICA	SVM	0.9518	0.9475	0.0042	0.4458
OVA_Omentum	PCA	SVM	0.9128	0.9093	0.0035	0.3825
OVA_Omentum	SVD	SVM	0.9121	0.9093	0.0028	0.3051
OVA_Prostate	ICA	XGBoost	0.9996	0.9971	0.0025	0.2501
OVA_Omentum	PCA	k-NN	0.9129	0.9111	0.0018	0.1998
OVA_Omentum	SVD	k-NN	0.9121	0.9111	0.0010	0.1103
OVA_Prostate	ICA	SVM	0.9993	0.9983	0.0010	0.1032
OVA_Prostate	SVD	SVM	0.9990	0.9983	0.0007	0.0739
OVA_Prostate	ICA	Logistic Regression	0.9990	0.9983	0.0007	0.0663
OVA_Prostate	PCA	SVM	0.9989	0.9983	0.0006	0.0585
OVA_Endometrium	SVD	k-NN	0.9485	0.9479	0.0006	0.0584
OVA_Endometrium	ICA	Random Forest	0.9421	0.9416	0.0005	0.0576
OVA_Prostate	ICA	Random Forest	0.9995	0.9990	0.0005	0.0494
OVA_Endometrium	PCA	k-NN	0.9483	0.9479	0.0003	0.0339
OVA_Omentum	ICA	k-NN	0.9113	0.9111	0.0002	0.0223
OVA_Prostate	SVD	Logistic Regression	0.9985	0.9983	0.0002	0.0185
OVA_Prostate	SVD	k-NN	0.9904	0.9903	0.0001	0.0078
OVA_Prostate	PCA	k-NN	0.9904	0.9903	0.0001	0.0062
OVA_Prostate	PCA	Logistic Regression	0.9982	0.9983	-0.0001	-0.0136
OVA_Omentum	ICA	Logistic Regression	0.9252	0.9254	-0.0002	-0.0196
OVA_Prostate	PCA	XGBoost	0.9968	0.9971	-0.0003	-0.0269
OVA_Prostate	SVD	XGBoost	0.9964	0.9971	-0.0007	-0.0687
OVA_Prostate	SVD	Random Forest	0.9941	0.9990	-0.0049	-0.4936
OVA_Prostate	PCA	Random Forest	0.9941	0.9990	-0.0050	-0.4982
OVA_Omentum	SVD	Logistic Regression	0.9177	0.9254	-0.0076	-0.8264
OVA_Omentum	PCA	Logistic Regression	0.9169	0.9254	-0.0085	-0.9181
OVA_Endometrium	SVD	Random Forest	0.9279	0.9416	-0.0137	-1.4532
OVA_Endometrium	PCA	Random Forest	0.9274	0.9416	-0.0141	-1.5004
OVA_Endometrium	PCA	XGBoost	0.9461	0.9687	-0.0227	-2.3394
OVA_Endometrium	ICA	XGBoost	0.9453	0.9687	-0.0234	-2.4203
OVA_Omentum	ICA	Random Forest	0.9121	0.9351	-0.0229	-2.4534
OVA_Endometrium	SVD	XGBoost	0.9445	0.9687	-0.0242	-2.5001
OVA_Omentum	PCA	XGBoost	0.9138	0.9400	-0.0261	-2.7818
OVA_Omentum	ICA	XGBoost	0.9127	0.9400	-0.0273	-2.9013
OVA_Omentum	SVD	XGBoost	0.9121	0.9400	-0.0279	-2.9671
OVA_Omentum	SVD	Random Forest	0.9044	0.9351	-0.0306	-3.2761
OVA_Omentum	PCA	Random Forest	0.9043	0.9351	-0.0307	-3.2869

We can see in Table A.4 a vast range of performance improvement or decline depending on dataset, classifier, and dimensionality reduction methods. The ICA on the Endometrium dataset in combination with the Naive Bayes classifier leads the field with 14.45 % improvement on AUC, with similar results for PCA and SVD in the same setting. Nine cases out of the top 10 beneficial combinations involve the Naive Bayes classifier. Improvement magnitude quickly drops off afterwards, with a large number of datasets, dimensionality reduction methods, and classifier combinations with very minimal improvements. Towards the end of the spectrum, we find many combinations with a performance decline. Most of these cases involve the Random Forest and XGBoost classifier, but also quite some cases involving the Logistic Regression.

A.1.2 Outer cross-validation results

Summary performance by classifier

Interestingly, the results of Table A.5 show that on the test sets from the outer cross-validation, Logistic Regression leads the field for the average over all dimensionality reduction methods and datasets, but comes in 3rd for the baseline (no dimensionality reduction methods). XGBoost comes in second for the AUC averaged over all dimensionality reduction methods, but leads the field on the full-dimensional datasets. Surprisingly, Random Forest comes in on the fourth position (for the baseline as well as for the average over all dimensionality reduction methods). One would have expected a powerful ensemble method like Random Forests to outperform a rather simple algorithm like Logistic Regression. Maybe the hyperparameters of the Random Forest were overfit despite the use of cross-validation.

Table A.5: Mean AUC (5-fold outer cross-validation) per classifier (averaged over all dimred methods) with baseline (no dimred) AUC and ranks.

algo	mean_auc	mean_baseline_auc	mean_auc_rank	mean_baseline_auc_rank
Logistic Regression	0.9506	0.9505	1	3
XGBoost	0.9488	0.9650	2	1
SVM (Linear)	0.9479	0.9509	3	2
Random Forest	0.9380	0.9503	4	4
k-NN	0.9308	0.9499	5	5
Naive Bayes	0.9262	0.8770	6	6

Results broken down by dataset

Table A.6: Mean AUC (5-fold outer cross-validation) by dataset and dimensionality reduction method, averaged over all classifiers.

dataset	dimred	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium					
OVA_Endometrium	ICA	0.9436	0.9248	0.0188	2.0329
OVA_Endometrium	PCA	0.9423	0.9248	0.0175	1.8923
OVA_Endometrium	SVD	0.9406	0.9248	0.0158	1.7085
OVA_Omentum					
OVA_Omentum	SVD	0.8968	0.9110	-0.0142	-1.5587
OVA_Omentum	PCA	0.8950	0.9110	-0.0160	-1.7563
OVA_Omentum	ICA	0.8661	0.9110	-0.0449	-4.9286
OVA_Prostate					
OVA_Prostate	ICA	0.9958	0.9859	0.0099	1.0042
OVA_Prostate	SVD	0.9921	0.9859	0.0062	0.6289
OVA_Prostate	PCA	0.9913	0.9859	0.0054	0.5477

As can be seen in Table A.6, the overall picture of the outer cross-validation results is similar on a dataset-by-dataset basis to that of the results from the inner cross-validation. OVA_Endometrium

gets to benefit most from the application of dimensionality reduction methods. ICA is again the most beneficial method for this dataset. The beneficial effect is again slightly less for the OVA_Prostate dataset in comparison to the OVA_Endometrium dataset, and again, the ICA is the most beneficial method. For the OVA_Omentum dataset, the application of dimensionality reduction methods is again harmful, and the effect is even more pronounced in this result set. Interestingly enough, the ICA is now the most harmful method, while for the inner cross-validation result set, it still provided a small benefit and was ahead of the other dimensionality reduction methods. This might potentially indicate an overfitting of the hyperparameter. These results confirm that there is not one single universally beneficial dimensionality reduction method across the various datasets, and the best choice of the method would have to be empirically determined.

Results broken down by classifier

Table A.7: Mean AUC (5-fold outer cross-validation) by classifier (algo) and dimred method, averaged over all datasets.

algo	dimred	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
Logistic Regression					
Logistic Regression	SVD	0.9551	0.9505	0.0046	0.4840
Logistic Regression	PCA	0.9548	0.9505	0.0043	0.4524
Logistic Regression	ICA	0.9417	0.9505	-0.0088	-0.9258
Naive Bayes					
Naive Bayes	SVD	0.9275	0.8770	0.0505	5.7583
Naive Bayes	ICA	0.9269	0.8770	0.0499	5.6899
Naive Bayes	PCA	0.9243	0.8770	0.0473	5.3934
Random Forest					
Random Forest	ICA	0.9499	0.9503	-0.0004	-0.0421
Random Forest	SVD	0.9326	0.9503	-0.0177	-1.8626
Random Forest	PCA	0.9316	0.9503	-0.0187	-1.9678
SVM (Linear)					
SVM (Linear)	SVD	0.9487	0.9509	-0.0022	-0.2314
SVM (Linear)	PCA	0.9482	0.9509	-0.0027	-0.2839
SVM (Linear)	ICA	0.9467	0.9509	-0.0042	-0.4417
XGBoost					
XGBoost	SVD	0.9496	0.9650	-0.0154	-1.5959
XGBoost	PCA	0.9485	0.9650	-0.0165	-1.7098
XGBoost	ICA	0.9483	0.9650	-0.0167	-1.7306
k-NN					
k-NN	PCA	0.9496	0.9499	-0.0003	-0.0316
k-NN	SVD	0.9453	0.9499	-0.0046	-0.4843
k-NN	ICA	0.8976	0.9499	-0.0523	-5.5058

Table A.7 provides per classifier insight by summarizing the performance of the various dimensionality reduction methods per classifier in the outer cross-validation loop. In comparison to the results of the inner cross-validation, the Naive Bayes classifier again gets to benefit most from dimensionality reduction methods (again with the SVD leading the field),

with a slightly smaller effect size, though. Tree ensemble methods like Random Forest and Boosted Decision Trees are again suffering most from the application of dimensionality reduction methods. PCA is the most harmful for Random Forests, and ICA only has a very small negative effect. ICA is the most harmful for boosted decision trees, and all other methods are similarly bad. For the rest of the classifier, the results don't fully line up with the ones from the inner cross-validation. For Logistic Regression, ICA is harmful, but SVD and PCA are slightly beneficial. For SVM, all dimensionality reduction methods are slightly harmful, with ICA being the worst one. For k-NN, also all dimensionality reduction methods are harmful, but ICA is distinctly worse than all others, and k-NN with ICA has the worst performance dip (compared to a baseline with no dimensionality reduction) from all classifiers and dimensionality reduction method combinations. These results show once more that applying dimensionality reduction methods is not universally beneficial, and the choice of the specific method is a model selection problem. Unfortunately, our results show that the best relative performance from the inner cross-validation doesn't necessarily carry over to a hold-out test dataset (outer cross-validation), which makes the efforts of model selection via cross-validation a potentially ineffective endeavor.

Results broken down dataset, dimensionality reduction method, and classifier

Table A.8: Mean AUC (5-fold outer cross-validation) by dataset, classifier (algo), and dimred method, including baseline (no dimred).

dataset	dimred	algo	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium	PCA	Naive Bayes	0.9347	0.8091	0.1256	15.5220
OVA_Endometrium	SVD	Naive Bayes	0.9345	0.8091	0.1254	15.5032
OVA_Endometrium	ICA	Naive Bayes	0.9279	0.8091	0.1188	14.6851
OVA_Prostate	ICA	Naive Bayes	0.9987	0.9628	0.0358	3.7214
OVA_Prostate	SVD	Naive Bayes	0.9861	0.9628	0.0232	2.4109
OVA_Prostate	PCA	Naive Bayes	0.9859	0.9628	0.0230	2.3897
OVA_Endometrium	PCA	Logistic Regression	0.9654	0.9465	0.0189	1.9932
OVA_Endometrium	SVD	Logistic Regression	0.9637	0.9465	0.0172	1.8125
OVA_Endometrium	ICA	Random Forest	0.9500	0.9339	0.0161	1.7289
OVA_Prostate	PCA	Logistic Regression	0.9964	0.9821	0.0143	1.4518
OVA_Prostate	SVD	Logistic Regression	0.9946	0.9821	0.0125	1.2765
OVA_Prostate	ICA	Random Forest	0.9985	0.9890	0.0095	0.9585
OVA_Prostate	ICA	Logistic Regression	0.9907	0.9821	0.0086	0.8734
OVA_Prostate	PCA	k-NN	0.9920	0.9846	0.0074	0.7519
OVA_Prostate	ICA	k-NN	0.9918	0.9846	0.0072	0.7359
OVA_Endometrium	ICA	Logistic Regression	0.9506	0.9465	0.0040	0.4277
OVA_Omentum	SVD	SVM (Linear)	0.9057	0.9022	0.0035	0.3840
OVA_Omentum	SVD	Naive Bayes	0.8619	0.8590	0.0030	0.3468
OVA_Prostate	SVD	XGBoost	0.9988	0.9981	0.0006	0.0613
OVA_Prostate	SVD	Random Forest	0.9894	0.9890	0.0004	0.0449
OVA_Prostate	PCA	XGBoost	0.9986	0.9981	0.0004	0.0437
OVA_Prostate	SVD	SVM (Linear)	0.9992	0.9988	0.0004	0.0388
OVA_Prostate	PCA	SVM (Linear)	0.9991	0.9988	0.0002	0.0242
OVA_Prostate	ICA	XGBoost	0.9981	0.9981	-0.0001	-0.0079
OVA_Prostate	SVD	k-NN	0.9844	0.9846	-0.0002	-0.0187
OVA_Prostate	ICA	SVM (Linear)	0.9973	0.9988	-0.0015	-0.1510
OVA_Omentum	PCA	SVM (Linear)	0.9004	0.9022	-0.0018	-0.1980
OVA_Endometrium	PCA	k-NN	0.9489	0.9520	-0.0031	-0.3224
OVA_Endometrium	ICA	SVM (Linear)	0.9470	0.9517	-0.0048	-0.5023
OVA_Omentum	PCA	k-NN	0.9080	0.9130	-0.0050	-0.5472

OVA_Omentum	ICA	Naive Bayes	0.8541	0.8590	-0.0049	-0.5658
OVA_Endometrium	SVD	k-NN	0.9463	0.9520	-0.0057	-0.5966
OVA_Endometrium	PCA	SVM (Linear)	0.9452	0.9517	-0.0066	-0.6914
OVA_Omentum	ICA	SVM (Linear)	0.8958	0.9022	-0.0064	-0.7050
OVA_Omentum	PCA	Naive Bayes	0.8524	0.8590	-0.0065	-0.7585
OVA_Endometrium	ICA	XGBoost	0.9483	0.9558	-0.0075	-0.7883
OVA_Omentum	SVD	k-NN	0.9051	0.9130	-0.0079	-0.8638
OVA_Endometrium	PCA	Random Forest	0.9242	0.9339	-0.0097	-1.0408
OVA_Endometrium	SVD	SVM (Linear)	0.9411	0.9517	-0.0106	-1.1140
OVA_Endometrium	SVD	XGBoost	0.9441	0.9558	-0.0118	-1.2297
OVA_Prostate	PCA	Random Forest	0.9757	0.9890	-0.0133	-1.3447
OVA_Endometrium	ICA	k-NN	0.9379	0.9520	-0.0140	-1.4725
OVA_Omentum	SVD	Logistic Regression	0.9071	0.9229	-0.0158	-1.7121
OVA_Endometrium	PCA	XGBoost	0.9353	0.9558	-0.0206	-2.1517
OVA_Endometrium	SVD	Random Forest	0.9137	0.9339	-0.0202	-2.1659
OVA_Omentum	PCA	Logistic Regression	0.9028	0.9229	-0.0201	-2.1777
OVA_Omentum	ICA	Random Forest	0.9013	0.9281	-0.0269	-2.8946
OVA_Omentum	PCA	XGBoost	0.9117	0.9411	-0.0294	-3.1221
OVA_Omentum	SVD	Random Forest	0.8948	0.9281	-0.0333	-3.5851
OVA_Omentum	PCA	Random Forest	0.8948	0.9281	-0.0333	-3.5865
OVA_Omentum	SVD	XGBoost	0.9061	0.9411	-0.0350	-3.7174
OVA_Omentum	ICA	Logistic Regression	0.8838	0.9229	-0.0391	-4.2365
OVA_Omentum	ICA	XGBoost	0.8985	0.9411	-0.0425	-4.5175
OVA_Omentum	ICA	k-NN	0.7630	0.9130	-0.1500	-16.4264

We can see in Table A.8 again a vast range of performance improvement or decline depending on dataset, classifier, and dimensionality reduction method. This time, the PCA on the Endometrium dataset in combination with the Naive Bayes classifier leads the field with 15.52 % improvement on AUC, with similar results for ICA and SVD in the same setting. Improvement magnitude very quickly drops off afterwards with a large number of datasets, dimensionality reduction methods, and classifier combinations with minimal to moderate performance declines. Towards the end of the spectrum, we see ICA with k-NN on the Omentum dataset with a -16.43 percent performance decline. Most other cases with pronounced performance decline involve the Random Forest and XGBoost classifier and Logistic Regression.

A.2 Synthetic Sample Generation

A.2.1 Inner cross-validation results

Summary performance by classifier

Table A.9: Best mean AUC (5-fold inner cross-validation) for each classification algorithm, averaged across all datasets and sample generation methods with ranking.

algo	mean_auc	mean_baseline_auc	mean_auc_rank	mean_baseline_auc_rank
XGBoost	0.9693	0.9686	1	1
Logistic Regression	0.9665	0.9565	2	3
Random Forest	0.9657	0.9586	3	2
SVM	0.9562	0.9517	4	4

k-NN	0.9356	0.9498	5	5
Naive Bayes	0.8591	0.8748	6	6

Table A.9 shows that XGBoost is the best performing classifier (both, averaged over all datasets and sample generation methods and on the original unbalanced dataset). Logistic Regression comes in second (averaged over all datasets and sample generation methods) but only third on the original dataset, while the situation is in reverse for the Random Forest classifier. The ranking for SVM, k-NN, and Naive Bayes is again consistent between the original dataset and averaged over all datasets and sample generation methods.

Results broken down by dataset

Table A.10: Best mean AUC (5-fold inner cross-validation) per dataset × sample generation method, averaged over all classifiers.

dataset	oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium					
OVA_Endometrium	RandomOverSampler	0.9370	0.9284	0.0085	0.9190
OVA_Endometrium	RandomForestSMOTE	0.9353	0.9284	0.0069	0.7386
OVA_Endometrium	SVMSMOTE	0.9350	0.9284	0.0066	0.7109
OVA_Endometrium	BorderlineSMOTE	0.9263	0.9284	-0.0021	-0.2257
OVA_Endometrium	SMOTE	0.9263	0.9284	-0.0021	-0.2303
OVA_Endometrium	ADASYN	0.9253	0.9284	-0.0032	-0.3408
OVA_Omentum					
OVA_Omentum	RandomForestSMOTE	0.9102	0.9095	0.0006	0.0702
OVA_Omentum	RandomOverSampler	0.9069	0.9095	-0.0026	-0.2907
OVA_Omentum	SVMSMOTE	0.9040	0.9095	-0.0055	-0.6063
OVA_Omentum	ADASYN	0.9002	0.9095	-0.0094	-1.0309
OVA_Omentum	BorderlineSMOTE	0.8995	0.9095	-0.0100	-1.1003
OVA_Omentum	SMOTE	0.8994	0.9095	-0.0102	-1.1173
OVA_Prostate					
OVA_Prostate	RandomForestSMOTE	0.9931	0.9920	0.0011	0.1099
OVA_Prostate	RandomOverSampler	0.9924	0.9920	0.0004	0.0421
OVA_Prostate	SVMSMOTE	0.9924	0.9920	0.0004	0.0417
OVA_Prostate	BorderlineSMOTE	0.9918	0.9920	-0.0002	-0.0220
OVA_Prostate	SMOTE	0.9914	0.9920	-0.0005	-0.0536
OVA_Prostate	ADASYN	0.9908	0.9920	-0.0012	-0.1174

Table A.10 provides some insights into the degree to which the summary results for the various sample generation techniques vary by dataset. It is encouraging to see some consistency for the Random Oversampling and the RandomForestSMOTE method. Both land consistently in the top two (like in the summary results). The RandomForestSMOTE method scores the best result twice and the Random Oversampling method once. ADASYN, SMOTE, and BorderlineSMOTE are all harmful across all datasets (averaged over all classifiers). SVMSMOTE is beneficial for 2 out of 3 datasets. Our new RandomForestSMOTE algorithm is the only method that has a beneficial effect on all 3 datasets (averaged over all classifiers) and could therefore be considered a choice that adapts well to the particularities of the various datasets.

Results broken down by classifier

Table A.11: Best mean AUC (5-fold inner cross-validation) per classifier × sample generation method, averaged over all datasets.

algo	oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
Logistic Regression					
Logistic Regression	RandomForestSMOTE	0.9675	0.9565	0.0110	1.1553
Logistic Regression	SVMSMOTE	0.9673	0.9565	0.0108	1.1305
Logistic Regression	RandomOverSampler	0.9669	0.9565	0.0104	1.0903
Logistic Regression	BorderlineSMOTE	0.9660	0.9565	0.0095	0.9934
Logistic Regression	SMOTE	0.9658	0.9565	0.0093	0.9694
Logistic Regression	ADASYN	0.9657	0.9565	0.0092	0.9648
Naive Bayes					
Naive Bayes	RandomOverSampler	0.8749	0.8748	0.0001	0.0116
Naive Bayes	RandomForestSMOTE	0.8687	0.8748	-0.0061	-0.6923
Naive Bayes	SVMSMOTE	0.8674	0.8748	-0.0074	-0.8445
Naive Bayes	SMOTE	0.8486	0.8748	-0.0262	-2.9949
Naive Bayes	BorderlineSMOTE	0.8477	0.8748	-0.0271	-3.0939
Naive Bayes	ADASYN	0.8477	0.8748	-0.0271	-3.0963
Random Forest					
Random Forest	RandomOverSampler	0.9667	0.9586	0.0082	0.8536
Random Forest	RandomForestSMOTE	0.9664	0.9586	0.0078	0.8162
Random Forest	ADASYN	0.9655	0.9586	0.0070	0.7290
Random Forest	SMOTE	0.9654	0.9586	0.0068	0.7146
Random Forest	SVMSMOTE	0.9650	0.9586	0.0065	0.6737
Random Forest	BorderlineSMOTE	0.9649	0.9586	0.0064	0.6672
SVM					
SVM	SVMSMOTE	0.9576	0.9517	0.0059	0.6179
SVM	RandomForestSMOTE	0.9568	0.9517	0.0051	0.5355
SVM	BorderlineSMOTE	0.9568	0.9517	0.0050	0.5275
SVM	RandomOverSampler	0.9555	0.9517	0.0038	0.3963
SVM	SMOTE	0.9554	0.9517	0.0036	0.3808
SVM	ADASYN	0.9553	0.9517	0.0036	0.3734
XGBoost					
XGBoost	SVMSMOTE	0.9709	0.9686	0.0023	0.2344
XGBoost	BorderlineSMOTE	0.9701	0.9686	0.0016	0.1602
XGBoost	ADASYN	0.9694	0.9686	0.0008	0.0874
XGBoost	SMOTE	0.9690	0.9686	0.0004	0.0423
XGBoost	RandomForestSMOTE	0.9684	0.9686	-0.0002	-0.0223
XGBoost	RandomOverSampler	0.9678	0.9686	-0.0008	-0.0842
k-NN					
k-NN	RandomForestSMOTE	0.9493	0.9498	-0.0005	-0.0554
k-NN	RandomOverSampler	0.9407	0.9498	-0.0091	-0.9534
k-NN	SVMSMOTE	0.9348	0.9498	-0.0150	-1.5832
k-NN	SMOTE	0.9302	0.9498	-0.0196	-2.0661
k-NN	BorderlineSMOTE	0.9297	0.9498	-0.0200	-2.1109
k-NN	ADASYN	0.9289	0.9498	-0.0209	-2.2049

The breakdown of AUC results of the inner cross-validation by classifier in Table A.11 reveals some interesting patterns. The first finding is that no single sample generation method dominates all other methods across all classifiers. RandomForestSMOTE performed best for Logistic Regression and k-NN (even though it was harmful for k-NN). Random Oversampling did best for Random Forest and Naive Bayes (being the only method not harmful for Naive Bayes), and SVMSMOTE took the first spot for SVM and XGBoost. It is noteworthy (in the context of the overall good average performance of Random Oversampling and RandomForestSMOTE) that both methods were the only harmful sample generation algorithms for the XGBoost classifier. It is interesting to see that sample generation methods do more harm than help for Naive Bayes (except for the Random Oversampling), as well as the k-NN

classifier (where all sample generation algorithms were harmful).

Results broken down by dataset, sample generation method, and classifier

Table A.12: Best mean AUC (5-fold inner cross-validation) per dataset × algorithm × sample generation method with percentage improvement over baseline

dataset	algo	oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium	Logistic Regression	RandomOverSampler	0.9685	0.9458	0.0227	2.4024
OVA_Endometrium	Logistic Regression	SVMSMOTE	0.9671	0.9458	0.0213	2.2558
OVA_Endometrium	Random Forest	RandomOverSampler	0.9610	0.9416	0.0195	2.0702
OVA_Endometrium	Logistic Regression	ADASYN	0.9647	0.9458	0.0189	2.0028
OVA_Endometrium	Logistic Regression	RandomForestSMOTE	0.9645	0.9458	0.0187	1.9817
OVA_Endometrium	Logistic Regression	SMOTE	0.9645	0.9458	0.0187	1.9803
OVA_Endometrium	Logistic Regression	BorderlineSMOTE	0.9639	0.9458	0.0181	1.9184
OVA_Endometrium	SVM	ADASYN	0.9647	0.9475	0.0171	1.8077
OVA_Endometrium	SVM	RandomOverSampler	0.9646	0.9475	0.0170	1.7965
OVA_Endometrium	SVM	SVMSMOTE	0.9643	0.9475	0.0168	1.7738
OVA_Endometrium	SVM	SMOTE	0.9643	0.9475	0.0167	1.7646
OVA_Endometrium	SVM	BorderlineSMOTE	0.9638	0.9475	0.0163	1.7195
OVA_Endometrium	Random Forest	RandomForestSMOTE	0.9577	0.9416	0.0162	1.7174
OVA_Endometrium	Random Forest	BorderlineSMOTE	0.9577	0.9416	0.0162	1.7159
OVA_Endometrium	Random Forest	SVMSMOTE	0.9575	0.9416	0.0159	1.6914
OVA_Endometrium	Random Forest	SMOTE	0.9575	0.9416	0.0159	1.6894
OVA_Endometrium	Random Forest	ADASYN	0.9567	0.9416	0.0152	1.6121
OVA_Omentum	Logistic Regression	RandomForestSMOTE	0.9395	0.9254	0.0141	1.5251
OVA_Endometrium	SVM	RandomForestSMOTE	0.9593	0.9475	0.0118	1.2403
OVA_Omentum	Logistic Regression	SVMSMOTE	0.9360	0.9254	0.0106	1.1450
OVA_Omentum	Logistic Regression	BorderlineSMOTE	0.9350	0.9254	0.0096	1.0366
OVA_Omentum	Logistic Regression	RandomOverSampler	0.9339	0.9254	0.0085	0.9186
OVA_Omentum	Logistic Regression	SMOTE	0.9337	0.9254	0.0083	0.9004
OVA_Omentum	Logistic Regression	ADASYN	0.9336	0.9254	0.0082	0.8913
OVA_Omentum	Random Forest	RandomForestSMOTE	0.9422	0.9351	0.0071	0.7619
OVA_Endometrium	Naive Bayes	SVMSMOTE	0.8245	0.8191	0.0055	0.6655
OVA_Prostate	k-NN	RandomForestSMOTE	0.9968	0.9903	0.0065	0.6551
OVA_Prostate	k-NN	SMOTE	0.9967	0.9903	0.0064	0.6431
OVA_Omentum	Random Forest	ADASYN	0.9405	0.9351	0.0055	0.5865
OVA_Omentum	Random Forest	RandomOverSampler	0.9403	0.9351	0.0052	0.5587
OVA_Prostate	k-NN	RandomOverSampler	0.9957	0.9903	0.0054	0.5447
OVA_Omentum	Random Forest	SMOTE	0.9398	0.9351	0.0048	0.5105
OVA_Omentum	XGBoost	ADASYN	0.9444	0.9400	0.0045	0.4748
OVA_Omentum	XGBoost	SVMSMOTE	0.9440	0.9400	0.0040	0.4285
OVA_Prostate	k-NN	SVMSMOTE	0.9946	0.9903	0.0042	0.4273
OVA_Omentum	XGBoost	SMOTE	0.9437	0.9400	0.0037	0.3958
OVA_Omentum	Random Forest	SVMSMOTE	0.9388	0.9351	0.0037	0.3942
OVA_Endometrium	XGBoost	SVMSMOTE	0.9724	0.9687	0.0037	0.3799
OVA_Omentum	Random Forest	BorderlineSMOTE	0.9384	0.9351	0.0033	0.3578
OVA_Prostate	k-NN	BorderlineSMOTE	0.9938	0.9903	0.0035	0.3524
OVA_Omentum	SVM	RandomForestSMOTE	0.9125	0.9093	0.0032	0.3469
OVA_Omentum	XGBoost	RandomForestSMOTE	0.9427	0.9400	0.0027	0.2877
OVA_Omentum	XGBoost	BorderlineSMOTE	0.9426	0.9400	0.0026	0.2816
OVA_Prostate	k-NN	ADASYN	0.9926	0.9903	0.0023	0.2318
OVA_Endometrium	XGBoost	BorderlineSMOTE	0.9708	0.9687	0.0021	0.2139
OVA_Endometrium	Naive Bayes	RandomOverSampler	0.8207	0.8191	0.0016	0.1996
OVA_Prostate	Naive Bayes	RandomForestSMOTE	0.9706	0.9688	0.0018	0.1877
OVA_Prostate	Logistic Regression	BorderlineSMOTE	0.9991	0.9983	0.0008	0.0772
OVA_Prostate	Logistic Regression	SMOTE	0.9991	0.9983	0.0008	0.0756
OVA_Endometrium	XGBoost	ADASYN	0.9694	0.9687	0.0007	0.0752
OVA_Prostate	SVM	SVMSMOTE	0.9990	0.9983	0.0007	0.0693

OVA_Endometrium	XGBoost	SMOTE	0.9693	0.9687	0.0006	0.0611
OVA_Prostate	Logistic Regression	SVMSMOTE	0.9988	0.9983	0.0005	0.0511
OVA_Prostate	Logistic Regression	ADASYN	0.9988	0.9983	0.0005	0.0495
OVA_Prostate	SVM	RandomForestSMOTE	0.9987	0.9983	0.0004	0.0385
OVA_Omentum	XGBoost	RandomOverSampler	0.9403	0.9400	0.0004	0.0378
OVA_Prostate	Random Forest	ADASYN	0.9993	0.9990	0.0003	0.0301
OVA_Prostate	Logistic Regression	RandomForestSMOTE	0.9986	0.9983	0.0003	0.0295
OVA_Prostate	SVM	BorderlineSMOTE	0.9986	0.9983	0.0003	0.0262
OVA_Prostate	Random Forest	RandomForestSMOTE	0.9992	0.9990	0.0002	0.0177
OVA_Prostate	SVM	RandomOverSampler	0.9985	0.9983	0.0002	0.0170
OVA_Prostate	SVM	SMOTE	0.9985	0.9983	0.0002	0.0170
OVA_Omentum	SVM	SVMSMOTE	0.9095	0.9093	0.0001	0.0158
OVA_Endometrium	XGBoost	RandomOverSampler	0.9689	0.9687	0.0002	0.0155
OVA_Prostate	Logistic Regression	RandomOverSampler	0.9984	0.9983	0.0001	0.0065
OVA_Prostate	Naive Bayes	RandomOverSampler	0.9688	0.9688	0.0000	0.0000
OVA_Prostate	XGBoost	BorderlineSMOTE	0.9970	0.9971	-0.0001	-0.0062
OVA_Prostate	Random Forest	SMOTE	0.9989	0.9990	-0.0001	-0.0131
OVA_Prostate	SVM	ADASYN	0.9982	0.9983	-0.0002	-0.0154
OVA_Prostate	Random Forest	RandomOverSampler	0.9989	0.9990	-0.0002	-0.0170
OVA_Prostate	Random Forest	SVMSMOTE	0.9988	0.9990	-0.0002	-0.0238
OVA_Prostate	Random Forest	BorderlineSMOTE	0.9987	0.9990	-0.0003	-0.0316
OVA_Endometrium	XGBoost	RandomForestSMOTE	0.9680	0.9687	-0.0007	-0.0755
OVA_Prostate	XGBoost	SVMSMOTE	0.9962	0.9971	-0.0009	-0.0899
OVA_Endometrium	k-NN	RandomForestSMOTE	0.9468	0.9479	-0.0012	-0.1228
OVA_Omentum	Naive Bayes	RandomOverSampler	0.8351	0.8364	-0.0013	-0.1592
OVA_Omentum	SVM	BorderlineSMOTE	0.9079	0.9093	-0.0015	-0.1644
OVA_Prostate	Naive Bayes	SVMSMOTE	0.9669	0.9688	-0.0018	-0.1877
OVA_Prostate	XGBoost	RandomForestSMOTE	0.9945	0.9971	-0.0026	-0.2628
OVA_Prostate	XGBoost	ADASYN	0.9944	0.9971	-0.0027	-0.2658
OVA_Prostate	XGBoost	RandomOverSampler	0.9941	0.9971	-0.0030	-0.2961
OVA_Prostate	XGBoost	SMOTE	0.9940	0.9971	-0.0031	-0.3091
OVA_Endometrium	Naive Bayes	RandomForestSMOTE	0.8154	0.8191	-0.0036	-0.4422
OVA_Prostate	Naive Bayes	BorderlineSMOTE	0.9633	0.9688	-0.0055	-0.5630
OVA_Omentum	SVM	RandomOverSampler	0.9035	0.9093	-0.0059	-0.6462
OVA_Omentum	SVM	SMOTE	0.9033	0.9093	-0.0060	-0.6619
OVA_Omentum	SVM	ADASYN	0.9030	0.9093	-0.0063	-0.6942
OVA_Prostate	Naive Bayes	ADASYN	0.9615	0.9688	-0.0073	-0.7507
OVA_Prostate	Naive Bayes	SMOTE	0.9615	0.9688	-0.0073	-0.7507
OVA_Omentum	k-NN	RandomForestSMOTE	0.9042	0.9111	-0.0069	-0.7576
OVA_Endometrium	k-NN	RandomOverSampler	0.9381	0.9479	-0.0098	-1.0364
OVA_Omentum	Naive Bayes	RandomForestSMOTE	0.8201	0.8364	-0.0164	-1.9565
OVA_Endometrium	k-NN	SVMSMOTE	0.9243	0.9479	-0.0236	-2.4893
OVA_Omentum	k-NN	RandomOverSampler	0.8884	0.9111	-0.0227	-2.4955
OVA_Omentum	k-NN	SVMSMOTE	0.8854	0.9111	-0.0257	-2.8257
OVA_Omentum	Naive Bayes	SVMSMOTE	0.8106	0.8364	-0.0258	-3.0839
OVA_Endometrium	k-NN	BorderlineSMOTE	0.9175	0.9479	-0.0304	-3.2122
OVA_Endometrium	k-NN	SMOTE	0.9162	0.9479	-0.0318	-3.3513
OVA_Endometrium	k-NN	ADASYN	0.9148	0.9479	-0.0331	-3.4912
OVA_Omentum	k-NN	ADASYN	0.8791	0.9111	-0.0320	-3.5152
OVA_Omentum	k-NN	BorderlineSMOTE	0.8779	0.9111	-0.0332	-3.6426
OVA_Omentum	k-NN	SMOTE	0.8776	0.9111	-0.0335	-3.6736
OVA_Endometrium	Naive Bayes	SMOTE	0.7861	0.8191	-0.0330	-4.0301
OVA_Endometrium	Naive Bayes	BorderlineSMOTE	0.7843	0.8191	-0.0348	-4.2473
OVA_Omentum	Naive Bayes	ADASYN	0.8003	0.8364	-0.0361	-4.3175
OVA_Omentum	Naive Bayes	SMOTE	0.7981	0.8364	-0.0383	-4.5805
OVA_Endometrium	Naive Bayes	ADASYN	0.7812	0.8191	-0.0379	-4.6233
OVA_Omentum	Naive Bayes	BorderlineSMOTE	0.7955	0.8364	-0.0409	-4.8958

The breakdown of results by dataset, classifier, and sample generation algorithm in Table A.12 shows that the worst cases are almost twice as harmful as the best cases are beneficial.

It is quite a mixed field in the sense that one can find beneficial as well as harmful examples for almost all sample generation algorithms (depending on the dataset and classifier). That said, overall, there are more line items with a positive effect than there are with a negative effect.

A.2.2 Outer cross-validation results

Summary performance by classifier

Table A.13: Mean AUC (5-fold outer cross-validation) per classifier, averaged across all datasets and sample generation methods with baseline (no sample generation) and ranking.

algo	mean_AUC	mean_baseline_AUC	mean_auc_rank	mean_baseline_auc_rank
XGBoost	0.9651	0.9650	1	1
Random Forest	0.9612	0.9503	2	4
SVM	0.9552	0.9509	3	2
Logistic Regression	0.9539	0.9505	4	3
k-NN	0.9317	0.9499	5	5
Naive Bayes	0.8596	0.8770	6	6

Table A.13 shows that XGBoost is the best performing classifier (be it averaged over all datasets and sample generation methods or on the original dataset). Random Forest comes in second (averaged over all datasets and sample generation methods) but only fourth on the original unbalanced dataset (behind SVM and Logistic Regression). This is rather surprising given that ensemble methods like Random Forest are a remedy in their own right against the class imbalance problem, and one would therefore expect it to outperform at least a rather simple algorithm like Logistic Regression on imbalanced datasets.

Results broken down by dataset

The results of the outer cross-validation broken down by dataset are shown in Table A.14. The picture for the OVA_Endometrium dataset is similar to the inner cross-validation results. RandomForestSMOTE and Random Oversampling are the two best methods (with RandomForestSMOTE leading in this case). SVMSMOTE occupies the third spot again and still provides a benefit, while all other methods are harmful. For the OVA_Omentum dataset, all methods are now harmful, but at least RandomForestSMOTE is the least harmful of all. For the OVA_Prostate dataset, all methods except the BorderlineSMOTE are helpful, but interestingly, SVMSMOTE leads the field, and the RandomForestSMOTE comes in only at 4th place. So while the inner cross-validation results showed consistency across datasets among the performance of the Random Oversampling and the RandomForestSMOTE, this consistency has unfortunately been partly lost in the results of the outer cross-validation, as neither the Random Oversampling method nor the RandomForestSMOTE is consistently within the first two methods if the results are broken down by dataset. That means the choice of synthetic sample generation method is a dataset-dependent model selection problem where the results from an inner cross-validation don't carry over to out-of-sample data from the outer cross-validation, which calls into question the whole model selection approach.

Table A.14: Mean AUC (5-fold outer cross-validation) per dataset and sample generation methods, averaged over all classifiers

dataset	oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium					
OVA_Endometrium	RandomForestSMOTE	0.9327	0.9248	0.0079	0.8542
OVA_Endometrium	RandomOverSampler	0.9322	0.9248	0.0074	0.8002
OVA_Endometrium	SVMSMOTE	0.9283	0.9248	0.0035	0.3785
OVA_Endometrium	SMOTE	0.9229	0.9248	-0.0019	-0.2054
OVA_Endometrium	ADASYN	0.9188	0.9248	-0.0060	-0.6488
OVA_Endometrium	BorderlineSMOTE	0.9178	0.9248	-0.0070	-0.7569
OVA_Omentum					
OVA_Omentum	RandomForestSMOTE	0.9045	0.9110	-0.0065	-0.7135
OVA_Omentum	SVMSMOTE	0.9005	0.9110	-0.0105	-1.1526
OVA_Omentum	RandomOverSampler	0.8995	0.9110	-0.0115	-1.2623
OVA_Omentum	ADASYN	0.8982	0.9110	-0.0128	-1.4050
OVA_Omentum	BorderlineSMOTE	0.8970	0.9110	-0.0140	-1.5368
OVA_Omentum	SMOTE	0.8951	0.9110	-0.0159	-1.7453
OVA_Prostate					
OVA_Prostate	SVMSMOTE	0.9923	0.9859	0.0064	0.6492
OVA_Prostate	RandomOverSampler	0.9913	0.9859	0.0054	0.5477
OVA_Prostate	SMOTE	0.9904	0.9859	0.0045	0.4564
OVA_Prostate	RandomForestSMOTE	0.9886	0.9859	0.0027	0.2739
OVA_Prostate	ADASYN	0.9881	0.9859	0.0022	0.2231
OVA_Prostate	BorderlineSMOTE	0.9825	0.9859	-0.0034	-0.3449

Results broken down by classifier

The per algorithm AUC results from the outer cross-validation, as shown in Table A.15, are in some respects similar to the ones from the inner cross-validation and in other respects quite different. All sample generation methods are again harmful to the k-NN classifier. This time, Random Oversampling is the least harmful method, but it is closely followed by the RandomForestSMOTE method. For XGBoost, RandomForestSMOTE takes second place this time and is not harmful anymore, while Random Oversampling remains a harmful method. For the SVM classifier, RandomForestSMOTE and Random OverSampling are the two least beneficial methods. Standard SMOTE counterintuitively performs rather well for the SVM classifier. It also takes the first spot for the Random Forest classifier, and RandomForestSMOTE takes the last place. For Naive Bayes, RandomForestSMOTE, and Random Oversampling are the only two beneficial methods, and this time RandomForestSMOTE takes the top spot. Last but not least, SVMSMOTE is the best performing method for Logistic Regression, and RandomForestSMOTE comes in second last. One could say that the main strength of RandomForestSMOTE and Random Oversampling is not necessarily to shine and vastly outperform other sample generation methods for classifiers that get to benefit from most sample generation methods, but rather to cause limited harm in scenarios where most other sample generation methods cause large performance degradation (e.g. Naive Bayes and k-NN).

Table A.15: Mean AUC (5-fold outer cross-validation) broken down by classifier

algo	oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
Logistic Regression					
Logistic Regression	SVMSMOTE	0.9579	0.9505	0.0074	0.7807
Logistic Regression	SMOTE	0.9556	0.9505	0.0051	0.5354
Logistic Regression	RandomOverSampler	0.9555	0.9505	0.0050	0.5288
Logistic Regression	BorderlineSMOTE	0.9534	0.9505	0.0029	0.3052
Logistic Regression	RandomForestSMOTE	0.9510	0.9505	0.0005	0.0511
Logistic Regression	ADASYN	0.9501	0.9505	-0.0004	-0.0401
Naive Bayes					
Naive Bayes	RandomForestSMOTE	0.8866	0.8770	0.0097	1.1010
Naive Bayes	RandomOverSampler	0.8773	0.8770	0.0003	0.0396
Naive Bayes	SVMSMOTE	0.8683	0.8770	-0.0086	-0.9841
Naive Bayes	ADASYN	0.8469	0.8770	-0.0301	-3.4305
Naive Bayes	SMOTE	0.8458	0.8770	-0.0312	-3.5529
Naive Bayes	BorderlineSMOTE	0.8328	0.8770	-0.0442	-5.0386
Random Forest					
Random Forest	SMOTE	0.9628	0.9503	0.0125	1.3135
Random Forest	ADASYN	0.9623	0.9503	0.0120	1.2646
Random Forest	RandomOverSampler	0.9615	0.9503	0.0112	1.1736
Random Forest	BorderlineSMOTE	0.9612	0.9503	0.0109	1.1419
Random Forest	SVMSMOTE	0.9609	0.9503	0.0106	1.1110
Random Forest	RandomForestSMOTE	0.9583	0.9503	0.0080	0.8439
SVM					
SVM	SMOTE	0.9569	0.9509	0.0059	0.6234
SVM	BorderlineSMOTE	0.9561	0.9509	0.0051	0.5405
SVM	SVMSMOTE	0.9559	0.9509	0.0050	0.5229
SVM	ADASYN	0.9556	0.9509	0.0047	0.4929
SVM	RandomForestSMOTE	0.9544	0.9509	0.0035	0.3651
SVM	RandomOverSampler	0.9526	0.9509	0.0017	0.1808
XGBoost					
XGBoost	BorderlineSMOTE	0.9662	0.9650	0.0012	0.1273
XGBoost	RandomForestSMOTE	0.9658	0.9650	0.0008	0.0846
XGBoost	SVMSMOTE	0.9657	0.9650	0.0007	0.0740
XGBoost	SMOTE	0.9651	0.9650	0.0001	0.0145
XGBoost	ADASYN	0.9648	0.9650	-0.0002	-0.0193
XGBoost	RandomOverSampler	0.9630	0.9650	-0.0020	-0.2028
k-NN					
k-NN	RandomOverSampler	0.9359	0.9499	-0.0139	-1.4652
k-NN	RandomForestSMOTE	0.9352	0.9499	-0.0147	-1.5471
k-NN	SVMSMOTE	0.9335	0.9499	-0.0163	-1.7210
k-NN	SMOTE	0.9304	0.9499	-0.0194	-2.0440
k-NN	ADASYN	0.9304	0.9499	-0.0195	-2.0514
k-NN	BorderlineSMOTE	0.9250	0.9499	-0.0248	-2.6132

Results broken down dataset, dimensionality reduction method, and classifier

Table A.16: Mean AUC (5-fold outer cross-validation) broken down by dataset, classifier and sample generation method

dataset	algo	oversampling	mean_auc	mean_baseline_auc	mean_diff	mean_pct_diff
OVA_Endometrium	Random Forest	RandomForestSMOTE	0.9633	0.9339	0.0294	3.1450
OVA_Endometrium	Random Forest	SMOTE	0.9578	0.9339	0.0239	2.5622
OVA_Endometrium	Random Forest	ADASYN	0.9557	0.9339	0.0218	2.3347
OVA_Endometrium	Random Forest	SVMSMOTE	0.9551	0.9339	0.0212	2.2687
OVA_Endometrium	Random Forest	RandomOverSampler	0.9541	0.9339	0.0202	2.1614

OVA_Endometrium	Logistic Regression	RandomOverSampler	0.9657	0.9465	0.0192	2.0231
OVA_Endometrium	Random Forest	BorderlineSMOTE	0.9520	0.9339	0.0181	1.9355
OVA_Endometrium	Logistic Regression	SVMSMOTE	0.9641	0.9465	0.0176	1.8547
OVA_Prostate	Naive Bayes	RandomForestSMOTE	0.9777	0.9628	0.0148	1.5408
OVA_Prostate	Logistic Regression	RandomOverSampler	0.9972	0.9821	0.0151	1.5367
OVA_Endometrium	Naive Bayes	RandomForestSMOTE	0.8211	0.8091	0.0120	1.4845
OVA_Prostate	Logistic Regression	SMOTE	0.9964	0.9821	0.0143	1.4571
OVA_Prostate	Logistic Regression	SVMSMOTE	0.9955	0.9821	0.0134	1.3694
OVA_Prostate	k-NN	SMOTE	0.9972	0.9846	0.0126	1.2777
OVA_Prostate	k-NN	RandomOverSampler	0.9971	0.9846	0.0125	1.2728
OVA_Endometrium	XGBoost	SMOTE	0.9679	0.9558	0.0120	1.2585
OVA_Endometrium	SVM	SMOTE	0.9634	0.9517	0.0117	1.2293
OVA_Endometrium	SVM	BorderlineSMOTE	0.9633	0.9517	0.0116	1.2144
OVA_Endometrium	SVM	SVMSMOTE	0.9624	0.9517	0.0107	1.1242
OVA_Endometrium	SVM	RandomOverSampler	0.9623	0.9517	0.0106	1.1142
OVA_Endometrium	Logistic Regression	BorderlineSMOTE	0.9570	0.9465	0.0104	1.1025
OVA_Prostate	k-NN	SVMSMOTE	0.9949	0.9846	0.0103	1.0495
OVA_Prostate	k-NN	ADASYN	0.9941	0.9846	0.0095	0.9689
OVA_Prostate	Logistic Regression	BorderlineSMOTE	0.9916	0.9821	0.0095	0.9664
OVA_Endometrium	SVM	ADASYN	0.9609	0.9517	0.0092	0.9662
OVA_Endometrium	Logistic Regression	RandomForestSMOTE	0.9556	0.9465	0.0091	0.9590
OVA_Omentum	Random Forest	BorderlineSMOTE	0.9368	0.9281	0.0087	0.9326
OVA_Omentum	Random Forest	RandomOverSampler	0.9368	0.9281	0.0086	0.9314
OVA_Endometrium	SVM	RandomForestSMOTE	0.9602	0.9517	0.0085	0.8949
OVA_Prostate	Random Forest	SMOTE	0.9978	0.9890	0.0088	0.8920
OVA_Omentum	Random Forest	ADASYN	0.9363	0.9281	0.0082	0.8810
OVA_Endometrium	XGBoost	RandomOverSampler	0.9638	0.9558	0.0080	0.8382
OVA_Prostate	Naive Bayes	SVMSMOTE	0.9705	0.9628	0.0077	0.7989
OVA_Endometrium	Logistic Regression	SMOTE	0.9539	0.9465	0.0073	0.7730
OVA_Prostate	Logistic Regression	ADASYN	0.9895	0.9821	0.0074	0.7530
OVA_Prostate	Random Forest	SVMSMOTE	0.9957	0.9890	0.0067	0.6818
OVA_Endometrium	XGBoost	RandomForestSMOTE	0.9619	0.9558	0.0061	0.6342
OVA_Omentum	SVM	SMOTE	0.9078	0.9022	0.0056	0.6161
OVA_Prostate	Random Forest	ADASYN	0.9950	0.9890	0.0061	0.6141
OVA_Endometrium	XGBoost	SVMSMOTE	0.9615	0.9558	0.0056	0.5896
OVA_Prostate	Random Forest	BorderlineSMOTE	0.9948	0.9890	0.0058	0.5890
OVA_Endometrium	Logistic Regression	ADASYN	0.9520	0.9465	0.0054	0.5743
OVA_Prostate	Logistic Regression	RandomForestSMOTE	0.9874	0.9821	0.0053	0.5403
OVA_Omentum	SVM	ADASYN	0.9069	0.9022	0.0047	0.5186
OVA_Omentum	Random Forest	SMOTE	0.9328	0.9281	0.0047	0.5062
OVA_Endometrium	XGBoost	BorderlineSMOTE	0.9605	0.9558	0.0047	0.4918
OVA_Prostate	Random Forest	RandomOverSampler	0.9936	0.9890	0.0046	0.4680
OVA_Omentum	SVM	BorderlineSMOTE	0.9063	0.9022	0.0041	0.4504
OVA_Omentum	SVM	SVMSMOTE	0.9058	0.9022	0.0036	0.4038
OVA_Omentum	Random Forest	SVMSMOTE	0.9319	0.9281	0.0037	0.4033
OVA_Prostate	k-NN	BorderlineSMOTE	0.9884	0.9846	0.0038	0.3814
OVA_Omentum	Naive Bayes	RandomForestSMOTE	0.8611	0.8590	0.0021	0.2468
OVA_Omentum	SVM	RandomForestSMOTE	0.9042	0.9022	0.0020	0.2167
OVA_Endometrium	XGBoost	ADASYN	0.9571	0.9558	0.0013	0.1353
OVA_Omentum	Naive Bayes	RandomOverSampler	0.8599	0.8590	0.0009	0.1094
OVA_Omentum	XGBoost	BorderlineSMOTE	0.9420	0.9411	0.0010	0.1032
OVA_Prostate	SVM	RandomOverSampler	0.9994	0.9988	0.0006	0.0575
OVA_Prostate	SVM	SVMSMOTE	0.9994	0.9988	0.0006	0.0575
OVA_Prostate	SVM	SMOTE	0.9994	0.9988	0.0005	0.0526
OVA_Omentum	XGBoost	ADASYN	0.9414	0.9411	0.0003	0.0336
OVA_Prostate	SVM	ADASYN	0.9990	0.9988	0.0002	0.0187
OVA_Endometrium	Naive Bayes	RandomOverSampler	0.8092	0.8091	0.0001	0.0126
OVA_Prostate	Naive Bayes	RandomOverSampler	0.9628	0.9628	0.0000	0.0000
OVA_Prostate	SVM	RandomForestSMOTE	0.9988	0.9988	-0.0001	-0.0056
OVA_Prostate	SVM	BorderlineSMOTE	0.9986	0.9988	-0.0002	-0.0201
OVA_Prostate	XGBoost	RandomOverSampler	0.9976	0.9981	-0.0005	-0.0528

OVA_Prostate	XGBoost	SVMSMOTE	0.9976	0.9981	-0.0006	-0.0567
OVA_Prostate	k-NN	RandomForestSMOTE	0.9839	0.9846	-0.0007	-0.0684
OVA_Prostate	Random Forest	RandomForestSMOTE	0.9880	0.9890	-0.0009	-0.0951
OVA_Omentum	XGBoost	RandomForestSMOTE	0.9400	0.9411	-0.0011	-0.1122
OVA_Prostate	XGBoost	SMOTE	0.9963	0.9981	-0.0019	-0.1866
OVA_Prostate	XGBoost	BorderlineSMOTE	0.9962	0.9981	-0.0020	-0.1989
OVA_Prostate	XGBoost	ADASYN	0.9960	0.9981	-0.0022	-0.2172
OVA_Prostate	XGBoost	RandomForestSMOTE	0.9956	0.9981	-0.0026	-0.2562
OVA_Omentum	XGBoost	SVMSMOTE	0.9381	0.9411	-0.0029	-0.3110
OVA_Omentum	Random Forest	RandomForestSMOTE	0.9238	0.9281	-0.0044	-0.4708
OVA_Omentum	SVM	RandomOverSampler	0.8962	0.9022	-0.0060	-0.6672
OVA_Omentum	Logistic Regression	SMOTE	0.9165	0.9229	-0.0064	-0.6891
OVA_Prostate	Naive Bayes	ADASYN	0.9552	0.9628	-0.0077	-0.7989
OVA_Prostate	Naive Bayes	SMOTE	0.9552	0.9628	-0.0077	-0.7989
OVA_Omentum	Logistic Regression	SVMSMOTE	0.9141	0.9229	-0.0087	-0.9472
OVA_Omentum	XGBoost	SMOTE	0.9313	0.9411	-0.0097	-1.0356
OVA_Omentum	Logistic Regression	BorderlineSMOTE	0.9117	0.9229	-0.0112	-1.2160
OVA_Endometrium	Naive Bayes	SVMSMOTE	0.7981	0.8091	-0.0110	-1.3577
OVA_Omentum	Logistic Regression	RandomForestSMOTE	0.9100	0.9229	-0.0129	-1.4005
OVA_Omentum	XGBoost	RandomOverSampler	0.9277	0.9411	-0.0134	-1.4191
OVA_Endometrium	k-NN	RandomOverSampler	0.9380	0.9520	-0.0140	-1.4718
OVA_Omentum	Logistic Regression	ADASYN	0.9089	0.9229	-0.0140	-1.5142
OVA_Endometrium	k-NN	RandomForestSMOTE	0.9338	0.9520	-0.0182	-1.9099
OVA_Omentum	Logistic Regression	RandomOverSampler	0.9037	0.9229	-0.0192	-2.0763
OVA_Endometrium	k-NN	SVMSMOTE	0.9289	0.9520	-0.0231	-2.4264
OVA_Omentum	Naive Bayes	SVMSMOTE	0.8364	0.8590	-0.0226	-2.6310
OVA_Omentum	k-NN	RandomForestSMOTE	0.8878	0.9130	-0.0252	-2.7634
OVA_Endometrium	k-NN	ADASYN	0.9198	0.9520	-0.0322	-3.3790
OVA_Endometrium	k-NN	SMOTE	0.9194	0.9520	-0.0326	-3.4221
OVA_Endometrium	k-NN	BorderlineSMOTE	0.9157	0.9520	-0.0362	-3.8041
OVA_Prostate	Naive Bayes	BorderlineSMOTE	0.9255	0.9628	-0.0374	-3.8804
OVA_Omentum	k-NN	ADASYN	0.8772	0.9130	-0.0358	-3.9244
OVA_Omentum	k-NN	SVMSMOTE	0.8767	0.9130	-0.0363	-3.9733
OVA_Omentum	k-NN	SMOTE	0.8747	0.9130	-0.0382	-4.1892
OVA_Endometrium	Naive Bayes	SMOTE	0.7750	0.8091	-0.0341	-4.2171
OVA_Omentum	k-NN	RandomOverSampler	0.8727	0.9130	-0.0403	-4.4112
OVA_Omentum	k-NN	BorderlineSMOTE	0.8710	0.9130	-0.0420	-4.6011
OVA_Omentum	Naive Bayes	ADASYN	0.8185	0.8590	-0.0405	-4.7117
OVA_Omentum	Naive Bayes	BorderlineSMOTE	0.8145	0.8590	-0.0445	-5.1752
OVA_Endometrium	Naive Bayes	ADASYN	0.7670	0.8091	-0.0421	-5.2019
OVA_Omentum	Naive Bayes	SMOTE	0.8073	0.8590	-0.0517	-6.0144
OVA_Endometrium	Naive Bayes	BorderlineSMOTE	0.7584	0.8091	-0.0507	-6.2718

Table A.16 shows some interesting patterns. The number of beneficial cases is roughly on a similar level as the number of harmful cases. So if one were to apply in a random fashion a synthetic sample generation method to a random dataset and classifier (from our experiments), one stands a rough 50:50 chance to make things better or worse. This is, to a degree, surprising as the range of sample generation methods we tested was, to a large part, supposed not to cope well in a high-dimensional setting. That said, we can see that in the extremes, the most harmful cases are roughly twice as harmful (in terms of effect size) as the most beneficial case is beneficial. There seem to be some unique effects for certain pairings of datasets and classifiers. For example, in the case of the Endometrium dataset and the Random Forest classifier, virtually all sample generation methods lead to a sizable improvement. This combination of dataset and classifier leads the field with 6 positions in the top 7 list of beneficial effects. It is rather surprising to see that an ensemble-based classifier, which is supposed to be a remedy for class imbalance in its own right, gets to benefit from other remedies, namely synthetic sample generation methods, but only in the case of one particular dataset.

Annex B

Optimal hyperparameter settings

B.1 Dimensionality reduction experiments

k-NN

Detailed results are shown in Table B.1. For the k-NN classifier, it is interesting to see that often rather large neighborhoods (11 to 30 neighbors) are preferred to determine class membership, with a few exceptions being present (3-8 neighbors). It is also interesting to see that, on average, rather high numbers of independent or principal components are preferred (357-987), with the occasional outlier more in the range of 10-284. Most of the time, a distance-weighted voting is chosen as an optimal hyperparameter when determining class memberships from the k neighbors. This is intuitive, given that a high number of neighbors are frequently chosen, and one would expect these not to be all close by at a similar distance.

Logistic Regression

Detailed results are shown in Table B.2. In the case of Logistic Regression, the ncomp hyperparameter covers quite a range (8-952) of optimal number of principal or independent components, with the bulk of the results being in the upper/middle section of this parameter range. It is interesting to note that in most circumstances, regularization is beneficial (numbers below 1 for the parameter C). With a few exceptions, with values between 13 and 91 (very little regularization).

Naive Bayes

Detailed results are shown in Table B.3. In all experiments, the optimal value for var_smoothing was 0. That indicated that there were no numerical problems with the dataset, and adding small amounts of variance ended up being harmful. Furthermore, it is noteworthy that on average Naive Bayes settled for rather small numbers of independent or principal components (8-72) with some exceptions (171-807) mainly in the Omentum dataset.

Random Forest

Detailed results are shown in Table B.4. The optimal number of trees covered a wide range (103-1000). It is unclear, though, if the performance differences are sizable between these optimal settings and other settings. Theoretically, larger numbers of trees shouldn't harm the results but simply cost more computation power, so usually the upper number of trees is chosen more based on the computational time one is willing to invest than in the hope of achieving measurably better performance with a lower number of trees. The optimal

depth of the decision trees covers a range of 3 to 20. Overall, the Random Forest aims for a comparatively low number of independent or principal components (6-133). Not as stringent as Naive Bayes, but still comparatively low. There aren't any distinct outliers for that pattern.

SVM

Detailed results are shown in Table B.5. High levels of regularization (very low values for C) can be observed with a few outliers. Overall, a rather high number of independent or principal components (251-984) with a few outliers (19-147).

XGBoost

Detailed results are shown in Table B.6. Quite a variety of values for the learning rate (0.01-0.299), with the majority on the lower and middle part of that range, were chosen during the hyperparameter optimization. The maximum tree depth varied over the whole allowed parameter range. The optimal number of trees was comparatively high (122-999), with most cases in the upper and middle part of that range. The optimal number of independent or principal components was in the mid-range compared to other optimal values for other classifiers (13-988).

Hyperparameter importance

We performed a hyperparameter importance analysis of the Optuna hyperparameter optimisation studies. We used the Optuna get_param_importances[1] method with default parameters, so that the FanovaImportanceEvaluator[2] [HHLB14] method is used. The results are shown in Table B.14. The results are shown in Table B.7. In most cases where dimensionality reduction methods were used, the ncomp parameter is by far the most important one. In many cases completely uncontested (Naive Bayes, Random Forest, SVM). In the case of Logistic Regression, the regularization parameter is rather important as well (half of the importance of the ncomp parameter), and for the XGBoost, the learning_rate is of quite some importance as well, at least for the PCA and SVD case. The case of the K-NN classifier is less clear, as for the ICA, the ncomp parameter is by far the most important hyperparameter, while for the PCA and SVD, the n_neighbours parameter comes out twice as important as the ncomp parameter. In the baseline scenario, without any dimensionality reduction methods, we can observe that for the Random Forest, the n_estimators seems to be by a factor of 3 more impactful than the max_depth parameter. For XGBoost, the learning_rate is by far the most important hyperparameter (much more than max_depth and n_estimators). One could try to take these results into consideration for future experiments to reduce the number of hyperparameters to optimize and maybe achieve better stability of relative performance between inner and outer cross-validation.

[1] https://optuna.readthedocs.io/en/stable/reference/generated/optuna.importance.get_param_importances.html

[2] https://optuna.readthedocs.io/en/stable/reference/generated/optuna.importance.FanovaImportanceEvaluator.html

Table B.1: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for k-NN classifier, grouped by dimered method.

dataset	algorithm	dimred	fold	AUC	n_neighbors	weights	ncomp
OVA_Endometrium	k-NN	None	1	0.945	21	1	NA
OVA_Endometrium	k-NN	None	2	0.940	19	1	NA
OVA_Endometrium	k-NN	None	3	0.949	6	1	NA
OVA_Endometrium	k-NN	None	4	0.951	15	1	NA
OVA_Endometrium	k-NN	None	5	0.955	16	1	NA
OVA_Omentum	k-NN	None	1	0.909	14	1	NA
OVA_Omentum	k-NN	None	2	0.890	23	1	NA
OVA_Omentum	k-NN	None	3	0.922	12	1	NA
OVA_Omentum	k-NN	None	4	0.926	22	1	NA
OVA_Omentum	k-NN	None	5	0.908	23	1	NA
OVA_Prostate	k-NN	None	1	0.990	3	0	NA
OVA_Prostate	k-NN	None	2	0.990	12	1	NA
OVA_Prostate	k-NN	None	3	0.990	12	1	NA
OVA_Prostate	k-NN	None	4	0.981	4	0	NA
OVA_Prostate	k-NN	None	5	0.999	16	1	NA
OVA_Endometrium	k-NN	ica	1	0.959	29	1	32
OVA_Endometrium	k-NN	ica	2	0.946	19	1	111
OVA_Endometrium	k-NN	ica	3	0.950	28	1	20
OVA_Endometrium	k-NN	ica	4	0.955	15	1	103
OVA_Endometrium	k-NN	ica	5	0.951	15	1	284
OVA_Omentum	k-NN	ica	1	0.914	30	1	984
OVA_Omentum	k-NN	ica	2	0.887	25	1	21
OVA_Omentum	k-NN	ica	3	0.919	29	1	987
OVA_Omentum	k-NN	ica	4	0.918	22	0	984
OVA_Omentum	k-NN	ica	5	0.919	24	1	19
OVA_Prostate	k-NN	ica	1	1.000	26	1	62
OVA_Prostate	k-NN	ica	2	1.000	30	1	98
OVA_Prostate	k-NN	ica	3	1.000	29	1	49
OVA_Prostate	k-NN	ica	4	0.998	28	0	62
OVA_Prostate	k-NN	ica	5	1.000	27	0	10
OVA_Endometrium	k-NN	pca	1	0.945	20	1	783
OVA_Endometrium	k-NN	pca	2	0.938	20	1	978
OVA_Endometrium	k-NN	pca	3	0.949	8	1	959
OVA_Endometrium	k-NN	pca	4	0.952	15	1	389
OVA_Endometrium	k-NN	pca	5	0.956	15	1	846
OVA_Omentum	k-NN	pca	1	0.911	14	1	559
OVA_Omentum	k-NN	pca	2	0.896	21	0	697
OVA_Omentum	k-NN	pca	3	0.925	12	1	689
OVA_Omentum	k-NN	pca	4	0.926	26	1	820
OVA_Omentum	k-NN	pca	5	0.907	20	1	488
OVA_Prostate	k-NN	pca	1	0.990	8	0	237
OVA_Prostate	k-NN	pca	2	0.990	14	1	465
OVA_Prostate	k-NN	pca	3	0.990	15	1	930
OVA_Prostate	k-NN	pca	4	0.981	3	0	861
OVA_Prostate	k-NN	pca	5	0.999	11	1	33
OVA_Endometrium	k-NN	svd	1	0.946	20	1	788
OVA_Endometrium	k-NN	svd	2	0.939	21	1	902
OVA_Endometrium	k-NN	svd	3	0.949	15	1	624
OVA_Endometrium	k-NN	svd	4	0.951	15	1	482
OVA_Endometrium	k-NN	svd	5	0.957	17	1	357
OVA_Omentum	k-NN	svd	1	0.911	14	1	547
OVA_Omentum	k-NN	svd	2	0.893	21	1	680
OVA_Omentum	k-NN	svd	3	0.924	17	1	290
OVA_Omentum	k-NN	svd	4	0.926	25	0	848
OVA_Omentum	k-NN	svd	5	0.907	18	1	582
OVA_Prostate	k-NN	svd	1	0.990	16	1	16
OVA_Prostate	k-NN	svd	2	0.990	8	1	111
OVA_Prostate	k-NN	svd	3	0.991	8	1	56
OVA_Prostate	k-NN	svd	4	0.981	3	0	784
OVA_Prostate	k-NN	svd	5	0.999	12	1	771

Table B.2: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for Logistic Regression classifier, grouped by dimered method.

dataset	algorithm	dimred	fold	AUC	C	ncomp
OVA_Endometrium	Logistic Regression	None	1	0.945	0.179	NA
OVA_Endometrium	Logistic Regression	None	2	0.943	0.093	NA
OVA_Endometrium	Logistic Regression	None	3	0.949	0.190	NA
OVA_Endometrium	Logistic Regression	None	4	0.945	0.094	NA
OVA_Endometrium	Logistic Regression	None	5	0.946	0.033	NA
OVA_Omentum	Logistic Regression	None	1	0.930	0.127	NA
OVA_Omentum	Logistic Regression	None	2	0.900	0.262	NA
OVA_Omentum	Logistic Regression	None	3	0.945	88.663	NA
OVA_Omentum	Logistic Regression	None	4	0.948	66.960	NA
OVA_Omentum	Logistic Regression	None	5	0.904	0.102	NA
OVA_Prostate	Logistic Regression	None	1	0.999	0.020	NA
OVA_Prostate	Logistic Regression	None	2	0.997	0.060	NA
OVA_Prostate	Logistic Regression	None	3	0.998	5.063	NA
OVA_Prostate	Logistic Regression	None	4	0.998	0.065	NA
OVA_Prostate	Logistic Regression	None	5	1.000	60.666	NA
OVA_Endometrium	Logistic Regression	ica	1	0.960	1.045	39
OVA_Endometrium	Logistic Regression	ica	2	0.955	0.848	72
OVA_Endometrium	Logistic Regression	ica	3	0.966	0.575	75
OVA_Endometrium	Logistic Regression	ica	4	0.963	0.319	150
OVA_Endometrium	Logistic Regression	ica	5	0.955	0.306	114
OVA_Omentum	Logistic Regression	ica	1	0.936	0.421	653
OVA_Omentum	Logistic Regression	ica	2	0.891	0.067	952
OVA_Omentum	Logistic Regression	ica	3	0.935	0.513	205
OVA_Omentum	Logistic Regression	ica	4	0.944	0.514	791
OVA_Omentum	Logistic Regression	ica	5	0.920	0.164	332
OVA_Prostate	Logistic Regression	ica	1	0.999	1.204	155
OVA_Prostate	Logistic Regression	ica	2	0.999	13.756	53
OVA_Prostate	Logistic Regression	ica	3	0.999	0.254	215
OVA_Prostate	Logistic Regression	ica	4	0.998	45.425	369
OVA_Prostate	Logistic Regression	ica	5	1.000	0.053	270
OVA_Endometrium	Logistic Regression	pca	1	0.965	0.115	72
OVA_Endometrium	Logistic Regression	pca	2	0.956	0.062	320
OVA_Endometrium	Logistic Regression	pca	3	0.973	0.095	562
OVA_Endometrium	Logistic Regression	pca	4	0.966	0.072	331
OVA_Endometrium	Logistic Regression	pca	5	0.953	0.022	26
OVA_Omentum	Logistic Regression	pca	1	0.917	0.873	678
OVA_Omentum	Logistic Regression	pca	2	0.896	0.042	195
OVA_Omentum	Logistic Regression	pca	3	0.940	0.153	747
OVA_Omentum	Logistic Regression	pca	4	0.934	0.033	312
OVA_Omentum	Logistic Regression	pca	5	0.898	0.016	536
OVA_Prostate	Logistic Regression	pca	1	0.997	91.671	231
OVA_Prostate	Logistic Regression	pca	2	0.999	0.176	269
OVA_Prostate	Logistic Regression	pca	3	0.999	0.682	544
OVA_Prostate	Logistic Regression	pca	4	0.998	0.174	662
OVA_Prostate	Logistic Regression	pca	5	0.999	0.913	166
OVA_Endometrium	Logistic Regression	svd	1	0.965	0.110	72
OVA_Endometrium	Logistic Regression	svd	2	0.960	0.116	80
OVA_Endometrium	Logistic Regression	svd	3	0.975	0.146	358
OVA_Endometrium	Logistic Regression	svd	4	0.967	0.062	122
OVA_Endometrium	Logistic Regression	svd	5	0.955	0.045	33
OVA_Omentum	Logistic Regression	svd	1	0.920	0.070	434
OVA_Omentum	Logistic Regression	svd	2	0.895	0.045	593
OVA_Omentum	Logistic Regression	svd	3	0.938	0.068	685
OVA_Omentum	Logistic Regression	svd	4	0.932	0.028	827
OVA_Omentum	Logistic Regression	svd	5	0.904	0.025	90
OVA_Prostate	Logistic Regression	svd	1	0.998	9.635	8
OVA_Prostate	Logistic Regression	svd	2	0.999	0.185	66
OVA_Prostate	Logistic Regression	svd	3	0.999	1.497	461
OVA_Prostate	Logistic Regression	svd	4	0.998	0.518	57
OVA_Prostate	Logistic Regression	svd	5	0.999	1.899	75

Table B.3: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for Naive Bayes classifier, grouped by dimered method.

dataset	algorithm	dimred	fold	AUC	var_smoothing	ncomp
OVA_Endometrium	Naive Bayes	None	1	0.819	0	NA
OVA_Endometrium	Naive Bayes	None	2	0.821	0	NA
OVA_Endometrium	Naive Bayes	None	3	0.836	0	NA
OVA_Endometrium	Naive Bayes	None	4	0.814	0	NA
OVA_Endometrium	Naive Bayes	None	5	0.805	0	NA
OVA_Omentum	Naive Bayes	None	1	0.822	0	NA
OVA_Omentum	Naive Bayes	None	2	0.829	0	NA
OVA_Omentum	Naive Bayes	None	3	0.857	0	NA
OVA_Omentum	Naive Bayes	None	4	0.847	0	NA
OVA_Omentum	Naive Bayes	None	5	0.828	0	NA
OVA_Prostate	Naive Bayes	None	1	0.963	0	NA
OVA_Prostate	Naive Bayes	None	2	0.963	0	NA
OVA_Prostate	Naive Bayes	None	3	0.982	0	NA
OVA_Prostate	Naive Bayes	None	4	0.963	0	NA
OVA_Prostate	Naive Bayes	None	5	0.972	0	NA
OVA_Endometrium	Naive Bayes	ica	1	0.948	0	25
OVA_Endometrium	Naive Bayes	ica	2	0.922	0	9
OVA_Endometrium	Naive Bayes	ica	3	0.939	0	26
OVA_Endometrium	Naive Bayes	ica	4	0.939	0	22
OVA_Endometrium	Naive Bayes	ica	5	0.938	0	26
OVA_Omentum	Naive Bayes	ica	1	0.868	0	16
OVA_Omentum	Naive Bayes	ica	2	0.860	0	8
OVA_Omentum	Naive Bayes	ica	3	0.893	0	15
OVA_Omentum	Naive Bayes	ica	4	0.907	0	15
OVA_Omentum	Naive Bayes	ica	5	0.876	0	9
OVA_Prostate	Naive Bayes	ica	1	0.999	0	41
OVA_Prostate	Naive Bayes	ica	2	0.999	0	18
OVA_Prostate	Naive Bayes	ica	3	1.000	0	50
OVA_Prostate	Naive Bayes	ica	4	0.999	0	31
OVA_Prostate	Naive Bayes	ica	5	1.000	0	15
OVA_Endometrium	Naive Bayes	pca	1	0.925	0	19
OVA_Endometrium	Naive Bayes	pca	2	0.919	0	18
OVA_Endometrium	Naive Bayes	pca	3	0.936	0	61
OVA_Endometrium	Naive Bayes	pca	4	0.935	0	21
OVA_Endometrium	Naive Bayes	pca	5	0.938	0	27
OVA_Omentum	Naive Bayes	pca	1	0.911	0	765
OVA_Omentum	Naive Bayes	pca	2	0.850	0	17
OVA_Omentum	Naive Bayes	pca	3	0.894	0	171
OVA_Omentum	Naive Bayes	pca	4	0.891	0	19
OVA_Omentum	Naive Bayes	pca	5	0.903	0	807
OVA_Prostate	Naive Bayes	pca	1	0.999	0	44
OVA_Prostate	Naive Bayes	pca	2	0.998	0	9
OVA_Prostate	Naive Bayes	pca	3	0.997	0	21
OVA_Prostate	Naive Bayes	pca	4	0.995	0	21
OVA_Prostate	Naive Bayes	pca	5	1.000	0	10
OVA_Endometrium	Naive Bayes	svd	1	0.925	0	19
OVA_Endometrium	Naive Bayes	svd	2	0.918	0	16
OVA_Endometrium	Naive Bayes	svd	3	0.936	0	72
OVA_Endometrium	Naive Bayes	svd	4	0.935	0	21
OVA_Endometrium	Naive Bayes	svd	5	0.938	0	27
OVA_Omentum	Naive Bayes	svd	1	0.910	0	765
OVA_Omentum	Naive Bayes	svd	2	0.851	0	675
OVA_Omentum	Naive Bayes	svd	3	0.899	0	722
OVA_Omentum	Naive Bayes	svd	4	0.892	0	19
OVA_Omentum	Naive Bayes	svd	5	0.901	0	797
OVA_Prostate	Naive Bayes	svd	1	0.999	0	44
OVA_Prostate	Naive Bayes	svd	2	0.998	0	9
OVA_Prostate	Naive Bayes	svd	3	0.997	0	20
OVA_Prostate	Naive Bayes	svd	4	0.995	0	21
OVA_Prostate	Naive Bayes	svd	5	1.000	0	10

Table B.4: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for Random Forest classifier, grouped by dimered method.

dataset	algorithm	dimred	fold	AUC	n_estimators	max_depth	ncomp
OVA_Endometrium	Random Forest	None	1	0.942	440	4	NA
OVA_Endometrium	Random Forest	None	2	0.938	598	4	NA
OVA_Endometrium	Random Forest	None	3	0.941	640	4	NA
OVA_Endometrium	Random Forest	None	4	0.938	827	5	NA
OVA_Endometrium	Random Forest	None	5	0.949	663	3	NA
OVA_Omentum	Random Forest	None	1	0.935	197	20	NA
OVA_Omentum	Random Forest	None	2	0.913	698	6	NA
OVA_Omentum	Random Forest	None	3	0.945	272	18	NA
OVA_Omentum	Random Forest	None	4	0.943	101	6	NA
OVA_Omentum	Random Forest	None	5	0.938	452	6	NA
OVA_Prostate	Random Forest	None	1	0.999	210	11	NA
OVA_Prostate	Random Forest	None	2	0.998	615	10	NA
OVA_Prostate	Random Forest	None	3	0.999	108	4	NA
OVA_Prostate	Random Forest	None	4	0.999	210	5	NA
OVA_Prostate	Random Forest	None	5	1.000	105	17	NA
OVA_Endometrium	Random Forest	ica	1	0.951	787	18	22
OVA_Endometrium	Random Forest	ica	2	0.936	304	10	23
OVA_Endometrium	Random Forest	ica	3	0.941	963	16	25
OVA_Endometrium	Random Forest	ica	4	0.942	784	19	22
OVA_Endometrium	Random Forest	ica	5	0.940	855	17	35
OVA_Omentum	Random Forest	ica	1	0.903	618	9	65
OVA_Omentum	Random Forest	ica	2	0.898	651	4	37
OVA_Omentum	Random Forest	ica	3	0.926	153	16	29
OVA_Omentum	Random Forest	ica	4	0.907	503	19	28
OVA_Omentum	Random Forest	ica	5	0.927	524	6	19
OVA_Prostate	Random Forest	ica	1	1.000	906	3	75
OVA_Prostate	Random Forest	ica	2	1.000	257	7	72
OVA_Prostate	Random Forest	ica	3	0.999	932	16	48
OVA_Prostate	Random Forest	ica	4	0.999	376	8	68
OVA_Prostate	Random Forest	ica	5	1.000	403	10	99
OVA_Endometrium	Random Forest	pca	1	0.934	501	19	23
OVA_Endometrium	Random Forest	pca	2	0.906	756	9	35
OVA_Endometrium	Random Forest	pca	3	0.943	338	9	34
OVA_Endometrium	Random Forest	pca	4	0.920	935	11	33
OVA_Endometrium	Random Forest	pca	5	0.934	528	7	65
OVA_Omentum	Random Forest	pca	1	0.903	708	10	133
OVA_Omentum	Random Forest	pca	2	0.893	594	13	18
OVA_Omentum	Random Forest	pca	3	0.912	346	11	28
OVA_Omentum	Random Forest	pca	4	0.902	614	19	26
OVA_Omentum	Random Forest	pca	5	0.912	913	17	14
OVA_Prostate	Random Forest	pca	1	0.998	140	10	82
OVA_Prostate	Random Forest	pca	2	0.995	375	5	11
OVA_Prostate	Random Forest	pca	3	0.989	746	14	77
OVA_Prostate	Random Forest	pca	4	0.990	114	19	48
OVA_Prostate	Random Forest	pca	5	0.998	140	15	101
OVA_Endometrium	Random Forest	svd	1	0.930	400	17	25
OVA_Endometrium	Random Forest	svd	2	0.909	988	7	6
OVA_Endometrium	Random Forest	svd	3	0.944	1000	6	28
OVA_Endometrium	Random Forest	svd	4	0.921	639	20	24
OVA_Endometrium	Random Forest	svd	5	0.935	590	20	70
OVA_Omentum	Random Forest	svd	1	0.907	899	18	74
OVA_Omentum	Random Forest	svd	2	0.891	599	18	18
OVA_Omentum	Random Forest	svd	3	0.912	663	15	59
OVA_Omentum	Random Forest	svd	4	0.903	644	20	28
OVA_Omentum	Random Forest	svd	5	0.909	711	14	14
OVA_Prostate	Random Forest	svd	1	0.997	275	8	76
OVA_Prostate	Random Forest	svd	2	0.995	357	9	15
OVA_Prostate	Random Forest	svd	3	0.991	320	18	88
OVA_Prostate	Random Forest	svd	4	0.992	103	12	23
OVA_Prostate	Random Forest	svd	5	0.997	852	17	51

Table B.5: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for SVM (Linear) classifier, grouped by dimered method.

dataset	algorithm	dimred	fold	AUC	C	ncomp
OVA_Endometrium	SVM (Linear)	None	1	0.950	0.001	NA
OVA_Endometrium	SVM (Linear)	None	2	0.942	0.000	NA
OVA_Endometrium	SVM (Linear)	None	3	0.968	0.001	NA
OVA_Endometrium	SVM (Linear)	None	4	0.939	0.001	NA
OVA_Endometrium	SVM (Linear)	None	5	0.940	0.000	NA
OVA_Omentum	SVM (Linear)	None	1	0.916	0.000	NA
OVA_Omentum	SVM (Linear)	None	2	0.870	0.000	NA
OVA_Omentum	SVM (Linear)	None	3	0.933	0.001	NA
OVA_Omentum	SVM (Linear)	None	4	0.938	0.001	NA
OVA_Omentum	SVM (Linear)	None	5	0.890	0.000	NA
OVA_Prostate	SVM (Linear)	None	1	0.999	0.000	NA
OVA_Prostate	SVM (Linear)	None	2	0.998	0.000	NA
OVA_Prostate	SVM (Linear)	None	3	0.999	0.005	NA
OVA_Prostate	SVM (Linear)	None	4	0.996	0.000	NA
OVA_Prostate	SVM (Linear)	None	5	1.000	0.000	NA
OVA_Endometrium	SVM (Linear)	ica	1	0.949	1.254	75
OVA_Endometrium	SVM (Linear)	ica	2	0.949	0.264	75
OVA_Endometrium	SVM (Linear)	ica	3	0.965	0.075	89
OVA_Endometrium	SVM (Linear)	ica	4	0.951	0.080	85
OVA_Endometrium	SVM (Linear)	ica	5	0.944	81.632	48
OVA_Omentum	SVM (Linear)	ica	1	0.921	0.003	699
OVA_Omentum	SVM (Linear)	ica	2	0.888	0.000	852
OVA_Omentum	SVM (Linear)	ica	3	0.938	0.034	833
OVA_Omentum	SVM (Linear)	ica	4	0.944	0.005	547
OVA_Omentum	SVM (Linear)	ica	5	0.910	0.000	929
OVA_Prostate	SVM (Linear)	ica	1	1.000	0.045	147
OVA_Prostate	SVM (Linear)	ica	2	0.999	0.037	48
OVA_Prostate	SVM (Linear)	ica	3	0.999	51.918	615
OVA_Prostate	SVM (Linear)	ica	4	0.999	0.067	49
OVA_Prostate	SVM (Linear)	ica	5	1.000	0.028	68
OVA_Endometrium	SVM (Linear)	pca	1	0.950	0.000	33
OVA_Endometrium	SVM (Linear)	pca	2	0.954	0.001	262
OVA_Endometrium	SVM (Linear)	pca	3	0.969	36.129	859
OVA_Endometrium	SVM (Linear)	pca	4	0.954	0.001	132
OVA_Endometrium	SVM (Linear)	pca	5	0.951	0.016	121
OVA_Omentum	SVM (Linear)	pca	1	0.919	0.000	476
OVA_Omentum	SVM (Linear)	pca	2	0.871	0.000	549
OVA_Omentum	SVM (Linear)	pca	3	0.942	0.006	828
OVA_Omentum	SVM (Linear)	pca	4	0.939	0.001	898
OVA_Omentum	SVM (Linear)	pca	5	0.893	0.000	666
OVA_Prostate	SVM (Linear)	pca	1	0.999	0.000	445
OVA_Prostate	SVM (Linear)	pca	2	0.998	0.000	49
OVA_Prostate	SVM (Linear)	pca	3	0.999	24.601	48
OVA_Prostate	SVM (Linear)	pca	4	0.998	0.000	19
OVA_Prostate	SVM (Linear)	pca	5	1.000	0.000	430
OVA_Endometrium	SVM (Linear)	svd	1	0.952	88.452	34
OVA_Endometrium	SVM (Linear)	svd	2	0.958	0.001	75
OVA_Endometrium	SVM (Linear)	svd	3	0.970	0.004	859
OVA_Endometrium	SVM (Linear)	svd	4	0.955	0.002	124
OVA_Endometrium	SVM (Linear)	svd	5	0.945	0.001	112
OVA_Omentum	SVM (Linear)	svd	1	0.915	0.000	984
OVA_Omentum	SVM (Linear)	svd	2	0.870	0.000	976
OVA_Omentum	SVM (Linear)	svd	3	0.941	0.003	829
OVA_Omentum	SVM (Linear)	svd	4	0.939	0.000	706
OVA_Omentum	SVM (Linear)	svd	5	0.895	0.000	643
OVA_Prostate	SVM (Linear)	svd	1	1.000	0.000	251
OVA_Prostate	SVM (Linear)	svd	2	0.999	0.001	63
OVA_Prostate	SVM (Linear)	svd	3	0.999	0.000	592
OVA_Prostate	SVM (Linear)	svd	4	0.999	0.001	57
OVA_Prostate	SVM (Linear)	svd	5	1.000	0.000	485

Table B.6: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for XGBoost classifier, grouped by dimered method.

dataset	algorithm	dimred	fold	AUC	learning_rate	max_depth	n_estimators	ncomp
OVA_Endometrium	XGBoost	None	1	0.966	0.259	9	155	NA
OVA_Endometrium	XGBoost	None	2	0.960	0.189	4	290	NA
OVA_Endometrium	XGBoost	None	3	0.978	0.087	6	139	NA
OVA_Endometrium	XGBoost	None	4	0.961	0.098	3	295	NA
OVA_Endometrium	XGBoost	None	5	0.978	0.079	6	931	NA
OVA_Omentum	XGBoost	None	1	0.946	0.103	9	959	NA
OVA_Omentum	XGBoost	None	2	0.916	0.182	6	146	NA
OVA_Omentum	XGBoost	None	3	0.966	0.130	3	958	NA
OVA_Omentum	XGBoost	None	4	0.942	0.176	10	610	NA
OVA_Omentum	XGBoost	None	5	0.929	0.128	10	941	NA
OVA_Prostate	XGBoost	None	1	0.999	0.284	7	521	NA
OVA_Prostate	XGBoost	None	2	0.995	0.010	6	212	NA
OVA_Prostate	XGBoost	None	3	0.996	0.014	7	152	NA
OVA_Prostate	XGBoost	None	4	0.996	0.174	5	798	NA
OVA_Prostate	XGBoost	None	5	0.999	0.265	9	899	NA
OVA_Endometrium	XGBoost	ica	1	0.948	0.010	10	927	28
OVA_Endometrium	XGBoost	ica	2	0.937	0.110	5	819	28
OVA_Endometrium	XGBoost	ica	3	0.952	0.025	5	733	22
OVA_Endometrium	XGBoost	ica	4	0.942	0.017	7	122	20
OVA_Endometrium	XGBoost	ica	5	0.948	0.070	6	780	24
OVA_Omentum	XGBoost	ica	1	0.903	0.174	5	237	125
OVA_Omentum	XGBoost	ica	2	0.881	0.236	6	379	145
OVA_Omentum	XGBoost	ica	3	0.936	0.065	5	643	110
OVA_Omentum	XGBoost	ica	4	0.921	0.120	4	695	162
OVA_Omentum	XGBoost	ica	5	0.923	0.103	8	534	224
OVA_Prostate	XGBoost	ica	1	1.000	0.159	10	580	42
OVA_Prostate	XGBoost	ica	2	0.999	0.277	7	158	35
OVA_Prostate	XGBoost	ica	3	1.000	0.155	8	997	24
OVA_Prostate	XGBoost	ica	4	0.999	0.057	8	845	19
OVA_Prostate	XGBoost	ica	5	1.000	0.010	8	999	93
OVA_Endometrium	XGBoost	pca	1	0.940	0.145	3	937	128
OVA_Endometrium	XGBoost	pca	2	0.943	0.139	8	506	170
OVA_Endometrium	XGBoost	pca	3	0.965	0.071	6	653	123
OVA_Endometrium	XGBoost	pca	4	0.939	0.117	3	470	216
OVA_Endometrium	XGBoost	pca	5	0.943	0.246	9	228	65
OVA_Omentum	XGBoost	pca	1	0.917	0.138	3	768	577
OVA_Omentum	XGBoost	pca	2	0.902	0.086	9	283	881
OVA_Omentum	XGBoost	pca	3	0.933	0.080	3	701	362
OVA_Omentum	XGBoost	pca	4	0.909	0.165	7	812	313
OVA_Omentum	XGBoost	pca	5	0.908	0.070	6	199	106
OVA_Prostate	XGBoost	pca	1	0.996	0.030	7	249	245
OVA_Prostate	XGBoost	pca	2	0.999	0.276	5	983	241
OVA_Prostate	XGBoost	pca	3	0.995	0.260	8	355	64
OVA_Prostate	XGBoost	pca	4	0.997	0.262	5	803	13
OVA_Prostate	XGBoost	pca	5	0.997	0.299	8	632	984
OVA_Endometrium	XGBoost	svd	1	0.945	0.243	10	953	94
OVA_Endometrium	XGBoost	svd	2	0.932	0.020	4	991	129
OVA_Endometrium	XGBoost	svd	3	0.962	0.034	4	950	150
OVA_Endometrium	XGBoost	svd	4	0.934	0.147	4	348	132
OVA_Endometrium	XGBoost	svd	5	0.950	0.216	9	194	69
OVA_Omentum	XGBoost	svd	1	0.916	0.077	6	841	391
OVA_Omentum	XGBoost	svd	2	0.897	0.059	6	491	757
OVA_Omentum	XGBoost	svd	3	0.934	0.195	5	283	179
OVA_Omentum	XGBoost	svd	4	0.907	0.086	7	966	209
OVA_Omentum	XGBoost	svd	5	0.905	0.037	9	577	110
OVA_Prostate	XGBoost	svd	1	0.995	0.236	9	345	52
OVA_Prostate	XGBoost	svd	2	0.999	0.084	7	177	89
OVA_Prostate	XGBoost	svd	3	0.995	0.189	6	989	186
OVA_Prostate	XGBoost	svd	4	0.996	0.142	9	739	14
OVA_Prostate	XGBoost	svd	5	0.998	0.253	9	223	988

Table B.7: Average Hyperparameter Importance by classifier and dimensionality reduction method averaged over all datasets and folds of inner cross-validation

model	dimred	C	ncomp	var_smoothing	max_depth	n_estimators	learning_rate	n_neighbors	weights
Logistic Regression									
Logistic Regression	None	1.000	NA	NA	NA	NA	NA	NA	NA
Logistic Regression	ica	0.299	0.701	NA	NA	NA	NA	NA	NA
Logistic Regression	pca	0.336	0.664	NA	NA	NA	NA	NA	NA
Logistic Regression	svd	0.386	0.614	NA	NA	NA	NA	NA	NA
Naive Bayes									
Naive Bayes	None	NA	NA	1.000	NA	NA	NA	NA	NA
Naive Bayes	ica	NA	1.000	0.000	NA	NA	NA	NA	NA
Naive Bayes	pca	NA	0.995	0.005	NA	NA	NA	NA	NA
Naive Bayes	svd	NA	0.985	0.015	NA	NA	NA	NA	NA
Random Forest									
Random Forest	None	NA	NA	NA	0.271	0.729	NA	NA	NA
Random Forest	ica	NA	0.996	NA	0.003	0.001	NA	NA	NA
Random Forest	pca	NA	0.943	NA	0.039	0.017	NA	NA	NA
Random Forest	svd	NA	0.935	NA	0.046	0.020	NA	NA	NA
SVM (Linear)									
SVM (Linear)	None	1.000	NA	NA	NA	NA	NA	NA	NA
SVM (Linear)	ica	0.028	0.972	NA	NA	NA	NA	NA	NA
SVM (Linear)	pca	0.019	0.981	NA	NA	NA	NA	NA	NA
SVM (Linear)	svd	0.014	0.986	NA	NA	NA	NA	NA	NA
XGBoost									
XGBoost	None	NA	NA	NA	0.065	0.156	0.779	NA	NA
XGBoost	ica	NA	0.996	NA	0.001	0.001	0.002	NA	NA
XGBoost	pca	NA	0.697	NA	0.027	0.067	0.209	NA	NA
XGBoost	svd	NA	0.698	NA	0.023	0.063	0.215	NA	NA
k-NN									
k-NN	None	NA	NA	NA	NA	NA	NA	0.985	0.015
k-NN	ica	NA	0.907	NA	NA	NA	NA	0.085	0.008
k-NN	pca	NA	0.259	NA	NA	NA	NA	0.732	0.009
k-NN	svd	NA	0.273	NA	NA	NA	NA	0.713	0.014

B.2 Sample generation experiments

k-NN

Detailed results are shown in Table B.8. For the k-NN classifier, it is interesting to see that often rather large neighborhoods (28 to 30 neighbors) are preferred to determine class membership, with a few exceptions being present (3-26 neighbors). About half of the time, a distance-weighted voting is chosen as an optimal hyperparameter when determining class memberships from the k neighbors. The smote_k parameter for the SMOTE sample generation algorithm shows variability over the whole spectrum of allowed values (2-10 neighbors). This is also the case for all other SMOTE variants (ADASYN, BorderlineSMOTE, SVMSMOTE). The smote_m parameter for BorderlineSMOTE leans towards the higher end of the allowed parameter range, with a few exceptions in the lower range that finding is mirrored for the same parameter in SVMSMOTE. The optimal choices for min_leaf_purity and max_leaf_purity for our RandomForestSMOTE algorithm interestingly clustered around the middle section of the allowed parameter range. So neither did the algorithm favor a setting of extreme required leaf purity, nor did it favor highly contested parts of the feature space (borderline region); it settled somewhere in the middle of the two extremes (with one exception). In most cases, the algorithm favored shallow trees of 4-6 levels with some exceptions (6-10) levels. The results show quite some variability with regard to the choice for the top_pair_fraction of the neighborhood matrix. Sometimes the algorithm showed a tendency to only select samples that have very strong neighbors for interpolation (top_pair_fraction close to 5%), while sometimes the algorithm was more lenient (top_pair_fraction close to 10%).

Logistic Regression

Detailed results are shown in Table B.9. The regularization parameter showed a rather high variability between as well as within datasets, classifiers, and sample generation algorithms. Overall, the Endometrium dataset favored comparatively consistently a high level of regularization (low values of C below 1). The Prostate dataset, in return, favored comparatively consistently a low level of regularization (high values of C above 1). The Omentum dataset sat somewhere in between, showing a high level of variability of the optimal C value across different settings (classifier and sample generators). The smote_k parameter for the SMOTE sample generation algorithm shows variability over the whole spectrum of allowed values (2-10 neighbors). This is also the case for all other SMOTE variants (ADASYN, BorderlineSMOTE, SVMSMOTE). The smote_m parameter for BorderlineSMOTE leans towards the higher end of the allowed parameter range, with a few exceptions in the lower range. The same parameter in SVMSMOTE leans more towards the middle of the allowed parameter range. The optimal choices for min_leaf_purity and max_leaf_purity for our RandomForestSMOTE algorithm interestingly clustered more around the lower/mid section of the allowed parameter range (particularly in comparison to the results of the k-NN parameter optimization results). In many cases, the parameter choices allowed for a rather broad interval for the leaf purity, maximizing the number of leaves taken into account. The number of levels in the trees (os_maxdepth) showed variability across the whole spectrum of the allowed parameter range. The results also show quite some variability with regard to the choice for the top_pair_fraction of the neighborhood matrix. Sometimes the algorithm showed a tendency to only select samples that have very strong neighbors for interpolation (top_pair_fraction close to 5%), while sometimes the algorithm was more lenient (top_pair_fraction at the maximum of 10%).

Naive Bayes

Detailed results are shown in Table B.10. In all experiments, the optimal value for var_smoothing was 0. That indicated that there were no numerical problems with the dataset, and adding small amounts of variance ended up being harmful. The smote_k parameter for the SMOTE sample generation algorithm shows variability over the whole spectrum of allowed values (2-10 neighbors). This is also the case for all other SMOTE variants (ADASYN, BorderlineSMOTE, SVMSMOTE). The smote_m parameter for BorderlineSMOTE showed high variability across the allowed parameter range, with the same finding for the SVMSMOTE case. The optimal choices for min_leaf_purity and max_leaf_purity for our RandomForestSMOTE algorithm are clustered more around the upper/mid section of the allowed parameter range (with a few exceptions). In many cases, the parameter choices allowed for a rather narrow interval for the leaf purity, being selective about what leaves are taken into account. The number of levels in the trees (os_maxdepth) leaned towards rather shallow trees (with few exceptions). The results also show a preference for the high end of the parameter range of the top_pair_fraction of the neighborhood matrix (with few exceptions).

Random Forest

Detailed results are shown in Table B.11. The optimal number of trees covered a wide range (100-989). It is unclear, though, if the performance differences are sizable between these optimal settings and other settings. Theoretically, larger numbers of trees shouldn't harm the results but simply cost more computation power, so usually the upper number of trees is chosen more based on the computational time one is willing to invest than in the hope of achieving measurably better performance with a lower number of trees. The optimal depth of the decision trees covers the whole allowed range of 2 to 20 with high levels of variability. The smote_k parameter for the SMOTE sample generation algorithm shows variability over the whole spectrum of allowed values (2-10 neighbors). This is also the case for all other SMOTE variants (ADASYN, BorderlineSMOTE, SVMSMOTE). The smote_m parameter for BorderlineSMOTE showed high variability across the allowed parameter range, with the same finding for the SVMSMOTE case. The optimal choices for min_leaf_purity and max_leaf_purity for our RandomForestSMOTE algorithm are clustered more around the upper/mid section of the allowed parameter range (with a few exceptions). In many cases, the parameter choices allowed for a rather narrow interval for the leaf purity, being selective about what leaves are taken into account. The number of levels in the trees (os_maxdepth) leaned towards rather shallow trees (with some exceptions). The results for the top_pair_fraction of the neighborhood matrix vary across the whole allowed parameter range.

SVM

Detailed results are shown in Table B.12. High levels of regularization (very low values for C, with most of them zero) can be observed with very few outliers. The smote_k parameter for the SMOTE sample generation algorithm shows variability over the whole spectrum of allowed values (2-10 neighbors). This is also the case for all other SMOTE variants (ADASYN, BorderlineSMOTE, SVMSMOTE). The smote_m parameter for BorderlineSMOTE showed high variability across the allowed parameter range, with the same finding for the SVMSMOTE case. The optimal choices for min_leaf_purity and max_leaf_purity for our RandomForestSMOTE algorithm are clustered more around the lower/mid section of the allowed parameter range (with a few exceptions). In many cases, the parameter choices allowed for a rather wide interval for the leaf purity, being lenient about what leaves are

taken into account. The number of levels in the trees (os_maxdepth) is spread out over the whole allowed parameter range. The results for the top_pair_fraction of the neighborhood matrix vary across the whole allowed parameter range.

XGBoost

Detailed results are shown in Table B.13. Quite a variety of values for the learning rate (0.01-0.299), with the majority in the middle and upper part of that range. The maximum tree depth varied over the whole parameter range. The optimal number of trees also had a high variability across the parameter spectrum (106-995), with most cases in the upper and middle part of that range. The smote_k parameter for the SMOTE sample generation algorithm shows variability over the whole spectrum of allowed values (2-10 neighbors). This is also the case for all other SMOTE variants (ADASYN, BorderlineSMOTE, SVMSMOTE). The smote_m parameter for BorderlineSMOTE showed high variability across the allowed parameter range, with the same finding for the SVMSMOTE case. The optimal choices for min_leaf_purity and max_leaf_purity for our RandomForestSMOTE algorithm are clustered more around the lower/mid section of the allowed parameter range (with a few exceptions). In many cases, the parameter choices allowed for a rather wide interval for the leaf purity, being lenient about what leaves are taken into account. The number of levels in the trees (os_maxdepth) is spread out over the whole allowed parameter range. The results for the top_pair_fraction of the neighborhood matrix clusters around the mid-section of the allowed parameter range.

Hyperparameter importance

We performed a hyperparameter importance analysis of the Optuna hyperparameter optimisation studies. We used the Optuna get_param_importances[3] method with default parameters, so that the FanovaImportanceEvaluator[4] [HHLB14] method is used. The results are shown in Table B.14. For Logistic Regression, the regularisation parameter C was in all cases the single most important parameter. In combination with the RandomForestSMOTE algorithm is was less pronounced, though, as importance had to be shared with some of the hyperparameters of that sample generation algorithm. For Naive Bayes, the var_smoothing parameter was only of importance in the case of the Random Oversampling algorithm. The smote_k parameter is the most important parameter by far for Naive Bayes in combination with plain SMOTE as well as ADASYN. Interestingly, in the case of borderline SMOTE and SVM SMOTE, the smote_m parameter seems to be far more important. For the RandomForestSMOTE in combination with Naive Bayes, the min_leaf_purity and the os_max_depth parameters seem to be the two most important ones. The Random Forest classifier exhibits some interesting patterns in the sense that the importance of the hyperparameters of the actual classifier by far dominate the importance of the hyperparameters of any of the sample generation algorithms. This is, to a degree, intuitive, as the Random Forest, as an ensemble method, is a remedy for class imbalance in its own right, and tuning its hyperparameters seemingly provides the best result improvements. The SVM classifier is a mixed bag in terms of hyperparameter importance. For ADASYN and plain SMOTE, the smote_k parameter is most important. For Borderline SMOTE and SVM SMOTE, once more, the smote_m parameter is the most important one. For the SVM classifier with RandomForestSMOTE, the

[3] https://optuna.readthedocs.io/en/stable/reference/generated/optuna.importance.get_param_importances.html

[4] https://optuna.readthedocs.io/en/stable/reference/generated/optuna.importance.FanovaImportanceEvaluator.html

3 hyperparameters of the RandomForestSMOTE method are the most important ones. The k-NN classifier is an interesting, clear-cut case where the hyperparameter of the classifier itself (n_neighbors) is by far the most important hyperparameter across all sample generation methods. For the XGBoost, the hyperparameters belonging to the classifier itself are certainly in aggregate the most important ones, with the learning_rate frequently leading the field. Interestingly, in the case of ADASYN, the smote_k parameter turns out to be more important than the learning_rate of the classifier.
It is noteworthy that the max_leaf_purity parameter of the RandomForestSMOTE algorithm did not make the cut as an important hyperparameter at all and should therefore probably be abandoned. This parameter was supposed to allow the optimization pipeline to push for selecting samples in the contested borderline region of the two classes if that turns out to be more beneficial to the performance. At least in our experiments, that seems not to have been a setting that sufficiently influenced performance. Maybe for different datasets, the situation might change.

Table B.8: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for k-NN classifier, grouped by sample generation method.

dataset	algorithm	oversampling	fold	AUC	smote_k	n_neighbors	weights	smote_m	min_leaf_purity	max_leaf_purity	top_pair_fraction	os_max_depth
OVA_Endometrium	k-NN	ADASYN	1	0.909	3	29	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	ADASYN	2	0.907	5	29	0	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	ADASYN	3	0.918	2	30	0	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	ADASYN	4	0.923	6	30	0	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	ADASYN	5	0.918	9	28	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	ADASYN	1	0.877	10	30	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	ADASYN	2	0.852	10	30	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	ADASYN	3	0.899	3	24	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	ADASYN	4	0.892	2	29	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	ADASYN	5	0.875	3	18	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	ADASYN	1	0.995	9	25	0	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	ADASYN	2	0.992	8	30	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	ADASYN	3	0.993	5	30	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	ADASYN	4	0.987	9	28	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	ADASYN	5	0.995	10	30	0	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	BorderlineSMOTE	1	0.909	2	30	1	9	NA	NA	NA	NA
OVA_Endometrium	k-NN	BorderlineSMOTE	2	0.904	3	28	0	12	NA	NA	NA	NA
OVA_Endometrium	k-NN	BorderlineSMOTE	3	0.923	7	29	1	18	NA	NA	NA	NA
OVA_Endometrium	k-NN	BorderlineSMOTE	4	0.928	4	28	0	16	NA	NA	NA	NA
OVA_Endometrium	k-NN	BorderlineSMOTE	5	0.923	3	18	1	16	NA	NA	NA	NA
OVA_Omentum	k-NN	BorderlineSMOTE	1	0.875	10	30	1	15	NA	NA	NA	NA
OVA_Omentum	k-NN	BorderlineSMOTE	2	0.855	7	30	0	9	NA	NA	NA	NA
OVA_Omentum	k-NN	BorderlineSMOTE	3	0.893	9	29	1	20	NA	NA	NA	NA
OVA_Omentum	k-NN	BorderlineSMOTE	4	0.892	8	30	1	6	NA	NA	NA	NA
OVA_Omentum	k-NN	BorderlineSMOTE	5	0.874	7	18	1	5	NA	NA	NA	NA
OVA_Prostate	k-NN	BorderlineSMOTE	1	0.998	10	30	1	14	NA	NA	NA	NA
OVA_Prostate	k-NN	BorderlineSMOTE	2	0.996	10	21	1	17	NA	NA	NA	NA
OVA_Prostate	k-NN	BorderlineSMOTE	3	0.991	8	28	1	20	NA	NA	NA	NA
OVA_Prostate	k-NN	BorderlineSMOTE	4	0.987	2	29	0	11	NA	NA	NA	NA
OVA_Prostate	k-NN	BorderlineSMOTE	5	0.997	4	4	1	16	NA	NA	NA	NA
OVA_Endometrium	k-NN	RandomForestSMOTE	1	0.946	NA	24	0	NA	0.752	0.934	0.062	5
OVA_Endometrium	k-NN	RandomForestSMOTE	2	0.941	NA	30	1	NA	0.757	0.902	0.081	5
OVA_Endometrium	k-NN	RandomForestSMOTE	3	0.957	NA	25	1	NA	0.746	0.935	0.065	5
OVA_Endometrium	k-NN	RandomForestSMOTE	4	0.950	NA	26	0	NA	0.670	0.883	0.075	4
OVA_Endometrium	k-NN	RandomForestSMOTE	5	0.941	NA	6	1	NA	0.684	0.851	0.078	7
OVA_Omentum	k-NN	RandomForestSMOTE	1	0.896	NA	14	1	NA	0.718	0.905	0.077	4
OVA_Omentum	k-NN	RandomForestSMOTE	2	0.883	NA	26	0	NA	0.564	0.860	0.054	8
OVA_Omentum	k-NN	RandomForestSMOTE	3	0.921	NA	30	1	NA	0.712	0.845	0.094	5
OVA_Omentum	k-NN	RandomForestSMOTE	4	0.915	NA	25	0	NA	0.785	0.891	0.093	5
OVA_Omentum	k-NN	RandomForestSMOTE	5	0.906	NA	13	0	NA	0.790	0.909	0.098	4
OVA_Prostate	k-NN	RandomForestSMOTE	1	0.999	NA	7	0	NA	0.662	0.884	0.055	6
OVA_Prostate	k-NN	RandomForestSMOTE	2	0.998	NA	11	0	NA	0.796	0.936	0.072	4
OVA_Prostate	k-NN	RandomForestSMOTE	3	0.998	NA	9	1	NA	0.532	0.867	0.064	10
OVA_Prostate	k-NN	RandomForestSMOTE	4	0.989	NA	10	1	NA	0.705	0.899	0.075	8
OVA_Prostate	k-NN	RandomForestSMOTE	5	0.999	NA	3	1	NA	0.666	0.866	0.061	4
OVA_Endometrium	k-NN	RandomOverSampler	1	0.938	NA	29	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	RandomOverSampler	2	0.927	NA	30	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	RandomOverSampler	3	0.945	NA	30	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	RandomOverSampler	4	0.943	NA	29	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	RandomOverSampler	5	0.937	NA	30	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	RandomOverSampler	1	0.887	NA	30	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	RandomOverSampler	2	0.867	NA	30	0	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	RandomOverSampler	3	0.901	NA	11	0	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	RandomOverSampler	4	0.896	NA	30	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	RandomOverSampler	5	0.891	NA	11	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	RandomOverSampler	1	0.999	NA	19	0	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	RandomOverSampler	2	0.998	NA	28	0	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	RandomOverSampler	3	0.996	NA	30	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	RandomOverSampler	4	0.988	NA	9	0	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	RandomOverSampler	5	0.998	NA	18	0	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	SMOTE	1	0.904	3	25	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	SMOTE	2	0.909	7	29	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	SMOTE	3	0.922	2	25	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	SMOTE	4	0.928	2	30	0	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	SMOTE	5	0.918	10	20	0	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	SMOTE	1	0.877	10	30	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	SMOTE	2	0.854	7	30	0	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	SMOTE	3	0.893	9	29	1	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	SMOTE	4	0.891	7	30	0	NA	NA	NA	NA	NA
OVA_Omentum	k-NN	SMOTE	5	0.874	9	17	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	SMOTE	1	0.999	9	19	0	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	SMOTE	2	0.997	9	30	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	SMOTE	3	0.997	7	25	1	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	SMOTE	4	0.991	8	15	0	NA	NA	NA	NA	NA
OVA_Prostate	k-NN	SMOTE	5	0.999	2	30	1	NA	NA	NA	NA	NA
OVA_Endometrium	k-NN	SVMSMOTE	1	0.916	8	30	1	20	NA	NA	NA	NA
OVA_Endometrium	k-NN	SVMSMOTE	2	0.913	8	25	1	20	NA	NA	NA	NA
OVA_Endometrium	k-NN	SVMSMOTE	3	0.935	2	30	0	15	NA	NA	NA	NA
OVA_Endometrium	k-NN	SVMSMOTE	4	0.935	2	29	1	16	NA	NA	NA	NA
OVA_Endometrium	k-NN	SVMSMOTE	5	0.922	10	29	1	16	NA	NA	NA	NA
OVA_Omentum	k-NN	SVMSMOTE	1	0.879	7	30	0	19	NA	NA	NA	NA
OVA_Omentum	k-NN	SVMSMOTE	2	0.867	8	30	0	16	NA	NA	NA	NA
OVA_Omentum	k-NN	SVMSMOTE	3	0.898	9	26	1	20	NA	NA	NA	NA
OVA_Omentum	k-NN	SVMSMOTE	4	0.900	3	30	1	19	NA	NA	NA	NA
OVA_Omentum	k-NN	SVMSMOTE	5	0.882	8	8	0	6	NA	NA	NA	NA
OVA_Prostate	k-NN	SVMSMOTE	1	0.997	2	22	1	13	NA	NA	NA	NA
OVA_Prostate	k-NN	SVMSMOTE	2	0.996	10	30	1	17	NA	NA	NA	NA
OVA_Prostate	k-NN	SVMSMOTE	3	0.996	4	16	1	20	NA	NA	NA	NA
OVA_Prostate	k-NN	SVMSMOTE	4	0.986	8	23	0	9	NA	NA	NA	NA
OVA_Prostate	k-NN	SVMSMOTE	5	0.998	2	15	0	20	NA	NA	NA	NA

Table B.9: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for Logistic Regression classifier, grouped by sample generation method.

dataset	algorithm	oversampling	fold	AUC	smote_k	C	smote_m	min_leaf_purity	max_leaf_purity	top_pair_fraction	os_max_depth
OVA_Endometrium	Logistic Regression	ADASYN	1	0.966	4	0.025	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	ADASYN	2	0.963	9	0.039	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	ADASYN	3	0.971	8	13.608	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	ADASYN	4	0.960	3	0.020	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	ADASYN	5	0.964	7	0.039	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	ADASYN	1	0.934	6	5.678	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	ADASYN	2	0.900	5	0.011	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	ADASYN	3	0.955	7	1.729	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	ADASYN	4	0.955	5	0.084	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	ADASYN	5	0.924	6	6.609	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	ADASYN	1	0.999	9	87.576	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	ADASYN	2	0.999	8	39.957	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	ADASYN	3	0.999	8	44.577	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	ADASYN	4	0.997	10	92.721	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	ADASYN	5	1.000	8	5.327	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	BorderlineSMOTE	1	0.968	10	0.062	13	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	BorderlineSMOTE	2	0.960	5	0.039	14	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	BorderlineSMOTE	3	0.972	7	0.079	17	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	BorderlineSMOTE	4	0.959	5	0.028	18	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	BorderlineSMOTE	5	0.960	4	0.055	17	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	BorderlineSMOTE	1	0.939	5	9.828	12	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	BorderlineSMOTE	2	0.902	5	0.019	5	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	BorderlineSMOTE	3	0.956	6	14.509	13	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	BorderlineSMOTE	4	0.956	10	0.062	9	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	BorderlineSMOTE	5	0.921	5	4.353	15	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	BorderlineSMOTE	1	0.999	4	30.721	11	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	BorderlineSMOTE	2	0.999	7	83.613	16	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	BorderlineSMOTE	3	0.999	7	72.488	14	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	BorderlineSMOTE	4	0.998	9	0.066	12	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	BorderlineSMOTE	5	1.000	8	0.078	19	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	RandomForestSMOTE	1	0.962	NA	0.049	NA	0.799	0.981	0.085	4
OVA_Endometrium	Logistic Regression	RandomForestSMOTE	2	0.965	NA	0.048	NA	0.786	0.938	0.085	5
OVA_Endometrium	Logistic Regression	RandomForestSMOTE	3	0.974	NA	0.475	NA	0.644	0.856	0.094	6
OVA_Endometrium	Logistic Regression	RandomForestSMOTE	4	0.959	NA	0.031	NA	0.674	0.869	0.071	4
OVA_Endometrium	Logistic Regression	RandomForestSMOTE	5	0.962	NA	0.192	NA	0.511	0.963	0.054	9
OVA_Omentum	Logistic Regression	RandomForestSMOTE	1	0.941	NA	30.562	NA	0.631	0.797	0.058	4
OVA_Omentum	Logistic Regression	RandomForestSMOTE	2	0.921	NA	0.090	NA	0.510	0.625	0.051	8
OVA_Omentum	Logistic Regression	RandomForestSMOTE	3	0.952	NA	70.925	NA	0.501	0.765	0.061	9
OVA_Omentum	Logistic Regression	RandomForestSMOTE	4	0.958	NA	19.574	NA	0.515	0.964	0.094	6
OVA_Omentum	Logistic Regression	RandomForestSMOTE	5	0.926	NA	98.139	NA	0.501	0.955	0.094	10
OVA_Prostate	Logistic Regression	RandomForestSMOTE	1	0.999	NA	95.301	NA	0.705	0.940	0.077	9
OVA_Prostate	Logistic Regression	RandomForestSMOTE	2	0.999	NA	23.757	NA	0.528	0.819	0.100	8
OVA_Prostate	Logistic Regression	RandomForestSMOTE	3	0.999	NA	29.520	NA	0.720	0.864	0.077	7
OVA_Prostate	Logistic Regression	RandomForestSMOTE	4	0.996	NA	0.003	NA	0.646	0.931	0.066	4
OVA_Prostate	Logistic Regression	RandomForestSMOTE	5	1.000	NA	3.071	NA	0.524	0.748	0.100	9
OVA_Endometrium	Logistic Regression	RandomOverSampler	1	0.971	NA	0.018	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	RandomOverSampler	2	0.968	NA	0.032	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	RandomOverSampler	3	0.974	NA	0.048	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	RandomOverSampler	4	0.965	NA	0.016	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	RandomOverSampler	5	0.965	NA	0.042	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	RandomOverSampler	1	0.934	NA	49.567	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	RandomOverSampler	2	0.903	NA	0.019	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	RandomOverSampler	3	0.954	NA	70.272	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	RandomOverSampler	4	0.960	NA	6.211	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	RandomOverSampler	5	0.919	NA	50.180	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	RandomOverSampler	1	0.999	NA	43.566	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	RandomOverSampler	2	0.999	NA	51.936	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	RandomOverSampler	3	0.999	NA	37.796	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	RandomOverSampler	4	0.994	NA	0.003	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	RandomOverSampler	5	1.000	NA	0.054	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SMOTE	1	0.965	7	0.059	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SMOTE	2	0.965	9	0.029	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SMOTE	3	0.972	4	4.665	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SMOTE	4	0.956	8	0.021	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SMOTE	5	0.964	4	0.047	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SMOTE	1	0.937	4	13.477	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SMOTE	2	0.895	6	0.009	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SMOTE	3	0.957	8	26.305	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SMOTE	4	0.958	9	11.845	NA	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SMOTE	5	0.922	10	39.015	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SMOTE	1	0.999	4	23.775	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SMOTE	2	0.999	6	23.103	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SMOTE	3	1.000	5	39.453	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SMOTE	4	0.998	5	41.037	NA	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SMOTE	5	1.000	7	0.032	NA	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SVMSMOTE	1	0.970	3	0.043	15	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SVMSMOTE	2	0.963	10	0.068	8	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SVMSMOTE	3	0.976	2	0.081	13	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SVMSMOTE	4	0.961	2	0.028	8	NA	NA	NA	NA
OVA_Endometrium	Logistic Regression	SVMSMOTE	5	0.965	9	0.092	11	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SVMSMOTE	1	0.935	2	12.022	13	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SVMSMOTE	2	0.903	8	0.030	7	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SVMSMOTE	3	0.958	3	16.842	14	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SVMSMOTE	4	0.956	4	8.818	12	NA	NA	NA	NA
OVA_Omentum	Logistic Regression	SVMSMOTE	5	0.928	5	75.202	13	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SVMSMOTE	1	0.999	2	38.586	13	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SVMSMOTE	2	0.999	2	45.996	5	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SVMSMOTE	3	0.999	4	96.290	12	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SVMSMOTE	4	0.996	6	0.008	9	NA	NA	NA	NA
OVA_Prostate	Logistic Regression	SVMSMOTE	5	1.000	2	0.050	5	NA	NA	NA	NA

Table B.10: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for Naive Bayes classifier, grouped by sample generation method.

dataset	algorithm	oversampling	fold	AUC	smote_k	var_smoothing	smote_m	min_leaf_purity	max_leaf_purity	top_pair_fraction	os_max_depth
OVA_Endometrium	Naive Bayes	ADASYN	1	0.782	10	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	ADASYN	2	0.801	9	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	ADASYN	3	0.776	6	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	ADASYN	4	0.765	2	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	ADASYN	5	0.781	6	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	ADASYN	1	0.809	5	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	ADASYN	2	0.800	10	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	ADASYN	3	0.811	2	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	ADASYN	4	0.818	5	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	ADASYN	5	0.764	3	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	ADASYN	1	0.963	5	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	ADASYN	2	0.945	10	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	ADASYN	3	0.982	9	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	ADASYN	4	0.954	6	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	ADASYN	5	0.963	10	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	BorderlineSMOTE	1	0.773	4	0	20	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	BorderlineSMOTE	2	0.803	8	0	15	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	BorderlineSMOTE	3	0.778	2	0	17	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	BorderlineSMOTE	4	0.785	5	0	11	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	BorderlineSMOTE	5	0.782	7	0	15	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	BorderlineSMOTE	1	0.804	5	0	18	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	BorderlineSMOTE	2	0.802	5	0	16	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	BorderlineSMOTE	3	0.800	3	0	14	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	BorderlineSMOTE	4	0.804	9	0	7	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	BorderlineSMOTE	5	0.768	10	0	9	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	BorderlineSMOTE	1	0.963	6	0	12	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	BorderlineSMOTE	2	0.963	10	0	5	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	BorderlineSMOTE	3	0.982	4	0	6	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	BorderlineSMOTE	4	0.945	8	0	17	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	BorderlineSMOTE	5	0.963	9	0	6	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	RandomForestSMOTE	1	0.818	NA	0	NA	0.791	0.908	0.073	5
OVA_Endometrium	Naive Bayes	RandomForestSMOTE	2	0.816	NA	0	NA	0.778	0.946	0.085	5
OVA_Endometrium	Naive Bayes	RandomForestSMOTE	3	0.829	NA	0	NA	0.761	0.911	0.091	4
OVA_Endometrium	Naive Bayes	RandomForestSMOTE	4	0.787	NA	0	NA	0.731	0.884	0.100	4
OVA_Endometrium	Naive Bayes	RandomForestSMOTE	5	0.827	NA	0	NA	0.752	0.887	0.080	4
OVA_Omentum	Naive Bayes	RandomForestSMOTE	1	0.832	NA	0	NA	0.778	0.923	0.100	6
OVA_Omentum	Naive Bayes	RandomForestSMOTE	2	0.842	NA	0	NA	0.798	0.983	0.096	4
OVA_Omentum	Naive Bayes	RandomForestSMOTE	3	0.822	NA	0	NA	0.765	0.927	0.091	4
OVA_Omentum	Naive Bayes	RandomForestSMOTE	4	0.794	NA	0	NA	0.655	0.812	0.098	4
OVA_Omentum	Naive Bayes	RandomForestSMOTE	5	0.810	NA	0	NA	0.702	0.852	0.097	5
OVA_Prostate	Naive Bayes	RandomForestSMOTE	1	0.963	NA	0	NA	0.628	0.852	0.091	10
OVA_Prostate	Naive Bayes	RandomForestSMOTE	2	0.982	NA	0	NA	0.669	0.816	0.069	8
OVA_Prostate	Naive Bayes	RandomForestSMOTE	3	0.963	NA	0	NA	0.653	0.819	0.082	7
OVA_Prostate	Naive Bayes	RandomForestSMOTE	4	0.972	NA	0	NA	0.500	0.854	0.091	9
OVA_Prostate	Naive Bayes	RandomForestSMOTE	5	0.972	NA	0	NA	0.612	0.814	0.061	4
OVA_Endometrium	Naive Bayes	RandomOverSampler	1	0.829	NA	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	RandomOverSampler	2	0.819	NA	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	RandomOverSampler	3	0.836	NA	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	RandomOverSampler	4	0.814	NA	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	RandomOverSampler	5	0.805	NA	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	RandomOverSampler	1	0.822	NA	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	RandomOverSampler	2	0.830	NA	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	RandomOverSampler	3	0.857	NA	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	RandomOverSampler	4	0.847	NA	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	RandomOverSampler	5	0.819	NA	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	RandomOverSampler	1	0.963	NA	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	RandomOverSampler	2	0.963	NA	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	RandomOverSampler	3	0.982	NA	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	RandomOverSampler	4	0.963	NA	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	RandomOverSampler	5	0.972	NA	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SMOTE	1	0.792	6	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SMOTE	2	0.803	7	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SMOTE	3	0.778	7	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SMOTE	4	0.775	5	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SMOTE	5	0.782	6	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SMOTE	1	0.811	2	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SMOTE	2	0.800	2	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SMOTE	3	0.804	2	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SMOTE	4	0.810	6	0	NA	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SMOTE	5	0.766	9	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SMOTE	1	0.963	7	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SMOTE	2	0.945	9	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SMOTE	3	0.982	7	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SMOTE	4	0.954	6	0	NA	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SMOTE	5	0.963	3	0	NA	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SVMSMOTE	1	0.824	8	0	8	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SVMSMOTE	2	0.820	8	0	14	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SVMSMOTE	3	0.848	5	0	5	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SVMSMOTE	4	0.829	7	0	7	NA	NA	NA	NA
OVA_Endometrium	Naive Bayes	SVMSMOTE	5	0.801	6	0	12	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SVMSMOTE	1	0.820	4	0	5	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SVMSMOTE	2	0.808	6	0	20	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SVMSMOTE	3	0.827	5	0	7	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SVMSMOTE	4	0.825	3	0	5	NA	NA	NA	NA
OVA_Omentum	Naive Bayes	SVMSMOTE	5	0.773	4	0	5	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SVMSMOTE	1	0.981	7	0	11	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SVMSMOTE	2	0.954	3	0	15	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SVMSMOTE	3	0.982	2	0	20	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SVMSMOTE	4	0.945	5	0	8	NA	NA	NA	NA
OVA_Prostate	Naive Bayes	SVMSMOTE	5	0.972	10	0	18	NA	NA	NA	NA

Table B.11: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for Random Forest classifier, grouped by sample generation method.

dataset	algorithm	oversampling	fold	AUC	smote_k	n_estimators	max_depth	smote_m	min_leaf_purity	max_leaf_purity	top_pair_fraction	os_max_depth
OVA_Endometrium	Random Forest	ADASYN	1	0.959	3	477	18	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	ADASYN	2	0.952	9	232	20	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	ADASYN	3	0.961	3	193	11	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	ADASYN	4	0.953	4	330	15	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	ADASYN	5	0.959	5	989	12	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	ADASYN	1	0.935	8	530	13	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	ADASYN	2	0.927	6	155	15	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	ADASYN	3	0.949	7	503	19	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	ADASYN	4	0.945	8	217	6	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	ADASYN	5	0.947	4	104	19	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	ADASYN	1	0.999	8	214	19	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	ADASYN	2	0.999	10	248	6	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	ADASYN	3	0.999	8	168	7	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	ADASYN	4	0.999	3	386	3	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	ADASYN	5	1.000	3	105	20	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	BorderlineSMOTE	1	0.959	3	319	17	12	NA	NA	NA	NA
OVA_Endometrium	Random Forest	BorderlineSMOTE	2	0.952	7	584	19	18	NA	NA	NA	NA
OVA_Endometrium	Random Forest	BorderlineSMOTE	3	0.962	2	319	9	9	NA	NA	NA	NA
OVA_Endometrium	Random Forest	BorderlineSMOTE	4	0.956	2	297	9	11	NA	NA	NA	NA
OVA_Endometrium	Random Forest	BorderlineSMOTE	5	0.960	9	488	9	14	NA	NA	NA	NA
OVA_Omentum	Random Forest	BorderlineSMOTE	1	0.931	9	508	16	10	NA	NA	NA	NA
OVA_Omentum	Random Forest	BorderlineSMOTE	2	0.922	5	933	18	6	NA	NA	NA	NA
OVA_Omentum	Random Forest	BorderlineSMOTE	3	0.949	3	206	13	17	NA	NA	NA	NA
OVA_Omentum	Random Forest	BorderlineSMOTE	4	0.946	5	474	20	11	NA	NA	NA	NA
OVA_Omentum	Random Forest	BorderlineSMOTE	5	0.943	5	150	18	6	NA	NA	NA	NA
OVA_Prostate	Random Forest	BorderlineSMOTE	1	0.999	7	718	9	9	NA	NA	NA	NA
OVA_Prostate	Random Forest	BorderlineSMOTE	2	0.998	2	473	18	5	NA	NA	NA	NA
OVA_Prostate	Random Forest	BorderlineSMOTE	3	0.997	6	190	6	20	NA	NA	NA	NA
OVA_Prostate	Random Forest	BorderlineSMOTE	4	0.999	4	100	3	16	NA	NA	NA	NA
OVA_Prostate	Random Forest	BorderlineSMOTE	5	1.000	2	525	9	18	NA	NA	NA	NA
OVA_Endometrium	Random Forest	RandomForestSMOTE	1	0.958	NA	883	9	NA	0.795	0.952	0.052	4
OVA_Endometrium	Random Forest	RandomForestSMOTE	2	0.954	NA	363	12	NA	0.782	0.988	0.078	4
OVA_Endometrium	Random Forest	RandomForestSMOTE	3	0.964	NA	167	7	NA	0.786	0.908	0.071	5
OVA_Endometrium	Random Forest	RandomForestSMOTE	4	0.953	NA	283	12	NA	0.773	0.924	0.100	4
OVA_Endometrium	Random Forest	RandomForestSMOTE	5	0.960	NA	816	17	NA	0.721	0.915	0.074	4
OVA_Omentum	Random Forest	RandomForestSMOTE	1	0.940	NA	112	19	NA	0.775	0.931	0.090	5
OVA_Omentum	Random Forest	RandomForestSMOTE	2	0.922	NA	286	13	NA	0.507	0.741	0.078	7
OVA_Omentum	Random Forest	RandomForestSMOTE	3	0.952	NA	905	8	NA	0.708	0.822	0.063	4
OVA_Omentum	Random Forest	RandomForestSMOTE	4	0.949	NA	158	3	NA	0.626	0.964	0.081	7
OVA_Omentum	Random Forest	RandomForestSMOTE	5	0.948	NA	170	10	NA	0.675	0.870	0.098	8
OVA_Prostate	Random Forest	RandomForestSMOTE	1	0.999	NA	187	11	NA	0.780	0.944	0.066	5
OVA_Prostate	Random Forest	RandomForestSMOTE	2	0.999	NA	122	9	NA	0.612	0.997	0.051	7
OVA_Prostate	Random Forest	RandomForestSMOTE	3	0.999	NA	106	17	NA	0.587	0.825	0.073	8
OVA_Prostate	Random Forest	RandomForestSMOTE	4	0.999	NA	392	13	NA	0.678	0.808	0.057	10
OVA_Prostate	Random Forest	RandomForestSMOTE	5	1.000	NA	226	6	NA	0.664	0.931	0.080	8
OVA_Endometrium	Random Forest	RandomOverSampler	1	0.963	NA	113	17	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	RandomOverSampler	2	0.956	NA	752	16	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	RandomOverSampler	3	0.965	NA	263	14	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	RandomOverSampler	4	0.956	NA	854	9	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	RandomOverSampler	5	0.965	NA	242	10	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	RandomOverSampler	1	0.941	NA	205	9	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	RandomOverSampler	2	0.924	NA	341	14	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	RandomOverSampler	3	0.949	NA	170	11	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	RandomOverSampler	4	0.945	NA	399	15	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	RandomOverSampler	5	0.942	NA	163	17	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	RandomOverSampler	1	0.999	NA	419	12	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	RandomOverSampler	2	0.998	NA	659	17	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	RandomOverSampler	3	0.998	NA	961	20	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	RandomOverSampler	4	0.999	NA	232	19	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	RandomOverSampler	5	1.000	NA	212	7	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SMOTE	1	0.959	2	248	19	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SMOTE	2	0.954	9	239	13	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SMOTE	3	0.961	6	343	14	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SMOTE	4	0.954	9	480	13	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SMOTE	5	0.960	9	548	16	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	SMOTE	1	0.934	3	261	20	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	SMOTE	2	0.925	5	101	19	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	SMOTE	3	0.951	4	175	19	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	SMOTE	4	0.944	9	286	10	NA	NA	NA	NA	NA
OVA_Omentum	Random Forest	SMOTE	5	0.945	7	525	15	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	SMOTE	1	0.999	8	203	9	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	SMOTE	2	0.998	7	374	9	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	SMOTE	3	0.998	5	365	18	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	SMOTE	4	0.999	4	589	18	NA	NA	NA	NA	NA
OVA_Prostate	Random Forest	SMOTE	5	1.000	7	137	2	NA	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SVMSMOTE	1	0.958	9	463	15	13	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SVMSMOTE	2	0.955	10	481	19	9	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SVMSMOTE	3	0.960	2	659	15	8	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SVMSMOTE	4	0.953	8	393	15	7	NA	NA	NA	NA
OVA_Endometrium	Random Forest	SVMSMOTE	5	0.961	10	828	10	11	NA	NA	NA	NA
OVA_Omentum	Random Forest	SVMSMOTE	1	0.932	5	516	17	16	NA	NA	NA	NA
OVA_Omentum	Random Forest	SVMSMOTE	2	0.923	7	314	19	14	NA	NA	NA	NA
OVA_Omentum	Random Forest	SVMSMOTE	3	0.950	9	123	19	19	NA	NA	NA	NA
OVA_Omentum	Random Forest	SVMSMOTE	4	0.947	9	439	14	6	NA	NA	NA	NA
OVA_Omentum	Random Forest	SVMSMOTE	5	0.942	7	142	20	9	NA	NA	NA	NA
OVA_Prostate	Random Forest	SVMSMOTE	1	0.999	2	248	3	9	NA	NA	NA	NA
OVA_Prostate	Random Forest	SVMSMOTE	2	0.999	3	112	9	11	NA	NA	NA	NA
OVA_Prostate	Random Forest	SVMSMOTE	3	0.998	9	789	5	9	NA	NA	NA	NA
OVA_Prostate	Random Forest	SVMSMOTE	4	0.998	9	146	6	7	NA	NA	NA	NA
OVA_Prostate	Random Forest	SVMSMOTE	5	1.000	10	385	2	14	NA	NA	NA	NA

Table B.12: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for SVM (Linear) classifier, grouped by sample generation method.

dataset	algorithm	oversampling	fold	AUC	smote_k	C	smote_m	min_leaf_purity	max_leaf_purity	top_pair_fraction	os_max_depth
OVA_Endometrium	SVM (Linear)	ADASYN	1	0.967	6	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	ADASYN	2	0.958	3	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	ADASYN	3	0.975	2	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	ADASYN	4	0.966	8	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	ADASYN	5	0.957	3	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	ADASYN	1	0.911	4	0.004	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	ADASYN	2	0.857	9	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	ADASYN	3	0.928	10	5.060	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	ADASYN	4	0.932	10	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	ADASYN	5	0.888	9	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	ADASYN	1	1.000	10	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	ADASYN	2	0.998	9	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	ADASYN	3	0.999	9	0.007	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	ADASYN	4	0.995	9	0.001	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	ADASYN	5	1.000	2	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	BorderlineSMOTE	1	0.966	7	0.000	20	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	BorderlineSMOTE	2	0.956	3	0.000	19	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	BorderlineSMOTE	3	0.974	9	0.000	9	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	BorderlineSMOTE	4	0.966	7	0.000	19	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	BorderlineSMOTE	5	0.957	9	0.000	6	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	BorderlineSMOTE	1	0.910	3	0.005	8	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	BorderlineSMOTE	2	0.869	4	0.000	5	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	BorderlineSMOTE	3	0.933	2	0.002	19	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	BorderlineSMOTE	4	0.937	2	0.000	6	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	BorderlineSMOTE	5	0.891	3	0.000	9	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	BorderlineSMOTE	1	1.000	5	0.000	20	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	BorderlineSMOTE	2	0.999	10	0.000	19	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	BorderlineSMOTE	3	0.998	10	0.000	19	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	BorderlineSMOTE	4	0.996	4	0.000	10	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	BorderlineSMOTE	5	1.000	4	0.000	7	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	RandomForestSMOTE	1	0.965	NA	0.000	NA	0.797	1.000	0.063	4
OVA_Endometrium	SVM (Linear)	RandomForestSMOTE	2	0.947	NA	0.000	NA	0.793	0.981	0.080	10
OVA_Endometrium	SVM (Linear)	RandomForestSMOTE	3	0.975	NA	0.000	NA	0.643	0.775	0.055	4
OVA_Endometrium	SVM (Linear)	RandomForestSMOTE	4	0.957	NA	0.000	NA	0.562	0.696	0.081	9
OVA_Endometrium	SVM (Linear)	RandomForestSMOTE	5	0.953	NA	0.000	NA	0.501	0.635	0.059	9
OVA_Omentum	SVM (Linear)	RandomForestSMOTE	1	0.915	NA	0.005	NA	0.784	0.929	0.060	4
OVA_Omentum	SVM (Linear)	RandomForestSMOTE	2	0.881	NA	0.000	NA	0.512	0.886	0.081	5
OVA_Omentum	SVM (Linear)	RandomForestSMOTE	3	0.932	NA	0.002	NA	0.618	0.832	0.097	9
OVA_Omentum	SVM (Linear)	RandomForestSMOTE	4	0.940	NA	0.000	NA	0.669	0.920	0.068	6
OVA_Omentum	SVM (Linear)	RandomForestSMOTE	5	0.894	NA	0.000	NA	0.719	0.837	0.054	5
OVA_Prostate	SVM (Linear)	RandomForestSMOTE	1	1.000	NA	0.000	NA	0.657	0.849	0.079	4
OVA_Prostate	SVM (Linear)	RandomForestSMOTE	2	0.999	NA	0.000	NA	0.583	0.964	0.099	9
OVA_Prostate	SVM (Linear)	RandomForestSMOTE	3	0.999	NA	23.547	NA	0.725	0.888	0.073	4
OVA_Prostate	SVM (Linear)	RandomForestSMOTE	4	0.997	NA	0.000	NA	0.693	0.906	0.059	10
OVA_Prostate	SVM (Linear)	RandomForestSMOTE	5	1.000	NA	0.000	NA	0.642	0.828	0.091	5
OVA_Endometrium	SVM (Linear)	RandomOverSampler	1	0.967	NA	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	RandomOverSampler	2	0.958	NA	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	RandomOverSampler	3	0.974	NA	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	RandomOverSampler	4	0.967	NA	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	RandomOverSampler	5	0.956	NA	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	RandomOverSampler	1	0.911	NA	0.005	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	RandomOverSampler	2	0.857	NA	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	RandomOverSampler	3	0.928	NA	0.007	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	RandomOverSampler	4	0.932	NA	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	RandomOverSampler	5	0.889	NA	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	RandomOverSampler	1	1.000	NA	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	RandomOverSampler	2	0.999	NA	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	RandomOverSampler	3	0.999	NA	0.058	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	RandomOverSampler	4	0.996	NA	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	RandomOverSampler	5	1.000	NA	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SMOTE	1	0.966	2	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SMOTE	2	0.958	10	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SMOTE	3	0.975	2	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SMOTE	4	0.966	10	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SMOTE	5	0.956	2	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SMOTE	1	0.911	4	0.005	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SMOTE	2	0.858	9	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SMOTE	3	0.928	3	0.006	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SMOTE	4	0.932	10	0.000	NA	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SMOTE	5	0.888	6	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SMOTE	1	1.000	6	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SMOTE	2	0.998	8	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SMOTE	3	0.999	7	0.014	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SMOTE	4	0.996	4	0.000	NA	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SMOTE	5	1.000	2	0.000	NA	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SVMSMOTE	1	0.967	7	0.000	5	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SVMSMOTE	2	0.956	3	0.000	19	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SVMSMOTE	3	0.974	3	0.000	5	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SVMSMOTE	4	0.967	9	0.000	6	NA	NA	NA	NA
OVA_Endometrium	SVM (Linear)	SVMSMOTE	5	0.958	4	0.000	14	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SVMSMOTE	1	0.913	7	0.000	18	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SVMSMOTE	2	0.874	3	0.000	7	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SVMSMOTE	3	0.932	9	0.002	9	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SVMSMOTE	4	0.936	5	0.000	10	NA	NA	NA	NA
OVA_Omentum	SVM (Linear)	SVMSMOTE	5	0.892	4	0.000	16	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SVMSMOTE	1	1.000	2	0.000	16	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SVMSMOTE	2	0.999	2	0.000	10	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SVMSMOTE	3	0.999	10	0.001	15	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SVMSMOTE	4	0.998	4	0.000	20	NA	NA	NA	NA
OVA_Prostate	SVM (Linear)	SVMSMOTE	5	1.000	2	0.000	13	NA	NA	NA	NA

Table B.13: Best Hyperparameter settings and mean AUC (averaged over 5-fold inner cross-validation) for each of the 5 outer cross-validation folds for XGBoost classifier, grouped by sample generation method.

dataset	algorithm	oversampling	fold	AUC	smote_k	learning_rate	max_depth	n_estimators	smote_m	min_leaf_purity	max_leaf_purity	top_pair_fraction	os_max_depth
OVA_Endometrium	XGBoost	ADASYN	1	0.964	4	0.114	3	706	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	ADASYN	2	0.967	10	0.174	3	914	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	ADASYN	3	0.978	2	0.078	10	989	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	ADASYN	4	0.965	5	0.299	3	501	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	ADASYN	5	0.973	6	0.275	8	426	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	ADASYN	1	0.957	8	0.300	3	836	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	ADASYN	2	0.930	10	0.250	6	836	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	ADASYN	3	0.943	10	0.300	4	396	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	ADASYN	4	0.925	10	0.275	3	938	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	ADASYN	5	0.967	6	0.232	6	381	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	ADASYN	1	0.990	10	0.086	9	153	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	ADASYN	2	0.990	4	0.185	7	728	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	ADASYN	3	0.997	7	0.273	3	123	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	ADASYN	4	0.998	3	0.021	6	125	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	ADASYN	5	0.997	2	0.298	3	691	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	BorderlineSMOTE	1	0.969	4	0.127	3	983	14	NA	NA	NA	NA
OVA_Endometrium	XGBoost	BorderlineSMOTE	2	0.968	5	0.199	3	244	20	NA	NA	NA	NA
OVA_Endometrium	XGBoost	BorderlineSMOTE	3	0.978	7	0.127	6	145	11	NA	NA	NA	NA
OVA_Endometrium	XGBoost	BorderlineSMOTE	4	0.964	9	0.235	3	567	9	NA	NA	NA	NA
OVA_Endometrium	XGBoost	BorderlineSMOTE	5	0.975	6	0.241	3	650	20	NA	NA	NA	NA
OVA_Omentum	XGBoost	BorderlineSMOTE	1	0.953	5	0.299	4	224	16	NA	NA	NA	NA
OVA_Omentum	XGBoost	BorderlineSMOTE	2	0.930	7	0.096	9	154	5	NA	NA	NA	NA
OVA_Omentum	XGBoost	BorderlineSMOTE	3	0.941	2	0.293	5	852	16	NA	NA	NA	NA
OVA_Omentum	XGBoost	BorderlineSMOTE	4	0.924	9	0.045	3	252	15	NA	NA	NA	NA
OVA_Omentum	XGBoost	BorderlineSMOTE	5	0.965	4	0.299	6	788	11	NA	NA	NA	NA
OVA_Prostate	XGBoost	BorderlineSMOTE	1	0.999	4	0.035	10	313	7	NA	NA	NA	NA
OVA_Prostate	XGBoost	BorderlineSMOTE	2	0.997	7	0.010	6	783	14	NA	NA	NA	NA
OVA_Prostate	XGBoost	BorderlineSMOTE	3	0.995	6	0.299	3	882	5	NA	NA	NA	NA
OVA_Prostate	XGBoost	BorderlineSMOTE	4	0.997	4	0.285	4	451	13	NA	NA	NA	NA
OVA_Prostate	XGBoost	BorderlineSMOTE	5	0.997	3	0.227	3	991	6	NA	NA	NA	NA
OVA_Endometrium	XGBoost	RandomForestSMOTE	1	0.960	NA	0.087	7	608	NA	0.562	0.665	0.092	8
OVA_Endometrium	XGBoost	RandomForestSMOTE	2	0.977	NA	0.156	3	723	NA	0.700	0.896	0.067	4
OVA_Endometrium	XGBoost	RandomForestSMOTE	3	0.961	NA	0.221	3	185	NA	0.647	0.816	0.070	8
OVA_Endometrium	XGBoost	RandomForestSMOTE	4	0.966	NA	0.273	10	316	NA	0.773	0.960	0.082	4
OVA_Endometrium	XGBoost	RandomForestSMOTE	5	0.976	NA	0.097	3	169	NA	0.602	0.836	0.075	5
OVA_Omentum	XGBoost	RandomForestSMOTE	1	0.945	NA	0.178	4	222	NA	0.556	0.672	0.070	4
OVA_Omentum	XGBoost	RandomForestSMOTE	2	0.923	NA	0.156	9	303	NA	0.529	0.945	0.058	4
OVA_Omentum	XGBoost	RandomForestSMOTE	3	0.963	NA	0.051	4	632	NA	0.784	0.887	0.082	10
OVA_Omentum	XGBoost	RandomForestSMOTE	4	0.948	NA	0.254	7	325	NA	0.783	0.888	0.082	6
OVA_Omentum	XGBoost	RandomForestSMOTE	5	0.935	NA	0.256	5	728	NA	0.725	0.889	0.053	9
OVA_Prostate	XGBoost	RandomForestSMOTE	1	0.997	NA	0.041	10	663	NA	0.645	0.932	0.057	8
OVA_Prostate	XGBoost	RandomForestSMOTE	2	0.990	NA	0.298	3	529	NA	0.524	0.782	0.080	4
OVA_Prostate	XGBoost	RandomForestSMOTE	3	0.990	NA	0.282	5	140	NA	0.652	0.962	0.082	6
OVA_Prostate	XGBoost	RandomForestSMOTE	4	0.997	NA	0.297	8	649	NA	0.742	0.991	0.057	7
OVA_Prostate	XGBoost	RandomForestSMOTE	5	0.997	NA	0.211	5	521	NA	0.774	0.980	0.064	9
OVA_Endometrium	XGBoost	RandomOverSampler	1	0.968	NA	0.102	3	137	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	RandomOverSampler	2	0.953	NA	0.237	5	685	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	RandomOverSampler	3	0.977	NA	0.178	4	995	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	RandomOverSampler	4	0.967	NA	0.234	3	211	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	RandomOverSampler	5	0.979	NA	0.236	10	344	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	RandomOverSampler	1	0.947	NA	0.299	5	932	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	RandomOverSampler	2	0.925	NA	0.102	6	780	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	RandomOverSampler	3	0.952	NA	0.239	6	472	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	RandomOverSampler	4	0.914	NA	0.216	8	853	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	RandomOverSampler	5	0.963	NA	0.270	10	603	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	RandomOverSampler	1	0.990	NA	0.244	5	311	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	RandomOverSampler	2	0.990	NA	0.235	7	215	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	RandomOverSampler	3	0.996	NA	0.023	3	243	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	RandomOverSampler	4	0.997	NA	0.012	7	400	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	RandomOverSampler	5	0.997	NA	0.124	6	601	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SMOTE	1	0.964	2	0.288	10	917	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SMOTE	2	0.969	6	0.277	3	991	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SMOTE	3	0.978	3	0.276	9	299	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SMOTE	4	0.961	9	0.224	10	544	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SMOTE	5	0.975	7	0.112	3	528	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	SMOTE	1	0.951	5	0.102	3	426	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	SMOTE	2	0.935	8	0.230	10	825	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	SMOTE	3	0.942	7	0.158	3	161	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	SMOTE	4	0.922	10	0.074	3	136	NA	NA	NA	NA	NA
OVA_Omentum	XGBoost	SMOTE	5	0.968	4	0.249	7	669	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	SMOTE	1	0.990	2	0.130	3	112	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	SMOTE	2	0.990	4	0.248	10	106	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	SMOTE	3	0.996	3	0.255	3	664	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	SMOTE	4	0.996	7	0.232	5	755	NA	NA	NA	NA	NA
OVA_Prostate	XGBoost	SMOTE	5	0.996	4	0.187	7	332	NA	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SVMSMOTE	1	0.969	3	0.142	3	714	11	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SVMSMOTE	2	0.970	7	0.096	5	476	11	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SVMSMOTE	3	0.979	6	0.054	3	139	6	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SVMSMOTE	4	0.970	9	0.143	3	384	10	NA	NA	NA	NA
OVA_Endometrium	XGBoost	SVMSMOTE	5	0.975	10	0.262	7	753	8	NA	NA	NA	NA
OVA_Omentum	XGBoost	SVMSMOTE	1	0.953	2	0.165	3	805	14	NA	NA	NA	NA
OVA_Omentum	XGBoost	SVMSMOTE	2	0.932	2	0.082	7	110	20	NA	NA	NA	NA
OVA_Omentum	XGBoost	SVMSMOTE	3	0.944	2	0.269	7	495	19	NA	NA	NA	NA
OVA_Omentum	XGBoost	SVMSMOTE	4	0.924	10	0.057	3	281	13	NA	NA	NA	NA
OVA_Omentum	XGBoost	SVMSMOTE	5	0.966	3	0.236	5	446	18	NA	NA	NA	NA
OVA_Prostate	XGBoost	SVMSMOTE	1	0.999	6	0.169	6	376	6	NA	NA	NA	NA
OVA_Prostate	XGBoost	SVMSMOTE	2	0.990	2	0.227	6	118	9	NA	NA	NA	NA
OVA_Prostate	XGBoost	SVMSMOTE	3	0.996	5	0.016	3	343	6	NA	NA	NA	NA
OVA_Prostate	XGBoost	SVMSMOTE	4	0.996	3	0.158	5	170	6	NA	NA	NA	NA
OVA_Prostate	XGBoost	SVMSMOTE	5	1.000	8	0.141	7	390	18	NA	NA	NA	NA

Table B.14: Average Hyperparameter Importance by classifier and sample generation method averaged over all datasets and folds of inner cross-validation

model	samplegen	C	smote_k	smote_m	min_leaf_purity	os_max_depth	top_pair_fraction	var_smoothing	max_depth	n_estimators	learning_rate	n_neighbors	weights
Logistic Regression													
Logistic Regression	ADASYN	0.814	0.186	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
Logistic Regression	BorderlineSMOTE	0.718	0.102	0.180	NA	NA	NA	NA	NA	NA	NA	NA	NA
Logistic Regression	RandomForestSMOTE	0.528	NA	NA	0.204	0.118	0.150	NA	NA	NA	NA	NA	NA
Logistic Regression	RandomOverSampler	1.000	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
Logistic Regression	SMOTE	0.865	0.135	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
Logistic Regression	SVMSMOTE	0.657	0.134	0.209	NA	NA	NA	NA	NA	NA	NA	NA	NA
Naive Bayes													
Naive Bayes	ADASYN	NA	0.956	NA	NA	NA	NA	0.044	NA	NA	NA	NA	NA
Naive Bayes	BorderlineSMOTE	NA	0.340	0.630	NA	NA	NA	0.030	NA	NA	NA	NA	NA
Naive Bayes	RandomForestSMOTE	NA	NA	NA	0.313	0.469	0.166	0.052	NA	NA	NA	NA	NA
Naive Bayes	RandomOverSampler	NA	NA	NA	NA	NA	NA	1.000	NA	NA	NA	NA	NA
Naive Bayes	SMOTE	NA	0.884	NA	NA	NA	NA	0.116	NA	NA	NA	NA	NA
Naive Bayes	SVMSMOTE	NA	0.116	0.817	NA	NA	NA	0.067	NA	NA	NA	NA	NA
Random Forest													
Random Forest	ADASYN	NA	0.066	NA	NA	NA	NA	NA	0.529	0.405	NA	NA	NA
Random Forest	BorderlineSMOTE	NA	0.062	0.081	NA	NA	NA	NA	0.504	0.352	NA	NA	NA
Random Forest	RandomForestSMOTE	NA	NA	NA	0.143	0.188	0.087	NA	0.339	0.242	NA	NA	NA
Random Forest	RandomOverSampler	NA	NA	NA	NA	NA	NA	NA	0.481	0.519	NA	NA	NA
Random Forest	SMOTE	NA	0.054	NA	NA	NA	NA	NA	0.531	0.416	NA	NA	NA
Random Forest	SVMSMOTE	NA	0.067	0.110	NA	NA	NA	NA	0.566	0.257	NA	NA	NA
SVM (Linear)													
SVM (Linear)	ADASYN	0.010	0.990	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
SVM (Linear)	BorderlineSMOTE	0.073	0.271	0.656	NA	NA	NA	NA	NA	NA	NA	NA	NA
SVM (Linear)	RandomForestSMOTE	0.123	NA	NA	0.325	0.332	0.221	NA	NA	NA	NA	NA	NA
SVM (Linear)	RandomOverSampler	1.000	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
SVM (Linear)	SMOTE	0.090	0.910	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
SVM (Linear)	SVMSMOTE	0.030	0.320	0.650	NA	NA	NA	NA	NA	NA	NA	NA	NA
XGBoost													
XGBoost	ADASYN	NA	0.448	NA	NA	NA	NA	NA	0.060	0.154	0.338	NA	NA
XGBoost	BorderlineSMOTE	NA	0.169	0.159	NA	NA	NA	NA	0.103	0.139	0.429	NA	NA
XGBoost	RandomForestSMOTE	NA	NA	NA	0.236	0.111	0.111	NA	0.047	0.098	0.397	NA	NA
XGBoost	RandomOverSampler	NA	NA	NA	NA	NA	NA	NA	0.222	0.214	0.564	NA	NA
XGBoost	SMOTE	NA	0.370	NA	NA	NA	NA	NA	0.072	0.134	0.425	NA	NA
XGBoost	SVMSMOTE	NA	0.159	0.298	NA	NA	NA	NA	0.052	0.174	0.317	NA	NA
k-NN													
k-NN	ADASYN	NA	0.160	NA	NA	NA	NA	NA	NA	NA	NA	0.828	0.013
k-NN	BorderlineSMOTE	NA	0.052	0.167	NA	NA	NA	NA	NA	NA	NA	0.759	0.022
k-NN	RandomForestSMOTE	NA	NA	NA	0.036	0.048	0.024	NA	NA	NA	NA	0.887	0.004
k-NN	RandomOverSampler	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	0.999	0.001
k-NN	SMOTE	NA	0.149	NA	NA	NA	NA	NA	NA	NA	NA	0.834	0.017
k-NN	SVMSMOTE	NA	0.052	0.131	NA	NA	NA	NA	NA	NA	NA	0.812	0.005

Bibliography

[AAARA24] AL-AZANI, Sadam ; ALKHNBASHI, Omer S. ; RAMADAN, Emad ; ALFARRAJ, Motaz: Gene Expression-Based Cancer Classification for Handling the Class Imbalance Problem and Curse of Dimensionality. In: *International Journal of Molecular Sciences* 25 (2024), Nr. 4

[AIP24] ASSUNÇÃO, Gabriel O. ; IZBICKI, Rafael ; PRATES, Marcos O.: Is Augmentation Effective in Improving Prediction in Imbalanced Datasets? In: *Journal of Data Science* (2024), S. 1–16

[ASY+19] AKIBA, Takuya ; SANO, Shotaro ; YANASE, Toshihiko ; OHTA, Takeru ; KOYAMA, Masanori: Optuna: A Next-generation Hyperparameter Optimization Framework. In: *CoRR* abs/1907.10902 (2019)

[ASZ20] ABDULRAUF SHARIFAI, Garba ; ZAINOL, Zurinahni: Feature Selection for High-Dimensional and Imbalanced Biomedical Data Based on Robust Correlation Based Redundancy and Binary Grasshopper Optimization Algorithm. In: *Genes* 11 (2020), Nr. 7

[AT21] AL-TURAIKI, Isra: Dimensionality Reduction of RNA-Seq Data. In: *International Journal of Computer Science and Network Security* 21 (2021), Nr. 3, S. 31–36

[BDJ16] BELLINGER, Colin ; DRUMMOND, Christopher ; JAPKOWICZ, Nathalie: Beyond the Boundaries of SMOTE - A Framework for Manifold-Based Synthetically Oversampling. In: *European Conference on Machine Learning and Knowledge Discovery in Databases - Volume 9851*. Berlin, Heidelberg : Springer-Verlag, 2016 (ECML PKDD 2016). – ISBN 9783319461274, S. 248–263

[BDJ18] BELLINGER, Colin ; DRUMMOND, Christopher ; JAPKOWICZ, Nathalie: Manifold-based synthetic oversampling with manifold conformance estimation. In: *Mach. Learn.* 107 (2018), März, Nr. 3, S. 605–637

[BDW+21] BEJ, Saptarshi ; DAVTYAN, Narek ; WOLFIEN, Markus ; NASSAR, Mariam ; WOLKENHAUER, Olaf: LoRAS: an oversampling approach for imbalanced datasets. In: *Mach. Learn.* 110 (2021), Februar, Nr. 2, S. 279–301

[BL10] BLAGUS, Rok ; LUSA, Lara: Class prediction for high-dimensional class-imbalanced data. In: *BMC bioinformatics* 11 (2010), 10, S. 523

[BL13] BLAGUS, Rok ; LUSA, Lara: SMOTE for high-dimensional class-imbalanced data. In: *BMC Bioinformatics* 14 (2013), S. 106 – 106

[BMWX21] BAI, Yu ; MEI, Song ; WANG, Huan ; XIONG, Caiming: Don't Just Blame Over-parametrization for Over-confidence: Theoretical Analysis of Calibration in Binary Classification. In: MEILA, Marina (Hrsg.) ; ZHANG, Tong (Hrsg.): *Proceedings of the 38th International Conference on Machine Learning* Bd. 139, PMLR, 18–24 Jul 2021 (Proceedings of Machine Learning Research), S. 566–576

[Bos08] BOSTRÖM, Henrik: Calibrating Random Forests. In: *2008 Seventh International Conference on Machine Learning and Applications*, 2008, S. 121–126

[Bra25] BRAVO, João: NRGBoost: Energy-Based Generative Boosted Trees. In: *The Thirteenth International Conference on Learning Representations*, 2025

[Bre01] BREIMAN, Leo: Random Forests. In: *Machine Learning* 45 (2001), Nr. 1, S. 5–32

[Bri50] BRIER, Glenn W.: Verification of Forecasts Expressed in Terms of Probability. In: *Monthly Weather Review* 78 (1950), Januar, Nr. 1, S. 1

[BTW+07] BARRETT, Tanya ; TROUP, Dennis B. ; WILHITE, Stephen E. ; LEDOUX, Philippe ; RUDNEV, Dmitry ; EVANGELISTA, Cynthia ; KIM, In-Hee ; SOBOLEVA, Alexandra ; TOMASHEVSKY, Michael ; EDGAR, Ron: NCBI GEO: mining tens of millions of expression profiles—database and tools update. In: *Nucleic Acids Research* 35 (2007), Nr. suppl_1, S. D760–D765

[CBHK02] CHAWLA, Nitesh V. ; BOWYER, Kevin W. ; HALL, Lawrence O. ; KEGELMEYER, W. P.: SMOTE: Synthetic Minority Over-sampling Technique. In: *Journal of Artificial Intelligence Research* 16 (2002), S. 321–357

[CG16] CHEN, Tianqi ; GUESTRIN, Carlos: XGBoost: A Scalable Tree Boosting System. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA : ACM, 2016 (KDD '16), S. 785–794

[CH67] COVER, T. ; HART, P.: Nearest neighbor pattern classification. In: *IEEE Transactions on Information Theory* 13 (1967), Nr. 1, S. 21–27

[CLH+25] CARRIERO, Alex ; LUIJKEN, Kim ; HOND, Anne de ; MOONS, Karel G. M. ; CALSTER, Ben van ; SMEDEN, Maarten van: The Harms of Class Imbalance Corrections for Machine Learning Based Prediction Models: A Simulation Study. In: *Statistics in Medicine* 44 (2025), Nr. 3-4, S. e10320

[Coh60] COHEN, Jacob: A Coefficient of Agreement for Nominal Scales. In: *Educational and Psychological Measurement* 20 (1960), Nr. 1, S. 37–46. http://dx.doi.org/10.1177/001316446002000104. – DOI 10.1177/001316446002000104

[Cox58] COX, D. R.: The Regression Analysis of Binary Sequences. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 20 (1958), Nr. 2, S. 215–232

[CPC20] CORREIA, Alvaro H. C. ; PEHARZ, Robert ; CAMPOS, Cassio P.: Joints in Random Forests. In: LAROCHELLE, Hugo (Hrsg.) ; RANZATO, Marc'Aurelio (Hrsg.) ; HADSELL, Raia (Hrsg.) ; BALCAN, Maria-Florina

(Hrsg.) ; LIN, Hsuan-Tien (Hrsg.): *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, Curran Associates, 2020, S. 11404–11415

[CV95] CORTES, Corinna ; VAPNIK, Vladimir: Support-Vector Networks. In: *Mach. Learn.* 20 (1995), September, Nr. 3, S. 273–297

[CYY+24] CHEN, Wuxing ; YANG, Kaixiang ; YU, Zhiwen ; SHI, Yifan ; CHEN, C. L. P.: A survey on imbalanced learning: latest research, applications and future directions. In: *Artificial Intelligence Review* 57 (2024), Nr. 6

[EA19] ELREEDY, Dina ; ATIYA, Amir F.: A Comprehensive Analysis of Synthetic Minority Oversampling Technique (SMOTE) for handling class imbalance. In: *Inf. Sci.* 505 (2019), Dezember, Nr. C, S. 32–64

[EAE22] ELOR, Yotam ; AVERBUCH-ELOR, Hadar: *To SMOTE, or not to SMOTE?* arXiv preprint arXiv:2201.08528. https://arxiv.org/abs/2201.08528. Version: 2022. – revised version (v3) published 11 May 2022

[EY36] ECKART, Carl ; YOUNG, G. M.: The approximation of one matrix by another of lower rank. In: *Psychometrika* 1 (1936), S. 211–218

[Fak12] FAKULTETA ZA ZDRAVSTVENE ŠTUDIJE, UNIVERZA V MARIBORU: *About GEMLER*. https://web.archive.org/web/20120125200250/http://gemler.fzv.uni-mb.si/about.php, Jan 2012. – Archived on January 25, 2012

[FH51] FIX, E. ; HODGES, J.L.: *Discriminatory Analysis: Nonparametric Discrimination: Consistency Properties*. USAF School of Aviation Medicine, 1951

[FHOF11] FLACH, Peter ; HERNÁNDEZ-ORALLO, José ; FERRI, Cèsar: A coherent interpretation of AUC as a measure of aggregated classification performance. In: *Proceedings of the 28th International Conference on International Conference on Machine Learning*. Madison, WI, USA : Omnipress, 2011 (ICML'11). – ISBN 9781450306195, S. 657–664

[Fri00] FRIEDMAN, Jerome: Greedy Function Approximation: A Gradient Boosting Machine. In: *The Annals of Statistics* 29 (2000), 11

[FSS+25] FLORES, Gerardo ; SCHIFF, Abigail ; SMITH, Alyssa H. ; FUKUYAMA, Julia A. ; WILSON, Ashia C.: A Consequentialist Critique of Binary Classification Evaluation Practices. In: *arXiv preprint arXiv:2504.04528* (2025), Apr. https://arxiv.org/abs/2504.04528

[GLH11] GU, Quanquan ; LI, Zhenhui ; HAN, Jiawei: Generalized Fisher score for feature selection. In: *Proceedings of the Twenty-Seventh Conference on Uncertainty in Artificial Intelligence*. Arlington, Virginia, USA : AUAI Press, 2011 (UAI'11). – ISBN 9780974903972, S. 266–273

[GPAM+20] GOODFELLOW, Ian ; POUGET-ABADIE, Jean ; MIRZA, Mehdi ; XU, Bing ; WARDE-FARLEY, David ; OZAIR, Sherjil ; COURVILLE, Aaron ; BENGIO, Yoshua: Generative adversarial networks. In: *Commun. ACM* 63 (2020), Oktober, Nr. 11, S. 139–144

[GPSW17] GUO, Chuan ; PLEISS, Geoff ; SUN, Yu ; WEINBERGER, Kilian Q.: On Calibration of Modern Neural Networks. In: PRECUP, Doina (Hrsg.) ; TEH, Yee W. (Hrsg.): *Proceedings of the 34th International Conference on Machine Learning* Bd. 70, PMLR, Aug 2017 (Proceedings of Machine Learning Research), S. 1321–1330. – Available at PMLR

[GXZ+25] GAO, Xinyi ; XIE, Dongting ; ZHANG, Yihang ; WANG, Zhengren ; HE, Conghui ; YIN, Hongzhi ; ZHANG, Wentao: *A Comprehensive Survey on Imbalanced Data Learning*. 2025

[HA13] HAND, D.J. ; ANAGNOSTOPOULOS, C.: When is the area under the receiver operating characteristic curve an appropriate measure of classifier performance? In: *Pattern Recognition Letters* 34 (2013), Nr. 5, S. 492–495. – ISSN 0167–8655

[Han09] HAND, David J.: Measuring classifier performance: a coherent alternative to the area under the ROC curve. In: *Machine Learning* 77 (2009), Nr. 1, S. 103–123

[HG09] HE, Haibo ; GARCIA, Edwardo A.: Learning from Imbalanced Data. In: *IEEE Transactions on Knowledge and Data Engineering* 21 (2009), September, Nr. 9, S. 1263–1284

[HHLB14] HUTTER, Frank ; HOOS, Holger ; LEYTON-BROWN, Kevin: An Efficient Approach for Assessing Hyperparameter Importance. In: XING, Eric P. (Hrsg.) ; JEBARA, Tony (Hrsg.): *Proceedings of the 31st International Conference on Machine Learning* Bd. 32. Beijing, China : PMLR, 22–24 Jun 2014 (Proceedings of Machine Learning Research 1), S. 754–762

[HO00] HYVÄRINEN, A ; OJA, E: Independent component analysis: algorithms and applications. In: *Neural Networks: The Official Journal of the International Neural Network Society* 13 (2000), Juni, Nr. 4-5, S. 411–430. – ISSN 0893–6080. – PMID: 10946390

[Hot33] HOTELLING, Harold: Analysis of a complex of statistical variables into principal components. In: *Journal of Educational Psychology* 24 (1933), S. 498–520

[HTF09] HASTIE, Trevor ; TIBSHIRANI, Robert ; FRIEDMAN, Jerome: *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. New York : Springer, 2009

[HWM05] HAN, Hui ; WANG, WenYuan ; MAO, BingHuan: Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning. In: HUANG, De-Shuang (Hrsg.) ; ZHANG, Xiao-Ping (Hrsg.) ; HUANG, Guang-Bin (Hrsg.): *Advances in Intelligent Computing (ICIC 2005), Lecture Notes in Computer Science, Vol. 3644*. Hefei, China, August 23–26, 2005 : Springer, Berlin, Heidelberg, 2005, S. 878–887

[Int05] INTERNATIONAL GENOMICS CONSORTIUM: *Expression Project for Oncology (expO)*. https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE2109, 2005. – GEO Series accession: GSE2109

[JK19] JOHNSON, Justin M. ; KHOSHGOFTAAR, Taghi M.: Survey on deep learning with class imbalance. In: *Journal of Big Data* 6 (2019), März, Nr. 1, S. 27

[JS02] JAPKOWICZ, Nathalie ; STEPHEN, Shaju: The Class Imbalance Problem: A Systematic Study. In: *Intelligent Data Analysis* 6 (2002), Nr. 5, S. 429–449

[JTM25] JAFARIGOL, Elaheh ; TRAFALIS, Theodore ; MOHAMMADI, Neshat: *A Review of Machine Learning Techniques in Imbalanced Data and Future Trends*. https://arxiv.org/abs/2310.07917. Version: 2025

[KCL23] KABIR, Md F. ; CHEN, Tianjie ; LUDWIG, Simone A.: A performance analysis of dimensionality reduction algorithms in machine learning models for cancer prediction. In: *Healthcare Analytics* 3 (2023), S. 100125

[KLC23] KAMALOV, Firuz ; LEUNG, Ho-Hon ; CHERUKURI, Aswani K.: Keep it simple: random oversampling for imbalanced data. In: *2023 Advances in Science and Engineering Technology International Conferences (ASET)*, 2023, S. 1–4

[KPM19] KAUR, Harsurinder ; PANNU, Husanbir S. ; MALHI, Avleen K.: A Systematic Review on Imbalanced Data Challenges in Machine Learning: Applications and Solutions. In: *ACM Comput. Surv.* 52 (2019), August, Nr. 4

[KR92] KIRA, Kenji ; RENDELL, Larry A.: The feature selection problem: traditional methods and a new algorithm. In: *Proceedings of the Tenth National Conference on Artificial Intelligence*, AAAI Press, 1992 (AAAI'92). – ISBN 0262510634, S. 129–134

[Kra16] KRAWCZYK, Bartosz: Learning from imbalanced data: open challenges and future directions. In: *Progress in Artificial Intelligence* 5 (2016), April, Nr. 4, S. 221–232

[KW13] KINGMA, Diederik P. ; WELLING, Max: Auto-Encoding Variational Bayes. In: *CoRR* abs/1312.6114 (2013)

[LNA17] LEMAÎTRE, Guillaume ; NOGUEIRA, Fernando ; ARIDAS, Christos K.: Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. In: *Journal of Machine Learning Research* 18 (2017), Nr. 17, 1-5. http://jmlr.org/papers/v18/16-365

[Mat75] MATTHEWS, Brian W.: Comparison of the predicted and observed secondary structure of T4 phage lysozyme. In: *Biochimica et Biophysica Acta (BBA)-Protein Structure* 405 (1975), Nr. 2, S. 442–451

[MH08] MAATEN, Laurens van d. ; HINTON, Geoffrey: Visualizing data using t-SNE. In: *Journal of machine learning research* 9 (2008), Nr. Nov, S. 2579–2605

[MLV19] MALDONADO, Sebastián ; LÓPEZ, Julio ; VAIRETTI, Carla: An alternative SMOTE oversampling strategy for high-dimensional datasets. In: *Applied Soft Computing* 76 (2019), S. 380–389

[MP11] MARTIN, Carla D. M. ; PORTER, Mason A.: The Extraordinary SVD. In: *The American Mathematical Monthly* 119 (2011), S. 838 – 851

[Mur12] MURPHY, Kevin P.: *Machine Learning: A Probabilistic Perspective*. The MIT Press, 2012

[MVFH22] MALDONADO, Sebastián ; VAIRETTI, Carla ; FERNANDEZ, Alberto ; HERRERA, Francisco: FW-SMOTE: A feature-weighted oversampling approach for imbalanced classification. In: *Pattern Recognition* 124 (2022), 108511. http://dx.doi.org/https://doi.org/10.1016/j.patcog.2021.108511. – DOI https://doi.org/10.1016/j.patcog.2021.108511. – ISSN 0031–3203

[MZH+24] MCDERMOTT, Matthew B. ; ZHANG, Haoran ; HANSEN, Lasse H. ; ANGELOTTI, Giovanni ; GALLIFANT, Jack: A Closer Look at AUROC and AUPRC under Class Imbalance. In: GLOBERSON, A. (Hrsg.) ; MACKEY, L. (Hrsg.) ; BELGRAVE, D. (Hrsg.) ; FAN, A. (Hrsg.) ; PAQUET, U. (Hrsg.) ; TOMCZAK, J. (Hrsg.) ; ZHANG, C. (Hrsg.): *Advances in Neural Information Processing Systems* Bd. 37, Curran Associates, Inc., 2024, S. 44102–44163

[NCK09] NGUYEN, Hien M. ; COOPER, Eric W. ; KAMEI, Katsuari: Borderline over-sampling for imbalanced data classification. In: *International Journal of Knowledge Engineering and Soft Data Paradigms* 3 (2009), Nr. 1, S. 4–21

[NGB24] NOCK, Richard ; GUILLAME-BERT, Mathieu: Generative Forests. In: GLOBERSON, A. (Hrsg.) ; MACKEY, L. (Hrsg.) ; BELGRAVE, D. (Hrsg.) ; FAN, A. (Hrsg.) ; PAQUET, U. (Hrsg.) ; TOMCZAK, J. (Hrsg.) ; ZHANG, C. (Hrsg.): *Advances in Neural Information Processing Systems* Bd. 37, Curran Associates, Inc., 2024, S. 27919–27977

[NMC05] NICULESCU-MIZIL, Alexandru ; CARUANA, Rich: Predicting Good Probabilities with Supervised Learning. In: *Proceedings of the 22nd International Conference on Machine Learning (ICML)* Bd. 119, ACM, 2005 (ACM International Conference Proceeding Series), S. 625–632

[OLBF24] OSHTERNIAN, S. ; LOIPFINGER, S. ; BHATTACHARYA, Arkajyoti ; FEHRMANN, Rudolf: Exploring combinations of dimensionality reduction, transfer learning, and regularization methods for predicting binary phenotypes with transcriptomic data. In: *BMC Bioinformatics* 25 (2024), 04

[Ope08a] OPENML: *OVA Endometrium dataset*. https://www.openml.org/d/1142, 2008. – Accessed: 2025-07-15

[Ope08b] OPENML: *OVA Omentum dataset*. https://www.openml.org/d/1139, 2008. – Accessed: 2025-07-15

[Ope08c] OPENML: *OVA Prostate dataset*. https://www.openml.org/d/1146, 2008. – Accessed: 2025-07-15

[PF99] PROVOST, Foster ; FAWCETT, Tom: Analysis and Visualization of Classifier Performance: Comparison Under Imprecise Class and Cost Distributions. In: HECKERMAN, David (Hrsg.) ; MANNILA, Heikki (Hrsg.) ; PREGIBON, D. (Hrsg.) ; UTHURUSAMY, Ramasamy (Hrsg.): *Proceedings of*

the Third International Conference on Knowledge Discovery and Data Mining Bd. 43-48. United States : AAAI Press, 12 1999, S. 43 – 48

[Pla99] PLATT, John C.: Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In: *Advances in Large Margin Classifiers*, MIT Press, 1999, S. 61–74

[PRHK14] PADMAJA, T. M. ; RAJU, Bapi S. ; HOTA, Rudra N. ; KRISHNA, P. R.: Class imbalance and its effect on PCA preprocessing. In: *Int. J. Knowl. Eng. Soft Data Paradigm.* 4 (2014), August, Nr. 3, S. 272–294

[PSS+23] PETINRIN, Olutomilayo O. ; SAEED, Faisal ; SALIM, Naomie ; TOSEEF, Muhammad ; LIU, Zhe ; MUYIDE, Ibukun O.: Dimension Reduction and Classifier-Based Feature Selection for Oversampled Gene Expression Data and Cancer Classification. In: *Processes* 11 (2023), Nr. 7

[PVG+11] PEDREGOSA, F. ; VAROQUAUX, G. ; GRAMFORT, A. ; MICHEL, V. ; THIRION, B. ; GRISEL, O. ; BLONDEL, M. ; PRETTENHOFER, P. ; WEISS, R. ; DUBOURG, V. ; VANDERPLAS, J. ; PASSOS, A. ; COURNAPEAU, D. ; BRUCHER, M. ; PERROT, M. ; DUCHESNAY, E.: Scikit-learn: Machine Learning in Python. In: *Journal of Machine Learning Research* 12 (2011), S. 2825–2830

[RG11] RAM, Parikshit ; GRAY, Alexander G.: Density estimation trees. In: *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA : Association for Computing Machinery, 2011 (KDD '11). – ISBN 9781450308137, S. 627–635

[RSK03] ROBNIK-SIKONJA, M. ; KONONENKO, Igor: Theoretical and Empirical Analysis of ReliefF and RReliefF. In: *Machine Learning* 53 (2003), S. 23–69

[RWD88] ROBERTSON, Tim ; WRIGHT, F. T. ; DYKSTRA, R. L.: *Order Restricted Statistical Inference*. Reprint, illustrated. Chichester; New York : John Wiley & Sons, 1988 (Wiley Series in Probability and Mathematical Statistics). – xix + 521 S.

[Sah23] SAHEED, Yakub K.: Chapter 9 - Effective dimensionality reduction model with machine learning classification for microarray gene expression data. In: TYAGI, Amit K. (Hrsg.) ; ABRAHAM, Ajith (Hrsg.): *Data Science for Genomics*. Academic Press, 2023, S. 153–164

[SH05] SU, Chao-Ton ; HSU, Jyh-Hwa: An Extended Chi2 Algorithm for Discretization of Real Value Attributes. In: *IEEE Trans. on Knowl. and Data Eng.* 17 (2005), März, Nr. 3, S. 437–441

[SK10] STIGLIC, Gregor ; KOKOL, Peter: Stability of ranked gene lists in large microarray analysis studies. In: *BioMed Research International* 2010 (2010), Nr. 1, S. 616358

[SLK22] SALEKSHAHREZAEE, Zahra ; LEEVY, Joffrey L. ; KHOSHGOFTAAR, Taghi M.: A Class-Imbalanced Study with Feature Extraction via PCA and Convolutional Autoencoder. In: *2022 IEEE 23rd International Conference on Information Reuse and Integration for Data Science (IRI)*, 2022, S. 63–68

[Spa89] SPACKMAN, Kent A.: Signal Detection Theory: Valuable Tools for Evaluating Inductive Learning. In: SEGRE, Alberto M. (Hrsg.): *Proceedings of the sixth international workshop on Machine learning*, Morgan Kaufmann, 1989, S. 160–163

[TDSCDFCDM19] TAVEIRA DE SOUZA, Jovani ; CARLOS DE FRANCISCO, Antonio ; CARLA DE MACEDO, Dayana: Dimensionality Reduction in Gene Expression Data Sets. In: *IEEE Access* 7 (2019), S. 61136–61144

[THAA22] TARAWNEH, Ahmad ; HASSANAT, Ahmad ; ALTARAWNEH, Ghada ; ALMUHAIMEED, Abdullah: Stop Oversampling for Class Imbalance Learning: A Review. In: *IEEE Access* 10 (2022), 01, S. 1–1

[Tib96] TIBSHIRANI, Robert: Regression Shrinkage and Selection via the Lasso. In: *Journal of the Royal Statistical Society. Series B (Methodological)* 58 (1996), Nr. 1, S. 267–288

[Vap97] VAPNIK, Vladimir N.: The Support Vector method. In: GERSTNER, Wulfram (Hrsg.) ; GERMOND, Alain (Hrsg.) ; HASLER, Martin (Hrsg.) ; NICOUD, Jean-Daniel (Hrsg.): *Artificial Neural Networks — ICANN'97*. Berlin, Heidelberg : Springer Berlin Heidelberg, 1997, S. 261–271

[Wat25] WATANABE, Shuhei: *Tree-Structured Parzen Estimator: Understanding Its Algorithm Components and Their Roles for Better Empirical Performance*. https://arxiv.org/abs/2304.11127. Version: 2025

[WBKW23] WATSON, David S. ; BLESCH, Kristin ; KAPAR, Jan ; WRIGHT, Marvin N.: Adversarial Random Forests for Density Estimation and Generative Modeling. In: RUIZ, Francisco (Hrsg.) ; DY, Jennifer (Hrsg.) ; MEENT, Jan-Willem van d. (Hrsg.): *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics* Bd. 206, PMLR, 25–27 Apr 2023 (Proceedings of Machine Learning Research), S. 5357–5375

[WCN23a] WANG, Alex X. ; CHUKOVA, Stefanka ; NGUYEN, Binh: Ensemble k-Nearest Neighbors based on Centroid Displacement. In: *Information Sciences* 629 (2023), 02. http://dx.doi.org/10.1016/j.ins.2023.02.004. – DOI 10.1016/j.ins.2023.02.004

[WCN23b] WANG, Alex X. ; CHUKOVA, Stefanka ; NGUYEN, Binh: Synthetic minority oversampling using edited displacement-based k -nearest neighbors. In: *Applied Soft Computing* 148 (2023), 10, S. 110895. http://dx.doi.org/10.1016/j.asoc.2023.110895. – DOI 10.1016/j.asoc.2023.110895

[WH22] WEN, Hongwei ; HANG, Hanyuan: Random Forest Density Estimation. In: CHAUDHURI, Kamalika (Hrsg.) ; JEGELKA, Stefanie (Hrsg.) ; SONG, Le (Hrsg.) ; SZEPESVARI, Csaba (Hrsg.) ; NIU, Gang (Hrsg.) ; SABATO, Sivan (Hrsg.): *Proceedings of the 39th International Conference on Machine Learning* Bd. 162, PMLR, 17–23 Jul 2022 (Proceedings of Machine Learning Research), S. 23701–23722

[WLD+25] WANG, Alex X. ; LE, Minh Q. ; DUONG, Huu-Thanh ; VAN, Bay N. ; NGUYEN, Binh P.: CTVAE: Contrastive Tabular Variational Autoencoder for Imbalance Data. In: *Knowledge and Information Systems* 67 (2025), Nr. 6, S. 5335–5354

[ZE01] ZADROZNY, Bianca ; ELKAN, Charles: Obtaining Calibrated Probability Estimates from Decision Trees and Naive Bayesian Classifiers. In: *Proceedings of the 18th International Conference on Machine Learning (ICML)*, Morgan Kaufmann Publishers Inc., 2001, S. 609–616

[ZE02] ZADROZNY, Bianca ; ELKAN, Charles: Transforming Classifier Scores into Accurate Multiclass Probability Estimates. In: *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA : ACM, 2002 (KDD '02), S. 694–699